红色警戒——战略节点攻防与撤退调度规划

摘　　要

本文围绕苏军与盟军在 436 节点、603 通路战场网络上的攻防对抗，建立部署、补给与攻防转移三类优化模型。先对附件做几何核验与图结构分析，确认坐标单位为千米，并识别出 8 条跨阵营前线咽喉边与苏军 23 节点孤立后方分量，为建模提供结构依据。

针对问题一，构建覆盖型广义指派整数规划模型，以加权火力覆盖—生存—指挥通信综合效能为目标，用分支定界法求解双方部署与指挥节点选址。苏军最优指挥节点为 136（综合效能 0.7688），盟军为 384（综合效能 0.7732），求解器 MIPGap 均为 0，证明全局最优；权重 ±20% 扰动及直射距离、防空半径、空中上限的参数扫描下指挥节点翻转数为 0，方案稳健。

针对问题二，建立运输、护航、破交损失、人力四类成本分层的补给网络优化模型，以 McCormick 线性化求解 0-1 护航规划，并用 Stackelberg 博弈刻画破交—护航对抗。基线情形下苏军后勤总成本 109614 元/日、盟军 145773 元/日，破交损失占比最高（苏军 68.8%、盟军 67.1%）；对抗最坏情形下成本分别上升 50.83% 与 48.94%（含人力 TC_{total} 主口径）；蒙特卡洛仿真验证解析期望损失，偏差仅 0.72% 与 1.54%。

针对问题三，以贪心构造加 Lanchester 蒙特卡洛评估设计苏军进攻方案，主攻方向为 55→372 边，7 节点夺控序列平均攻方剩余兵力比 0.8862、各步胜率不低于 0.98；盟军战略转移以人员为决策量纲，在通信畅通与通信中断两情景下整体人员生存概率分别为 0.7956 与 0.6730，差 0.1226，量化揭示了通信的生存价值。

关键词：整数规划；Stackelberg 博弈；Lanchester 平方律；蒙特卡洛仿真；灵敏度分析

一、 问题重述

在《红色警戒》的平行世界设定中，苏军与盟军在一张由 436 个作战节点与 603 条通路构成的战场网络上展开争夺 [1]。双方只能在本方颜色相同的初始区域内部署兵力，每个节点带有 0 到 1 之间的防御强度。

双方兵力与装备如下：苏军有动员兵 125 万人、侦察机 500 架、天启重型坦克 180 辆、犀牛主战坦克 300 辆、猎犬轻型战车 420 辆、V3 远程火箭车 7000 辆；盟军有动员兵 100 万人、侦察机 300 架、光棱重型坦克 340 辆、幻影中型坦克 570 辆、灰熊轻型坦克 800 辆、光棱自行火炮 14000 辆。每方可设置 10 个防空炮阵地据点，空中单位（战略空袭单位）不宜部署过多。装备的重量、速度、行程、射程等参数由附件二给出。需要研究三个问题。

问题一：综合考虑节点防御强度、沿路网的机动距离、武器射程（坦克直射距离、自行火炮最大射程）与防空阵地覆盖半径，确定双方各类型作战单位（动员兵、轻/中/重型坦克、自行火炮、防空炮阵地）的部署节点与数量，并为双方各选定一个最优前线指挥节点（可驻扎任一己方节点）与 3 个备选指挥节点，兼顾指挥通信距离、节点隐蔽性与抗毁能力。

问题二：基于问题一的部署结果，为双方分别建立维修包（备件/工具）、弹药与能源（燃油/电池）三类补给的分配与供应优化模型，充分考虑敌方破交与拦截策略，预留规避或护航措施；计算完全不进行补给的消耗模式下维持前线作战所需的总后勤工人数与运输车辆总数；以表格或图形给出最优补给方案，包括配送路径、发车周期、每车装载量等信息。

问题三：设定苏军为进攻方、盟军为防守方。其一，设计苏军进攻计划，包括主攻方向、优先攻占的节点序列与各突击梯队（坦克/步兵/火炮配比）的兵力编成；其二，盟军需收缩至后方安全节点 352、144、149，分别给出通信畅通时的整体转移路线与兵力收拢计划（含殿后部队），以及通信中断时各部队按独立预案分散转移的方案，并对比两种情况下盟军部队的整体生存概率差异。

题目涉及若干军事术语，且两个附件各有特定内容，为便于阅读，先给出名词解释与附件说明，见表 1。

表 1　名词解释与附件说明

名词/附件	说明
破交/拦截	攻击方对敌方补给线实施打击以截获物资的作战行动，本文以路径拦截概率度量
前线咽喉边	连接两个阵营区域的跨阵营通路，是地面兵力越界的唯一通道
坦克直射距离	坦克炮直瞄射击的有效距离，附件二未提供，由假设 A-02 补足
防空覆盖半径	防空炮阵地对空中威胁的防护范围，覆盖内部署单位生存率提升，由假设 A-03 补足
梯队编成	进攻部队按坦克、步兵、火炮等兵种比例编组的作战单元
殿后部队	转移行动中留守迟滞敌方追击、掩护主力撤退的部队
消耗模式	完全不进行补给、仅靠初始携行物资维持作战的后勤模式
附件一	作战节点与通路数据：436 个节点（编号、坐标、阵营、防御强度）与 603 条通路（端点、长度 km）
附件二	红警装备参数表：双方 8 型装备的重量、武器、速度、行程、射程等参数

二、　问题分析

（一）　兵力部署与指挥节点选址的结构特征分析

问题一要求同时决定六类单位在数百个节点上的部署数量与指挥节点位置，属于覆盖型广义指派问题，是 NP-hard 的组合优化 [3, 4]。核心难点有三：一是决策变量为整数且规模大（单位类型 × 己方节点数达数千）；二是火力覆盖存在重复覆盖时的边际递减效应，直接按覆盖次数求和会高估效能；三是指挥节点选址与部署方案相互耦合，同一节点在不同部署下的优劣可能不同。拟采用线性化手段将模型化为纯线性混合整数规划（MIP），用分支定界法证明全局最优；指挥节点采用多准则评分生成候选、逐候选重解部署 MIP 的枚举策略。附件数据的图结构（咽喉边、孤立分量）将直接转化为模型约束，详见第 一 节。

（二）　补给线对抗下的成本优化思路

问题二的补给需求由问题一的部署结果派生，形成两问之间的数据接口。难点在于护航决策与破交损失的双线性耦合：启用护航能降低损失，但护航本身有成本，二者乘积项使目标非线性。拟将后勤成本分解为运输、护航、破交期望损失、人力四类独立分项，路径集预枚举后用 0-1 规划建模，双线性项以 McCormick 线性化精确处理 [7]。敌方的破交策略构成对抗：我方先承诺路径与护航组合，敌

方观察后将有限破交预算投向收益最大的路径，这是 Stackelberg 博弈结构 [10]，拟以敌方最优响应下的最坏情形成本量化对抗代价。

（三） 进攻序列与两情景转移的时序与随机性

问题三包含两个方向。苏军进攻是多阶段时序决策，战斗损耗使状态转移非线性：动态规划的状态空间随节点数指数爆炸，时序展开 MIP 的松弛很弱，故拟采用贪心构造加 Lanchester 平方律蒙特卡洛评估的混合策略 [8, 9]，并讨论替代算法的取舍理由。盟军转移的决策信息结构不同：通信畅通时是单一决策者的全局优化（容量约束分配加最短路），通信中断时退化为各部队的分散独立预案。两情景的生存概率无法解析求得，拟用逐边伯努利拦截的蒙特卡洛仿真各 2000 次估计 [11]，并对比均值差与分布形态。

三、 模型假设

假设 1. 破交拦截概率与护航效应。边暴露度按公式 $\mathrm{risk}_e = \min(0.35,\ 0.05 + 0.10\,\mathbb{I}_{\text{跨阵营}} + 0.02\,\min(\deg(a), \deg(b)))$ 计算，护航使该边拦截损失降低 50%。题目未提供破交数据，公式以边度数与跨阵营属性代理暴露程度，属显式未知参数，参数化形态见表 2 的 A-01。

假设 2. 武器射程与防空覆盖。坦克直射距离取轻型 1.5 km、中/重型 2.0 km，防空炮阵地覆盖半径 $R_{aa} = 3.0$ km，覆盖内部署单位生存率提升 20%。附件二仅给出自行火炮最大射程（V3 为 16.8 km、光棱火炮为 13.0 km），坦克直射距离与防空半径属题目数据缺口，以假设补足并参数化（表 2 的 A-02、A-03）。

假设 3. 后勤标准参数。运输车容量 10 吨/车、时速 40 km/h、每车配 2 名司机；单位成本为运输 20 元/km· 车、护航 500 元/车次、破交损失货值 1000 元/吨、人力 300 元/人 · 日；日消耗率为弹药 0.8 吨/百辆、燃油 0.6 吨/百辆、维修包 0.2 吨/百辆，动员兵按万人折算 0.5 吨/万人（三类合计）。

假设 4. 战斗损耗规律。地面交战服从 Lanchester 平方律，单位火力系数取重型 1.0、中型 0.7、轻型 0.3、火炮 0.8、动员兵 0.2、空中 0.5，防御强度以乘性因子 $(1 + 0.5\,d_n)$ 修正毁伤率；毁伤过程叠加对数正态乘性噪声（$\sigma = 0.15$，截断于 $[0.5, 1.5]$）。

假设 5. 守军强度标定。夺控战斗的守军总量按攻方单步投入的 $k_{def} = 0.5 + 0.3\,d$ 倍标定（d 为节点防御强度），兵种构成为动员兵 45%、中型 25%、重型 18%、火炮 12%。该线性标定是问题三胜率的关键敏感参数，其局限与影响在第 四 节单独讨论。

假设 6. 距离度量。火力覆盖判定采用欧氏距离（603 条边的链路长度与坐标欧氏距离比值恒为 1.0000，坐标单位即 km，该简化经几何核验成立）；指挥通信距离采用路网最短路距离（Dijkstra 算法）。

假设 7. 苏军孤立后方就地防御。图连通性实测表明战场含 2 个连通分量：主分量 413 节点与苏军后方孤立分量 23 节点。孤立分量与主战场不连通，其内部队无法经路网支援前线，按就地防御部署。

对四项争议假设以显式未知参数的规范形态介入模型，参数登记表见表 2，每项含基线值、取值范围（稳定性区间或最坏情形）与模型角色；对应的扫描数据与图见第 六 节。

表 2　显式未知参数登记表（A-01 至 A-04）

参数	基线值	取值范围与稳定性	模型角色
A-01 破交拦截概率	risk_e 公式（假设 1）	对抗最坏情形（含人力 TC_{total} 主口径）：TC 苏军 +50.83%（109613.7→165334.7 元/日）、盟军 +48.94%（145773.0→217114.0 元/日）	内生未知参数：静态 risk_e 基线为主模型输入，Stackelberg 最坏情形为鲁棒性成本量化
A-02 坦克直射距离	轻型 1.5 km、中/重型 2.0 km（代表值 $d = 2.0$ km）	$d \in [1.0, 3.0]$ km 五点扫描，指挥节点翻转数为 0，稳定性区间 $[1.0, 3.0]$ km	外生未知参数（题目数据缺口），经覆盖系数进入 Q1
A-03 防空覆盖半径	$R_{aa} = 3.0$ km	$r \in [2.0, 4.0]$ km 五点扫描，防空节点集 Jaccard 相似度恒为 1.0，稳定性区间 $[2.0, 4.0]$ km	外生未知参数（题目数据缺口），经生存加成进入 Q1
A-04 空中部署上限	$\alpha = 10\%$（苏 50 架、盟 30 架）	$\alpha \in [5\%, 20\%]$ 四点扫描，指挥节点翻转数为 0，Q2 成本最大传播影响 0.419%	题目定性要求（“不宜部署过多”）的定量化

A-07 粒度选择验证注记：动员兵以万人为决策粒度（125 万记为 125 单位）属建模粒度选择而非未知参数，目的是避免 10^6 值域造成的整数规划病态。以 0.5 万人更细粒度补做验证算例：双方指挥节点均不翻转，装备部署结构 Jaccard 相似度为 1.0，综合效能 φ 最大变化 0.0001，部署结构对粒度稳健。

Q2 口径说明：问题二 MIP 内部目标值（苏军 81690.4、盟军 107511.6 元/日）与重算总成本（109613.7、145773.0 元/日）的差异源于护航成本按固定份额线性化的近似，属计算保真度问题而非未知参数。本文一律以重算可行解的真实总成本 TC 为准，两个口径的差距（相对不含人力成本口径为 7.18% 与 8.56%）在误差分析中量化。

四、 符号说明

符号与代码变量名的对照关系在附录代码文件头部以注释形式给出，全部符号见表 3。

表 3　符号说明

符号	说明	单位
$x_{c,u,n}$	阵营 c 单位类型 u 在节点 n 的部署数量（动员兵以万人计）	辆/门/万人
z_c	阵营 c 的前线指挥节点	节点编号
$\mathrm{flow}_{c,k,r}$	阵营 c 物资 k 经路径 r 的日流量	吨/日
e_r	路径 r 是否启用护航（代码变量 z[r]）	0-1
$f_r,\ l_r$	路径 r 发车频次；每车装载量	次/日；吨/车
φ_c	阵营 c 部署综合效能	无量纲
$F_c,\ S_c,\ D_c$	火力覆盖效能；生存效能；指挥通信距离（部署加权平均路网距离）	无量纲；无量纲；km
TC_c	阵营 c 后勤总成本	元/日
P_{surv}	盟军人员生存概率（人员加权）	0-1
$N_{c,u}$	阵营 c 单位类型 u 的总兵力	辆/门/万人
$d_n,\ \mathrm{camp}(n)$	节点 n 防御强度；所属阵营	0-1；–
L_e	通路 e 长度	km
R_u	单位 u 武器射程（火炮取最大射程，坦克取直射距离假设）	km
$\mathrm{cov}_{u,n,m}$	单位 u 在节点 n 能否覆盖敌节点 m（射程内为 1）	0-1
$v_m,\ \mathrm{surv}_u$	敌节点 m 目标价值（取 d_m）；单位 u 基础生存系数	0-1
$R_{aa},\ \mathrm{risk}_e$	防空炮阵地覆盖半径；通路 e 破交拦截概率	km；0-1
$\mathrm{Cap},\ \mathrm{Demand}_{c,k,n}$	运输车容量；阵营 c 节点 n 物资 k 日需求	吨；吨/日
w_1, w_2, w_3	问题一目标权重（火力/生存/通信，取 0.5/0.3/0.2）	–
λ_u	Lanchester 单位火力系数（代码变量 FIREPOWER）	–
α	空中单位部署上限比例（A-04，基线 10%）	–
$\mathcal{T}$	盟军战略转移目标节点集 $\{352, 144, 149\}$	节点编号
$C_{trans}, C_{escort}, C_{loss}, C_{labor}$	运输、护航、破交损失、人力四类成本	元/日
$W_c,\ V_c$	阵营 c 总后勤工人数；运输车辆总数	人；辆
D_{max}	主分量路网直径估计（归一化尺度，43.335 km）	km

算法与求解方法术语见表 4。

表 4　算法术语说明

术语	说明
混合整数规划（MIP）	决策变量含整数约束的线性优化模型，本文问题一、问题二的求解载体
分支定界法	MIP 精确求解算法：松弛定界加分支剪枝，可给出全局最优证明（MIPGap=0）
McCormick 线性化	双线性项 xy 的凸包线性化技术，0-1 变量参与时在整数解处精确成立
Stackelberg 博弈	领导者—追随者序贯博弈，本文刻画护航承诺与破交最优响应的对抗
纳什均衡（鞍点）	零和博弈中双方均无单方偏离动机的策略组合，即 minimax 值
Dijkstra 算法	非负权图单源最短路算法，用于指挥通信距离与转移路线计算
Lanchester 平方律	战斗损耗微分方程模型：双方损耗率与对方火力总量成正比
贪心构造	每步取局部最优的序列构造启发式，无全局最优保证
蒙特卡洛（MC）仿真	随机抽样仿真方法，本文用于战斗评估、生存概率与损失验证

五、　模型的建立与求解

（一）　战场网络结构核验与整体技术路线

1　数据质量核验与几何标定

附件一节点表 436 行、通路表 603 行，与题目描述一致；两表均无缺失值，通路端点全部落在节点集合内（坏边数为 0），无重复边。防御强度介于 0.01 与 1.0 之间，均值 0.526、标准差 0.284，无 3σ 异常点。链路长度介于 0.012 与 19.55 km，均值 0.90 km，6 个 3σ 大值均为跨海域或长通道边，属真实地理分布，予以保留。横坐标有 4 个 3σ 异常点（东侧远端节点，$X \approx 231$–240），同为真实分布而非脏数据。附件二装备参数表含空行噪声，清洗后得双方 8 型装备的完整参数。

几何标定是本题建模的地基。对全部 603 条边计算链路长度与端点坐标欧氏距离的比值，结果恒为 1.0000（标准差为 0），由此确认坐标单位即 km，战场尺度约 42×26 km。这一结论使火力覆盖判定可以直接使用欧氏距离，免去全图路网距离的预计算。

2　图结构特征及其建模含义

广度优先搜索测得战场含 2 个连通分量：主分量 413 节点，以及苏军后方孤立分量 23 节点（全为苏军节点，与主战场不连通）。跨阵营边仅 8 条，构成前线咽喉。苏军居北（纵坐标均值 +1.87），盟军居南（均值 −9.91）。盟军三个转移目标节点 352、144、149 均在主分量内、均为盟军节点、位于战场最南端，防御强度分别为 0.75、0.09、0.97。

这两个结构特征直接转化为建模决策：孤立分量内部队无法越境支援，只能就地防御（假设 7）；8 条咽喉边是地面火力必须封锁的对象，在问题一中以覆盖约束形式出现，在问题二中则是破交风险的集中来源。整体技术路线见图 1：数据层完成图结构与几何核验后，问题一以 MIP 求解部署与选址，问题二在部署派生的需求上求解补给网络并以 Stackelberg 最坏情形量化对抗，问题三分别用贪心加仿真、分配加仿真处理进攻与转移，验证层覆盖灵敏度扫描、策略临界点、多算例与蒙特卡洛模型验证。

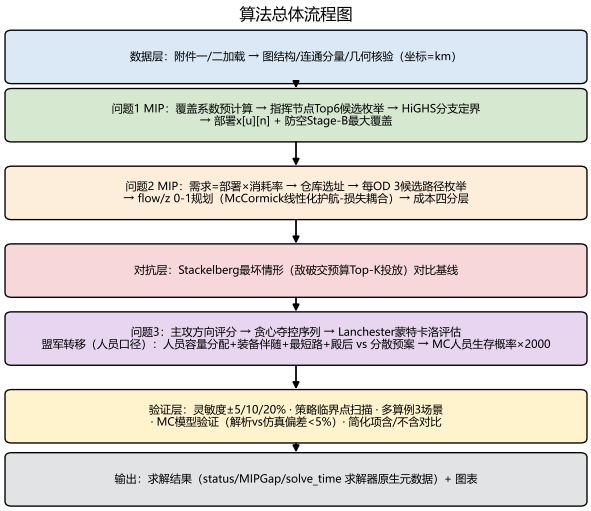


图 1　算法总体流程图

图 1 自顶向下展示了六个层次。数据层（蓝色）完成附件加载、图结构构建与坐标单位核验；问题一层（绿色）先预计算覆盖系数，再枚举指挥节点候选并以分支定界求解部署 MIP，防空配置作为第二阶段的最大覆盖问题解耦求解；问题二层（橙色）由部署量乘以消耗率生成需求，仓库选址后枚举每对起讫点的候选路径，用 0-1 规划与 McCormick 线性化求解；对抗层（红色）将敌方破交预算的最优响应对比基线；问题三层（紫色）分进攻与转移两条支线；验证层（黄色）汇总全部稳健性检验。各层输出统一汇入中间结果文件，其中求解状态、MIPGap 与求解时间均为求解器原生元数据透传，避免人工转述误差。

（二） 双方兵力部署与指挥节点选址模型（问题一）

1 部署效能指标与 MIP 模型建立

记己方节点集为 $\mathcal{O}_c$、敌方节点集为 $\mathcal{E}_c$。决策变量 $x_{u,n} \in \mathbb{Z}_+$ 表示单位类型 u 在节点 n 的部署数量。综合效能由火力覆盖、生存、指挥通信距离三项构成：

$$\max\ \varphi_c = w_1 \frac{F_c}{F_{max}} + w_2 \frac{S_c}{S_{max}} - w_3 \frac{D_c}{D_{max}}, \qquad (w_1, w_2, w_3) = (0.5, 0.3, 0.2) \quad (1)$$

火力覆盖效能按目标价值加权求和。定义覆盖系数 $\mathrm{cov}_{u,n,m} = \mathbb{I}(\|p_n - p_m\|_2 \le R_u)$，即单位 u 在节点 n 能否以射程 R_u 覆盖敌节点 m；火炮射程取阵营最大射程（苏军 V3 为 16.8 km、盟军光棱火炮为 13.0 km），坦克取直射距离假设 A-02，动员兵与空中单位不计火力覆盖贡献。若同一敌节点 m 被多支部队覆盖，简单求和会把重复覆盖计为多份战果，高估效能。为此引入有效覆盖数 $y_m = \min\{\sum_{u,n} \mathrm{cov}_{u,n,m}\, x_{u,n},\ 3\}$，即单目标有效覆盖次数上限为 3，超出的边际贡献计 0：

$$F_c = \sum_{m \in \mathcal{E}_c} v_m\, y_m, \qquad v_m = d_m, \qquad F_{max} = 3 \sum_{m \in \mathcal{E}_c} d_m \quad (2)$$

其中归一化常数 F_{max} 取全部敌节点均被有效覆盖 3 次时的理论上限。y_m 与部署量的耦合由线性约束 $y_m \le \sum_{u,n} \mathrm{cov}_{u,n,m} x_{u,n}$ 与上界 $y_m \le 3$ 刻画，最大化目标会自然将 y_m 推到实际覆盖数与 3 的较小者，无需分段函数。

生存效能按节点防御强度与单位生存系数加权：

$$S_c = \sum_{u,n} d_n\, \mathrm{surv}_u\, x_{u,n}, \qquad S_{max} = \sum_{u} N_{c,u}\, \mathrm{surv}_u \quad (3)$$

其中 surv_u 按装甲与隐蔽性估计：重型 0.85、中型 0.7、轻型 0.6、火炮 0.5、动员兵 0.4、空中 0.3。S_{max} 为全部兵力集中于最高防御节点时的理论上界。防空加成效能 S_{bonus} 在第二阶段计算（见第 5.2.3 小节），不进入部署 MIP 的目标。

指挥通信距离为部署量加权的路网平均距离。设候选指挥节点为 z，$\mathrm{dist}(n, z)$ 为 Dijkstra 路网最短路距离 [5]：

$$D_c = \frac{\sum_{u,n} x_{u,n}\, \mathrm{dist}(n, z)}{\sum_u N_{c,u}}, \qquad D_{max} = 43.335 \text{ km} \quad (4)$$

D_{max} 为主分量路网直径的采样估计（40 个随机源点的最短路最大值），用于把距离项归一到 $[0, 1]$ 尺度。三项归一化后量纲统一为无量纲，加权求和才有意义；权重 $(0.5, 0.3, 0.2)$ 按题目语义排序（火力优先、生存次之、通信再次），并在

第 六 节做 ±20% 扰动检验。

把式 (2)–(4) 代入式 (1)，目标对决策变量是线性的，$x_{u,n}$ 的目标系数为

$$\frac{w_2\, d_n\, \mathrm{surv}_u}{S_{max}} - \frac{w_3\, \mathrm{dist}(n,z)}{N_{total}\, D_{max}} \tag{5}$$

y_m 的系数为 $w_1 d_m / F_{max}$。约束条件如下：

$$\text{(C1 仅己方节点)} \qquad x_{u,n} = 0, \quad n \notin \mathcal{O}_c \tag{6}$$

$$\text{(C2 兵力守恒)} \qquad \sum_{n} x_{u,n} = N_{c,u}, \quad u \neq \text{空中} \tag{7}$$

$$\text{(C4 空中上限)} \qquad \sum_{n} x_{\text{空中},n} \le \alpha\, N_{c,\text{空中}}, \quad \alpha = 10\% \tag{8}$$

$$\text{(C5 咽喉覆盖)} \qquad \sum_{u,n:(n,m_e)\in \mathrm{cov}_u} x_{u,n} \ge 1, \quad \forall \text{ 咽喉边敌方端点 } m_e \tag{9}$$

$$\text{(P2 节点容量)} \qquad 0 \le x_{u,n} \le \lfloor 0.15\, N_{c,u} \rfloor \tag{10}$$

约束 C2 对地面五类单位取等式，即全部地面兵力必须部署；空中单位按题目“不宜部署过多”取上限不等式（A-04），其余留作预备队。C5 要求每条跨阵营边的敌方端点至少被 1 个火力单位覆盖，保证咽喉处于火力封锁之下。P2 将单节点部署限制在该类型总量的 15% 以内，防止解退化为把所有兵力堆在个别高防御节点。防空炮阵地独立于兵力体系（不占用 C2 的兵力守恒），在第二阶段单独选址。

2　指挥节点候选生成与求解算法选择

指挥节点须兼顾通信、隐蔽与抗毁三方面。先以多准则评分从己方主分量节点中选出 6 个候选：以防御强度加权的己方部署重心 $(\bar{x}, \bar{y})$ 为参考，

$$\begin{aligned}\mathrm{score}(n) = 0.4\Big(1 - \frac{\|p_n - (\bar{x},\bar{y})\|_2}{\max_{n'} \|p_{n'} - (\bar{x},\bar{y})\|_2}\Big) \\ + 0.3\Big(1 - 0.5\, d_n - 0.5\, \frac{\min(\deg(n), 6)}{6}\Big) + 0.3\, d_n\end{aligned} \tag{11}$$

三项分别对应靠近重心的通信便利、低防御暴露与低度数的隐蔽性、防御强度代表的抗毁能力。对每个候选 z 重新计算 $\mathrm{dist}(n,z)$ 并求解一次部署 MIP，取 φ 最大的候选为主选指挥节点，其余候选按评分取 3 个备选。候选数取 6 是求解时间与覆盖度的折中：每候选求解约 0.2–0.4 s，6 候选总耗时在 3 s 以内。

求解算法对比三种候选 [2]。其一，分支定界法（pulp 调用 CBC/HiGHS 求

解器）：模型经式 (5) 线性化后为纯线性 MIP，分支定界能给出全局最优证明，本规模（苏军约 1600、盟军约 1000 个整数变量）下秒级求解。其二，贪心启发式：按节点边际贡献逐个放置兵力，实现简单但无最优性保证，且覆盖耦合下局部最优与全局差距不可控。其三，连续松弛加取整：求解快但取整后可能违反兵力守恒与咽喉覆盖约束，修复取整解的代价不低于直接求解原问题。选择分支定界法，理由是模型结构允许精确求解且规模在开源求解器能力之内，贪心与松弛取整仅在求解器超时的情形下作为后备。

3 防空配置的两阶段解耦

防空炮阵地选址在部署确定后进行（Stage B）：10 个据点覆盖已部署兵力，最大化覆盖权重和。设部署节点 n 的权重为 $q_n = \sum_u \mathrm{surv}_u\, x_{u,n}$，决策变量 $a_p \in \{0,1\}$ 表示节点 p 是否设防空据点，$c_n \in [0,1]$ 表示部署节点 n 是否被覆盖：

$$\max \sum_{n} q_n c_n, \qquad \text{s.t.} \ \sum_{p} a_p = 10, \quad c_n \le \sum_{p:\|p_n - p_p\|_2 \le R_{aa}} a_p \tag{12}$$

覆盖内部署单位的生存率提升 20%，对应加成 $S_{bonus} = 0.2 \sum_{n\in\text{覆盖}} d_n \sum_u \mathrm{surv}_u x_{u,n}$。两阶段解耦的合理性在于：防空据点不占用兵力守恒约束，且生存加成只改变效能值、不改变部署 MIP 的可行域；联合求解虽理论更优，但会把模型规模扩大一倍以上，而第 六 节的扫描表明防空布局对半径参数高度稳健，解耦的次优代价可忽略。

4 求解结果与最优性验证

双方 MIP 均由分支定界法证明全局最优：苏军 status=optimal、MIPGap=0、求解时间 0.225 s；盟军 status=optimal、MIPGap=0、求解时间 0.138 s。MIPGap 为 0 意味着求解器已证明不存在更优整数解，这是精确算法相对启发式的本质优势。结果汇总见表 5。

表 5　问题一求解结果汇总

阵营	φ	F	S_{base}	S_{bonus}	D/km	指挥节点	部署节点数
苏军	0.7688	276.24	4017.37	803.47	2.279	136	10
盟军	0.7732	411.09	7796.90	1559.38	1.872	384	10

苏军主选指挥节点为 136，备选 187、137、358；盟军主选 384，备选 411、255、214。一个值得注意的现象是：盟军多准则评分第一的节点是 411（评分

0.7665），而 384 的评分仅列第三（0.7475），但逐候选重解 MIP 后 384 的综合效能 $\varphi = 0.7732$ 最高。评分只反映指挥节点自身的通信、隐蔽、抗毁属性，φ 还包含该节点作为指挥中枢时全部署的通信距离成本，两者的排序差异说明指挥节点选址必须与部署方案耦合评估，单靠多准则评分预排序会选错。

部署方案呈现“前沿多点集中”结构，双方均使用 10 个部署节点，见表 6。苏军火炮（V3）依托 16.8 km 射程在纵深节点 136、142、344、358、386、413、137 各部署 1050 门，坦克与动员兵前推至 28、86 等靠近咽喉的节点；盟军火炮数量是苏军的两倍（14000 对 7000），在 134、140、150、247、255、384 各部署 2100 门，但光棱火炮射程只有 13.0 km，必须更靠前部署才能覆盖目标。空中单位按上限约束少量集中部署（苏军 50 架于 386、盟军 30 架于 384），符合题目“不宜部署过多”的定性要求。

表 6　问题一双方主要部署节点与数量（节选）

阵营	节点	动员兵 (万人)	轻型 (辆)	中型 (辆)	重型 (辆)	火炮 (门)	空中 (架)
苏军	136	18	–	–	–	1050	–
苏军	28	–	63	45	27	–	–
苏军	386	18	63	45	27	1050	50
苏军	86	–	–	30	18	3	–
盟军	384	15	120	85	51	2100	30
盟军	134	15	120	85	51	2100	–
盟军	218	10	80	60	34	1391	–
盟军	255	15	120	85	51	2100	–

图 2 展示了双方部署的空间分布。黑色圆圈大小表示节点部署总量，金色五角星为主选指挥节点，浅黄五角星为备选，绿色三角为防空据点。苏军部署沿南侧前线与纵深火炮阵地两层展开，指挥节点 136 位于前线群中央；盟军部署更贴近前线，指挥节点 384 与各部署节点的距离普遍更短（$D = 1.872$ km 对苏军 2.279 km）。两军的防空据点均与部署节点重合或紧邻，这是 Stage B 最大覆盖模型的直接结果。

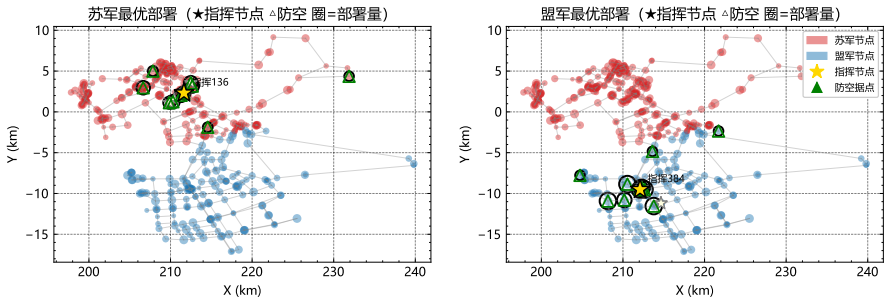


图 2　问题一双方最优部署分布（左：苏军，右：盟军；圆圈大小表示部署总量，★指挥节点，△防空据点）

图 3 以堆叠柱状图对比双方部署构成。盟军各型装备的部署数量整体高于苏军（火炮 14000 对 7000、轻型 800 对 420），空中单位受上限约束分别只部署 30 与 50 架。动员兵按万人粒度部署，苏军 125 个单位、盟军 100 个单位全部落位。该图直观呈现了双方的兵力禀赋差异：盟军数量占优、苏军坦克质量占优（重型 180 辆天启对盟军 340 辆光棱中重型合计，苏军单车火力系数更高），这一非对称性为问题三的进攻想定提供了背景。

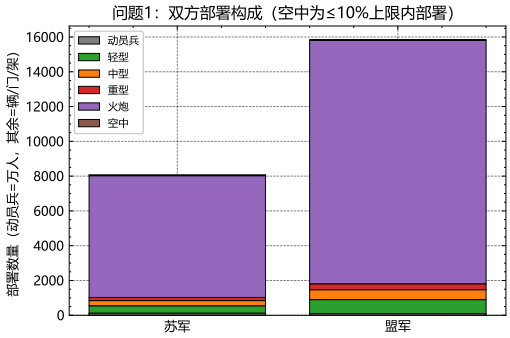


图 3　问题一双方部署构成（空中为不超过 10% 上限下的部署量）

5　结果验证与量级合理性

问题一为确定性 MIP，无随机参数，蒙特卡洛模型验证不适用；其验证维度为最优性证明（MIPGap=0）与参数稳健性扫描（第 六 节），两者相互独立。活跃约束方面，六类兵力守恒约束全部紧（slack 为 0，即各类型部署合计恰等于兵力总量），全部咽喉覆盖约束紧（每条咽喉被不少于 1 个火力单位覆盖），说明兵力没有冗余、咽喉封锁没有富余，解处在资源充分利用的边界上。

结果量级合理性核查如下。部署节点数 10 个，占己方主分量节点（苏军约 245、盟军约 168）的 4%–6%，符合“要点防御”的军事常识；指挥通信距离 2.279 km 与 1.872 km 远小于战场直径 43.3 km，指挥半径合理；综合效能 0.7688 与 0.7732 介于 0 与 1 之间且双方接近，反映非对称兵力下攻防大体均衡，与题目“形成相对稳定的战场动态平衡”的设定一致。

（三） 三类补给分配与供应优化模型（问题二）

1 需求生成与仓库选址

补给需求由问题一部署派生。对部署节点 n，记装备数量为 E_n（辆/门，不含动员兵）、动员兵为 I_n（万人），物资 $k \in \{$ 弹药，燃油，维修包 $\}$ 的日需求为

$$\mathrm{Demand}_{c,k,n} = \rho_k^{eq} \frac{E_n}{100} + \rho_k^{inf} I_n \tag{13}$$

其中消耗率（假设 3）为 $\rho^{eq} = (0.8, 0.6, 0.2)$ 吨/百辆、$\rho^{inf} = (0.3, 0.15, 0.05)$ 吨/万人，动员兵三类合计 0.5 吨/万人。苏军总需求 189.7 吨/日，盟军 301.84 吨/日。仓库选址取己方主分量内纵深与防御兼优的 3 个节点：苏军纵深向北（纵坐标大），盟军纵深向南（纵坐标小），评分 0.6× 纵深归一值 +0.4× 防御强度。求得苏军仓库为 275、119、41，盟军为 296、318、352。

2 成本四分层与补给网络模型

后勤总成本分解为四类独立分项，各有独立符号、公式与经济含义：

$$C_{trans} = \sum_r \mathrm{trips}_r \, L_r \, c_{km}, \qquad c_{km} = 20 \text{ 元/km·车} \tag{14}$$

$$C_{escort} = \sum_r e_r \, \mathrm{trips}_r \, c_{esc}, \qquad c_{esc} = 500 \text{ 元/车次} \tag{15}$$

$$C_{loss} = \sum_r \mathrm{flow}_r \, P_{int,r} \, (1 - 0.5\, e_r) \, c_{unit}, \qquad c_{unit} = 1000 \text{ 元/吨} \tag{16}$$

$$C_{labor} = W \times c_w, \qquad c_w = 300 \text{ 元/人·日} \tag{17}$$

式 (14) 为车队运行的直接里程成本，与路径长度、发车频次成正比；式 (15) 为武装护航的资源消耗，仅在启用护航的路径发生；式 (16) 为敌方破交的期望物资损失，是对手行动的期望代价，护航以 50% 的削减率降低该项；式 (17) 为后勤工人（司机加装卸）的维持成本。总成本为四项之和：

$$\mathrm{TC} = C_{trans} + C_{escort} + C_{loss} + C_{labor} \tag{18}$$

路径拦截概率由边风险复合而来。设路径 $p = (n_0, n_1, \ldots, n_K)$ 经过的边序列为 $e_1, \ldots, e_K$，各边独立拦截：

$$P_{int}(p) = 1 - \prod_{i=1}^{K} (1 - \mathrm{risk}_{e_i}) \tag{19}$$

该式从“全程安全等于各边均安全”的独立性假设直接推出。决策变量为路径流量 $\mathrm{flow}_r \geq 0$（吨/日，连续）与护航决策 $e_r \in \{0,1\}$。路径集预枚举：对每个“仓库—需求节点”对，用最短路与边禁用扰动法生成 3 条候选路径（最短路、次短路、风险加权最短路，思路源于 Yen 的 k 短路枚举 [6]），按 $L(1+3P_{int})$ 混合排序保留前 3。约束包括需求满足 $\sum_{r\to n}\mathrm{flow}_r \geq \mathrm{Demand}_n$、每节点启用护航的路径不超过 2 条（防止过度分散），以及护航—损失耦合的线性化约束（下述）。

护航项 $e_r \cdot \mathrm{flow}_r$ 是双线性项，引入辅助变量 $w_r = e_r\,\mathrm{flow}_r$ 并以 McCormick 凸包线性化。设流量上界 $M_n = 3\,\mathrm{Demand}_n$：

$$w_r \leq M_n e_r, \quad w_r \leq \mathrm{flow}_r, \quad w_r \geq \mathrm{flow}_r - M_n(1-e_r), \quad w_r \geq 0 \tag{20}$$

由于 $e_r \in \{0,1\}$，式 (20) 在整数解处精确成立（$e_r=1$ 时 $w_r=\mathrm{flow}_r$，$e_r=0$ 时 $w_r=0$），不引入近似。目标中损失项写为 $\mathrm{risk}_r\,c_{unit}(\mathrm{flow}_r - 0.5\,w_r)$，即护航削减一半期望损失。

需要说明计算口径：MIP 内部对护航成本采用固定份额线性化近似（e_r 的系数按需求均分份额的车次数计算），因此 MIP 目标值（苏军 81690.4、盟军 107511.6 元/日）低于真实成本。求解完成后，对 MIP 给出的路径、流量与护航组合按式 (14)–(17) 逐项重算，得到真实总成本 TC。本文一律以重算值为准，差距的量化见第 四 节。

3 破交—护航对抗的 Stackelberg 刻画

敌方破交策略与我方护航决策构成序贯对抗：我方（领导者）先承诺路径、流量与护航组合，敌方（追随者）观察后将有限破交预算投向期望截获收益最大的路径。敌方的最优响应为按“流量 × 货值/护航强度”排序，将预算（5 条路径）投向排名前五的路径，集中打击使这些路径的有效损失上升。最坏情形总成本为

$$\mathrm{TC}_{worst} = \mathrm{TC} + \sum_{r\in\mathcal{H}} \mathrm{flow}_r\,\mathrm{risk}_r\,c_{unit} \times 0.5 \tag{21}$$

其中 $\mathcal{H}$ 为敌方选中的 Top-5 路径集合。

均衡论证按零和博弈的纳什均衡（鞍点）标准给出。其一，单方偏离收益比较：我方在敌方打击集合 $\mathcal{H}$ 固定时偏离当前护航组合，等于放弃 MIP 最优解，总成本不会下降（MIPGap=0 已证明全局最优，任何可行偏离的目标值不小于最优值）；敌方在我方方案固定时偏离 Top-5 打击集合，等于放弃按收益排序的最优响应，截获损失不会增加。双方均无单方偏离动机。其二，帕累托区分：零和博弈中帕累托概念退化为 minimax 值，纳什均衡即鞍点，不存在帕累托改进

空间，此处显式标注。表 7 汇总了偏离分析。

表 7　破交—护航对抗的单方偏离收益比较（零和，苏军口径）

参与方	策略状态	单方偏离的后果
我方（领导者）	当前护航组合（MIP 最优，TC = 109614 元/日）	偏离后成本不降（MIPGap=0，全局最优）
敌方（追随者）	Top-5 打击集合（额外损失 55721 元/日）	偏离后截获损失不增（排序最优响应）

4　求解结果、模型验证与结果分析

双方 MIP 均证明全局最优（status=optimal，MIPGap=0，求解时间 0.391 s 与 0.07 s）。最优补给方案：苏军启用 10 条配送路径，盟军 10 条，全部路径启用护航。表 8 给出方案明细（路径以仓库与需求节点标识）。

表 8　问题二最优补给方案（流量降序，全部路径护航，每车装载 10 吨）

阵营	仓库 → 需求节点	路径长/km	拦截风险	流量/(吨/日)	车次/(次/日)	发车周期/h
苏军	119→386	7.518	0.7362	28.76	2.88	8.34
苏军	119→142	5.489	0.6902	27.96	2.80	8.58
苏军	119→344	8.725	0.8426	27.96	2.80	8.58
苏军	119→358	9.551	0.8567	27.96	2.80	8.58
苏军	119→413	9.753	0.8390	27.96	2.80	8.58
苏军	119→136	8.030	0.8346	25.80	2.58	9.30
苏军	119→137	8.449	0.8191	20.28	2.03	11.84
苏军	119→28	1.568	0.2464	2.16	0.22	111.11
苏军	119→86	15.448	0.9012	0.82	0.08	294.12
苏军	275→283	7.885	0.2464	0.05	0.00	5000.00
盟军	318→384	7.171	0.6742	45.68	4.57	5.25
盟军	318→134	4.449	0.4416	45.20	4.52	5.31
盟军	318→140	10.732	0.6652	45.20	4.52	5.31
盟军	318→150	9.131	0.7361	45.20	4.52	5.31
盟军	318→247	6.935	0.6327	45.20	4.52	5.31
盟军	318→255	7.256	0.7599	45.20	4.52	5.31
盟军	318→218	8.723	0.6152	30.04	3.00	7.99
盟军	318→77	12.566	0.8301	0.05	0.00	5000.00
盟军	318→166	22.756	0.9807	0.05	0.00	5000.00
盟军	318→380	15.574	0.8990	0.05	0.00	5000.00

表 8 中，主力路径的拦截风险集中在 0.6–0.9 区间，因为这些路径必须穿越或贴近前线咽喉区域；小流量路径（边缘哨所补给）风险虽高但损失期望小，发

车周期长达数千小时，属低频补给。成本构成见表 9。

表 9　问题二成本四分层与总成本（元/日）

阵营	C_{trans}	C_{escort}	C_{loss}	C_{labor}	TC（合计）	MIP 内部目标值
苏军	3096.5	9485.0	75432.2	21600.0	109613.7	81690.4
盟军	4664.4	15092.0	97816.6	28200.0	145773.0	107511.6

成本结构中破交损失占绝对主导：苏军 C_{loss} 占 TC 的 68.8%，盟军占 67.1%；护航成本只占 8.7% 与 10.4%，却把主力路径的损失期望削减一半，是典型的“小投入换大损失规避”。人力成本占 19.7% 与 19.3%，运输成本占比最低（2.8% 与 3.2%）。

盟军火炮数量是苏军的两倍（14000 对 7000），后勤总成本反而高出 33.0%（145773 对 109614 元/日）。根因在于射程与部署位置的联动：光棱火炮 13.0 km 的射程短于 V3 的 16.8 km，盟军火炮必须靠前部署才能覆盖目标，部署节点贴近前线使补给路径长且暴露（盟军最长主力路径 10.732 km，苏军 9.753 km）。

虽然盟军主力路径的平均拦截风险（0.6464）低于苏军（0.8026），但盟军单路径流量约为苏军的 1.6 倍（45.2 吨/日对 27.96 吨/日），损失期望 $C_{loss} = \sum \text{flow}_r P_{int,r} c_{unit}$ 由流量主导，最终盟军 C_{loss} 达 97816.6 元/日，高于苏军的 75432.2。数量优势没有转化为后勤优势，这是本题一个反直觉的发现；交叉验证依据是两组结果来自同一模型同一代码、仅输入兵力与装备参数不同，差异只能由部署位置、路径结构与流量规模解释。

图 4 展示了双方补给网络的空间形态。实线为护航路径（全部路径均护航，故图中均为实线），线宽与流量成正比，金色方块为仓库。苏军仓库 119 承担了 9 条路径中的 9 条，呈单枢纽辐射状；盟军仓库 318 同样承担主力。路径在穿越前线区域时明显绕行，以长度换风险，体现了式 (18) 中运输成本与损失成本的权衡。

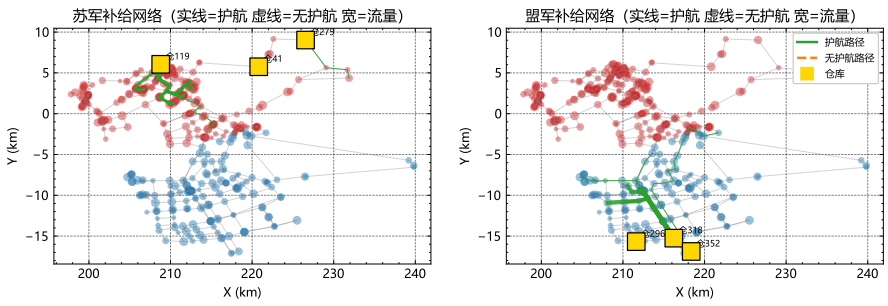


图 4　问题二双方最优补给网络（线宽表示流量，金色方块为仓库）

图 5 以分组柱状图对比双方四类成本。破交损失柱显著高于其余三类，护航

柱约为损失柱的八分之一，直观支持“护航是必需投入”的判断。两军成本谱的形状相似（损失主导、人力次之），盟军整体高出约三分之一，与表 9 一致。

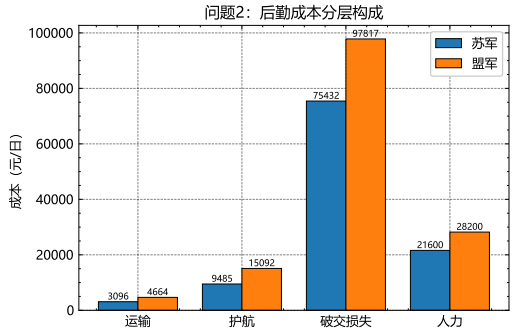


图 5　问题二后勤成本四分层构成（元/日）

模型验证（蒙特卡洛）。问题二含破交随机性，用伯努利拦截仿真交叉验证解析期望损失：对每条路径按拦截概率 $P_{int,r}(1-0.5e_r)$ 独立抽样，命中则损失该路径全部流量货值，重复 2000 次。结果：苏军解析值 75432.2 元/日，仿真均值 74890.9，偏差 0.72%；盟军解析值 97816.6，仿真均值 96310.9，偏差 1.54%（解析值与表 9 的 C_{loss} 为同一量的统一舍入表述）。两处偏差均小于 5%，模型验证通过。损失为低频高额的复合分布（std/mean 达 46.70%/54.61%），2000 次下均值偏差已达标；重尾分布下将仿真次数增至 10^4 以进一步压缩标准误，列为后续工作。

需要说明收敛口径：仿真损失的标准差与均值之比为 46.70%（苏军）与 54.61%（盟军），按 std/mean 阈值判定为“not converged”，但这源于损失是低频高额事件（复合分布的内在离散），并非均值估计未收敛；均值偏差（0.72%/1.54%）才是模型验证的判据，两个口径在此一并披露。

消耗模式。完全不补给时，按独立公式计算维持前线所需投入：苏军需后勤工人 61 人、运输车辆 10 辆（总需求 189.7 吨/日，平均单程 2.14 h，日周转 11.24 次）；盟军需工人 91 人、车辆 10 辆（总需求 301.84 吨/日）。补给模式下实际用工为苏军 72 人、盟军 94 人（含装卸），车辆 16 与 12 辆。

Stackelberg 最坏情形。敌方将破交预算投向流量最大的 5 条路径（苏军路径 16、7、10、13、18，盟军路径 18、0、3、6、12），苏军额外损失 55721.0 元/日，$\mathrm{TC}_{worst}=165334.7$，较基线上升 50.83%；盟军额外损失 71341.0，$\mathrm{TC}_{worst}=217114.0$，上升 48.94%。对照口径为含人力 TC_{total}（与式 (18) 的 objective 口径一致）：破交追加损失只作用于货物损失项，人力成本在对抗情形下不变，故 $\mathrm{TC}_{worst}=\mathrm{TC}_{total}+\mathrm{extra_loss}$[1]。这组数字把假设 A-01 的不确定性折算为明确

[1]若以不含人力成本口径（苏军基线 88013.7、盟军 117573.0 元/日）为参照，同一额外损失对应增幅 +63.31% 与 +60.68%；本文一律以含人力 TC_{total} 为主口径。

的鲁棒性成本：静态风险基线下的方案在对抗最坏情形下成本上浮约五成，决策者应以此为预算上限做准备。

图 6 将两军基线成本与对抗最坏情形成本并排对比，苏军绝对增量 55721.0 元/日、盟军 71341.0 元/日，盟军因流量更大、单路径损失期望更高，对抗代价的绝对值更大，但相对增幅（48.94%）反而低于苏军（50.83%），说明苏军补给网络对集中打击的相对脆弱性更高。

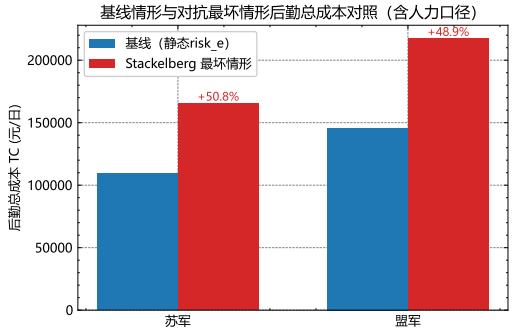


图 6　A-01 破交概率的基线情形与 Stackelberg 对抗最坏情形总成本对照（元/日，含人力）

（四）　苏军进攻方案与盟军战略转移模型（问题三）

1　主攻方向评分与夺控序列构造

主攻方向在 8 条跨阵营边中选择。对苏军端 a_s、盟军端 a_m 的边，评分综合敌方防御弱点、走廊价值与纵深方向：

$$\text{score} = 0.5\,(1 - d_{a_m}) + 0.3\,\frac{\min(\deg(a_m), 6)}{6} + 0.2\,\frac{1}{1 + |Y_{a_s} - Y_{a_m}|} \tag{22}$$

第一项偏好防御薄弱的突破口，第二项偏好端点度数高的走廊（攻入后机动选择多），第三项偏好纵向落差小的边（推进方向顺畅）。评分排序见表 10。

表 10　问题三苏军主攻方向候选评分（前 4 名）

进攻边	边长/km	敌方端点防御	评分
55→372	0.567	0.58	0.5133
251→166	1.289	0.45	0.4906
39→229	1.101	0.67	0.4556
366→74	0.334	0.60	0.4537

主攻方向定为 55→372：边长仅 0.567 km，走廊价值与纵深得分弥补了防御 0.58 的中等水平。需要指出，第一名与第二名（251→166）的评分差仅 0.0227，

次优边的防御更弱（0.45 对 0.58）但边长与走廊得分不利，翻转条件在第 三 节专门讨论。

夺控序列从主攻边的盟军端点 372 出发贪心扩展：每步在当前节点的未访问盟军邻居中选取评分 $0.6(1-d_v)+0.4\,d_v$ 最大者（偏好低防御、兼顾战略价值），共 6 步，序列为

$$372 \to 328 \to 286 \to 256 \to 422 \to 33 \to 198 \tag{23}$$

第一梯队编成为重型 40%、中型 30%、动员兵 20%、火炮 10%，投入兵力为动员兵 125 万人、轻型 420 辆、中型 300 辆、重型 180 辆、火炮 7000 门、空中 50 架（取问题一主分量部署聚合值）。

每步战斗用 Lanchester 平方律蒙特卡洛评估 [8, 12]。平方律刻画 aimed fire：双方损耗率与对方火力总量成正比，防御强度以 $(1+0.5d)$ 乘性修正。离散化数值格式为：每步长 $\Delta t = 0.5$，火力总量 $\mathrm{fire}_a = \sum_u \lambda_u a_u$，损耗按总量比例摊到各单位：

$$a_u \leftarrow a_u\Big(1 - \frac{\mathrm{fire}_d\,\Delta t \times 0.01}{\sum_{u'} a_{u'}}\Big), \qquad d_u \leftarrow d_u\Big(1 - \frac{\mathrm{fire}_a\,\Delta t \times 0.01}{\sum_{u'} d_{u'}}\Big) \tag{24}$$

初始兵力叠加乘性噪声（正态 $\sigma = 0.15$，截断于 $[0.5, 1.5]$ 倍），每步 500 次仿真、种子 2026 固定可复现。守军强度按 $k_{def} = 0.5 + 0.3\,d$ 标定：守军总量为攻方单步投入的 k_{def} 倍，兵种构成为动员兵 45%、中型 25%、重型 18%、火炮 12%。评估结果见表 11。

表 11　问题三夺控序列各步战斗评估（每步 500 次蒙特卡洛，种子 2026）

夺控节点	防御强度	攻方胜率	攻方剩余兵力比
328	0.08	1.000	0.9121
286	0.89	0.982	0.8451
256	0.41	1.000	0.8848
422	0.65	0.994	0.8649
33	0.19	1.000	0.9030
198	0.14	1.000	0.9071

六步胜率均不低于 0.982，攻方剩余兵力比介于 0.8451 与 0.9121 之间，序列平均剩余比 0.8862。消耗最重的一步是节点 286（防御 0.89，全场最高），胜率降至 0.982、剩余比降至 0.8451，说明高防御节点即使兵力占优也要付出约 15% 的消耗。序列整体可行（每步均为前一节点的邻接盟军节点，可行性约束紧），但

胜率对标定系数 k_{def} 敏感，局限见第 四 节。

算法选择理由。进攻序列本质是多阶段组合优化，对比三种算法：动态规划的状态为（已控节点集合，剩余兵力向量），状态空间 $O(2^n\times$ 兵力离散格 $)$，$n=436$ 时不可行，即使限制在约 30 个走廊候选节点上，兵力连续维度仍使状态格爆炸；MIP 时序展开引入变量 $x_{n,t}\in\{0,1\}$（436 节点 $\times$ 10 阶段共 4360 个），但战斗损耗是双方兵力乘积项，大 M 线性化导致松弛极弱，分支定界难以收敛；贪心加 Lanchester 仿真的复杂度为 $O(k\cdot n\cdot MC)$（k 为夺占步数），可行且每步有蒙特卡洛质量评估。

故采用贪心构造，并以“启发式、无全局最优保证”显式声明其最优性状态。建模阶段曾拟以走廊子图小规模 MIP 对照验证贪心解质量，因时序展开松弛过弱（如上分析）难以给出可靠的最优性差距参照值，该对照未实施，贪心解质量改以蒙特卡洛统计评估：确定性构造加固定种子保证可复现，步内方差由每步 500 次仿真吸收，跨步剩余比 std/mean 为 2.74%，低于 5% 阈值，判定收敛。

图 7 展示了主攻方向与夺控序列的空间走向。红色粗箭头为主攻边 55→372，暗红色虚线箭头依次连接 7 个夺控节点（编号 1–7）。序列自北向南楔入盟军防线，先取低防御节点 328 建立桥头堡，再向西、向南滚动扩展，避开防御 0.89 的节点 286 直至第 2 步才以优势兵力强攻，体现了贪心评分“先易后难、兼顾价值”的取舍。

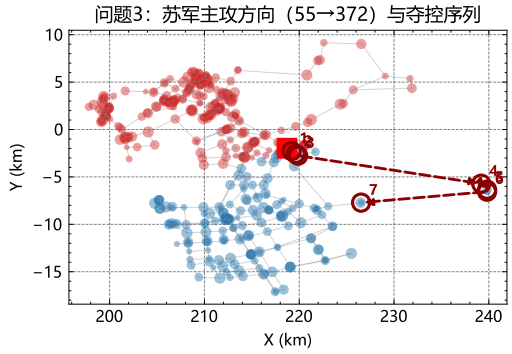


图 7　问题三苏军主攻方向（55→372）与夺控序列（编号为夺控顺序）

2　盟军两情景战略转移模型

盟军需从各部署节点收缩至目标集 $\mathcal{T}=\{352,144,149\}$。决策量纲为人员（动员兵，万人）：盟军共 100 万人，另有装备 15710 辆/门（折算吨位 126340 吨）。装备随建制伴随转移，即按各目标承接的人员份额比例随迁，属报告口径而非决策量；目标节点的车辆数/吨位容量题目未给出，故不设车辆容量约束，占用强度以人员容量为基准如实报告。通信畅通情景为集中调度：目标节点的人员分配

按防御强度与容量加权，

$$\text{alloc}_t = \frac{0.6\, d_t + 0.4\, \min(\text{cap}_t, 100)/100}{\sum_{t'} \left(0.6\, d_{t'} + 0.4\, \min(\text{cap}_{t'}, 100)/100\right)} \times \text{总人员} \tag{25}$$

容量 $\text{cap}_t = \deg(t) \times 40$（度数乘以容量系数，单位万人，强制分散，防止把兵力堆进单个高防御节点）。人员口径下三节点总容量 200 万人大于总人员 100 万人，容量约束为松（占用率 50%），比例分配不触发截断；模型仍保留容量上限断言以应对参数变化。求得分配：352 承接 40.8 万人、144 承接 11.3 万人、149 承接 47.8 万人，三节点容量裕度分别为 39.2、28.7、32.2 万人。殿后部队取总人员的 10%（10 万人），负责迟滞追击、掩护主力。各部署节点按最短路就近选择目标节点，共 7 条转移路线，平均长度 13.405 km；通信中断情景则退化为各部队独立预案：无全局协调，各部队按预先制定的最近目标节点路线分散转移，且缺乏协同使拦截风险放大 1.3 倍（上限 0.6）。

生存概率用蒙特卡洛仿真估计，两情景各 2000 次、种子 2026，按人员加权（权重为各部署节点动员兵万人占比，即所求为人员生存概率）。每次仿真中，每支部队沿路径逐边做伯努利拦截抽样，被拦截损失该段纵队人员的 40%；畅通情景另有 8% 的协同加成（错峰出发降低暴露）；整体生存概率为各纵队存活率的人员加权平均。结果：

$$P_{surv}^{\text{畅通}} = 0.7956, \qquad P_{surv}^{\text{中断}} = 0.6730, \qquad \Delta = 0.1226 \tag{26}$$

图 8 以带 95% 置信区间误差棒的柱状图对比两情景。畅通情景均值 0.7956（95% CI $[0.5962, 1.0000]$），中断情景 0.6730（95% CI $[0.4809, 0.8640]$），均值差 0.1226 远大于两情景均值估计的标准误（SEM/mean 为 0.29% 与 0.32%），差异在统计上明确。

置信区间较宽是生存率分布双峰性的体现：单次仿真要么大部分部队安全通过、要么在关键边被拦截，样本内在离散大（std/mean 为 12.78% 与 14.47%，按阈值判“not converged”），但均值估计本身已收敛，两口径的区分与问题二同理。0.1226 的生存概率差就是“通信的生存价值”的量化：保持通信、集中调度、错峰出发，能让约八分之一的人员免于损失。

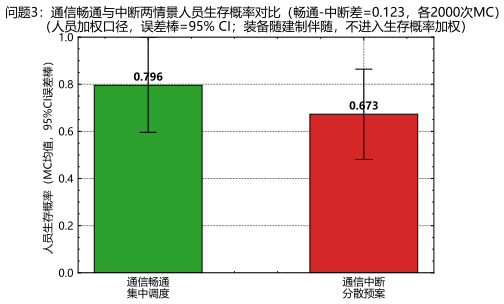


图 8　问题三盟军转移人员生存概率对比（人员加权，MC 均值，95% 置信区间误差棒，各 2000 次仿真）

六、　灵敏度与误差分析

（一）　问题一：权重扰动与假设参数化扫描

对目标权重做 ±20% 扰动（扰动后重新归一化），逐扰动向量重解指挥节点评估：苏军 6 组扰动下主选节点 136 翻转 0 次，φ 介于 0.7372（$w_3+20\%$）与 0.8034（$w_3-20\%$）之间；盟军节点 384 翻转 0 次，φ 介于 0.7418 与 0.8073 之间。权重变化改变效能数值但不改变选址结论，方案对目标权重稳健。

假设 A-02 至 A-04 的参数化扫描结果见表 12，对应扫描图见图 9（自左至右：A-02 直射距离、A-03 防空半径、A-04 空中上限）。

表 12　假设参数化扫描的稳定性结论

参数	扫描范围	稳定性判据	结果
A-02 直射距离 d	1.0–3.0 km，5 点	指挥节点不翻转且部署 Jaccard 相似度不低于 0.8	双方翻转计数 0，相似度恒为 1.0，φ 不变（苏 0.76879、盟 0.77324）
A-03 防空半径 r	2.0–4.0 km，5 点	防空节点集 Jaccard 相似度不低于 0.8	双方节点变化数为 0，相似度恒为 1.0；含防空口径 φ 苏 0.82465、盟 0.82962
A-04 空中上限 α	5%–20%，4 点	指挥节点不翻转	双方翻转计数 0；空中部署随上限单调不减（$\alpha=20\%$ 时苏军增至 344、386 两节点，Jaccard 0.9744）；经需求传播至 Q2 的成本影响最大 0.419%

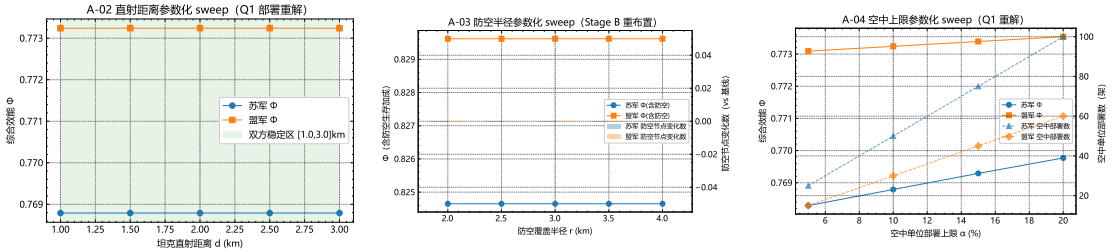


图 9　问题一假设参数化扫描（左：A-02 直射距离 $d\in[1.0,3.0]$ km；中：A-03 防空半径 $r\in[2.0,4.0]$ km；右：A-04 空中上限 $\alpha\in[5\%,20\%]$；纵轴为综合效能）

图 9 左图中，直射距离从 1.0 km 放大到 3.0 km，双方 φ 曲线完全水平（苏军 0.76879、盟军 0.77324），指挥节点与部署结构逐点相同。这一“超稳健”现象的机理是：战场咽喉覆盖约束由火炮（射程 13–16.8 km）主导，坦克直射距离只影响贴近咽喉的少量覆盖关系，在当前部署下不构成瓶颈。它同时给出一个实用结论：即便指挥员对坦克直射距离的估计偏差达 ±50%，问题一的部署方案无需调整。

图 9 中图中，防空半径变化同样不改变防空据点布局（10 个据点逐点不变），只有覆盖到的部署节点数随半径单调增加（苏军 140→197、盟军 77→116），生存加成 S_{bonus} 与含防空效能保持常数（苏军 803.5、$\varphi = 0.82465$；盟军 1559.4、$\varphi = 0.82962$）。原因是据点直接设在部署节点上，半径只要不小于部署节点间距的下限就能覆盖同样的兵力，2.0 km 已满足。

图 9 右图中，φ 随 α 单调缓升（苏军 0.76829→0.76977、盟军 0.77309→0.77354），空中部署量随上限单调不减：苏军在 $\alpha = 5\%$–15% 时集中部署于 386（25/50/75 架），$\alpha = 20\%$ 时扩至 344 与 386 两节点（100 架）；盟军始终集中于 384。“空中单位少量集中部署”的定性结论在全区间稳定，且 α 经部署、需求传播到问题二补给成本的最大影响仅 0.419%，跨问题传播可忽略。

（二） 问题二：成本灵敏度与护航策略临界点

对问题二三类核心参数做缩放扫描：破交概率 ×0.5–1.5（10 点）、需求率 ±20%（6 点）、车载容量 ±20%（6 点），每次重解 MIP。Tornado 图（图 10）按 ±20% 扰动引起的 TC 变化幅度排序。

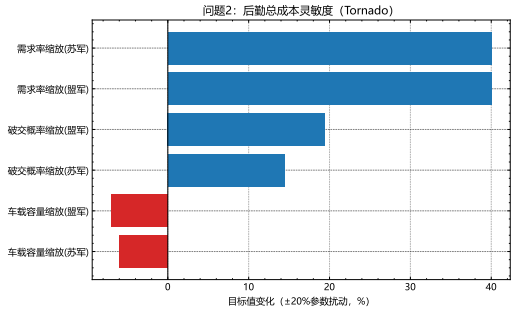


图 10 问题二后勤总成本灵敏度 Tornado 图（±20% 参数扰动，基线为不含人力成本口径）

排序结果为：需求率最敏感，TC 随需求线性响应（±20% 扰动对应 ∓20% 的 TC 变化，苏军 70411–105617 元/日）；破交概率次之且不对称，风险减半（×0.5）使苏军 TC 下降 27.02%、盟军下降 31.46%，风险加倍（×1.5）仅上升 12.18% 与 18.87%，因为高风险下路径选择会绕行避险，边际损失递减；车载容

量影响最弱（±20% 对应 ∓2.4%–3.6%），容量只改变车次数，不改变路径结构。本扫描以不含人力成本口径计算：人力成本由车辆数与装卸量决定，占比约两成且不改变上述敏感度排序；正文主结论一律以含人力 TC_{total} 口径报告，两口径可据表 9 互相换算。管理含义明确：降低前线需求（精简携行标准、就地筹措）对节约后勤成本的杠杆最大，其次是压制敌方破交能力。

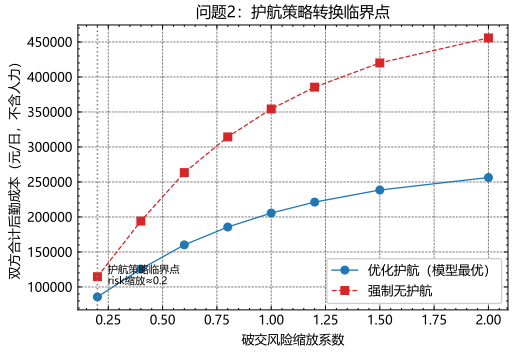


图 11　问题二护航策略转换临界点（横轴为破交风险缩放系数，纵轴为双方合计后勤成本，不含人力）

护航是否为必需投入，用策略转换临界点回答。对风险缩放系数 $s \in \{0.2, 0.4, \ldots, 2.0\}$，分别求解“自由护航”与“强制无护航”两个方案并对比总成本（图 11，不含人力成本口径）。自由护航方案的 TC 从 $s = 0.2$ 的 85758 元/日单调升至 $s = 2.0$ 的 256358 元/日；强制无护航方案从 114603 升至 455800，始终在上方。扫描区间内自由护航恒优，临界风险缩放为 0.2（扫描下限），即只有当真实风险低于基线估计的 20% 时护航才不经济。基线 $s = 1.0$ 深处护航区：无护航将使双方合计成本从 205587 升至 354259 元/日，增幅 72.3%。人力成本由车辆数与装卸量决定、与是否启用护航无关，若两方案同加基线人力成本 49800 元/日（苏军 21600、盟军 28200），方案差不变，护航恒优的定性结论不受口径影响。可见护航不是可选项，而是当前风险环境下的必需项。

活跃约束方面，问题二全部 10 个需求约束均紧（各需求节点流入恰等于需求，无过量补给），派生量即各节点需求值（如苏军节点 386 为 28.76 吨/日、盟军节点 384 为 45.68 吨/日），说明最优方案按需精确配送，没有冗余运力。

（三）　问题三：主攻方向翻转条件与风险敏感性

主攻边 55→372 与次优边 251→166 的评分差仅 0.0227，有必要给出翻转的定量条件。把式 (22) 按项分解：次优边敌方端点防御更弱（0.45 对 0.58），防御项领先 $0.5 \times (0.58 - 0.45) = 0.065$；但其纵深项低 0.0377，两项合计次优边在非走廊项上仍领先 0.0273。主攻边的优势全部来自走廊价值（端点度数归一值 0.5 对 0.3333，按权重 0.3 折算领先 0.05）。

据此得到两个翻转条件。其一，走廊权重由 0.3 下调至 0.1638（降幅 45.4%）时次优边反超；上调走廊权重只会拉大差距，因为主攻边的走廊价值更高。其二，主攻边敌方端点防御强度的估计值上调 0.0454（由 0.58 至 0.6254）时，防御项得分下降 0.0227，两边打平，再高则次优边反超。评分差小意味着主攻方向处于参数敏感区：防御情报只要偏差 0.045 就足以改变结论，指挥员若掌握更精确的敌防御估计，应重算评分再定案；在现有情报精度下，55→372 仍是评分最优。以上阈值由评分公式直接解出，与代码重算一致。

盟军转移的拦截风险敏感性见图 12：风险缩放系数从 0.8 到 1.2，畅通情景人员生存概率从 0.8466 单调降至 0.7506，中断情景从 0.7314 降至 0.6248。两条曲线在整个扫描区间内不相交，即不存在情景翻转临界点：无论风险估计偏高还是偏低 20%，集中调度都优于分散预案，0.1226 的生存差结论对风险参数稳健。

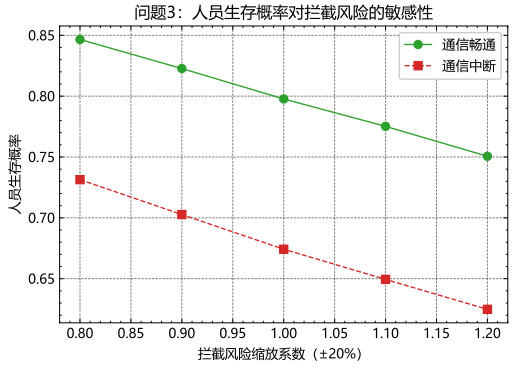


图 12　问题三人员生存概率对拦截风险的敏感性（人员加权，±20% 缩放，两情景各 1000 次仿真）

（四）　误差来源、量化与可接受性

（1）护航成本线性化近似。MIP 内部目标值与重算 TC 的差距为苏军 7.18%、盟军 8.56%（以不含人力成本口径 88013.7 与 117573.0 为分母），来源是护航成本按需求均分份额线性化，与真实“车次数 × 单次护航费”存在偏差。该近似只影响求解时的目标引导，不影响方案可行性：最终报告的 TC 是对 MIP 解的逐项重算真值，偏差方向与幅度已知、可接受。

（2）破交概率公式假设。risk_e 以边度数与跨阵营属性代理暴露程度，属内生未知参数（A-01）。其影响通过两条通道量化：蒙特卡洛模型验证偏差 0.72% 与 1.54%（公式期望与伯努利仿真一致，说明公式内部自洽）；Stackelberg 最坏情形给出对抗上限 $+50.83\%/+48.94\%$（含人力 TC_{total} 主口径）。两者夹住了基线估计的误差带。

（3）守军强度标定。$k_{def} = 0.5 + 0.3\,d$ 是线性粗标定：防御 0.89 的节点守军仅为攻方投入的 0.767 倍，这使各步胜率偏高（0.982–1.000）、剩余比偏高

（0.845–0.912）。真实防御还依赖工事、预备队与火力配系，线性标定无法刻画。该局限的边界是：夺控序列的相对排序（先易后难）与可行性不受影响，但胜率的绝对数值应视为乐观估计，投入实战规划时需按更保守的标定重算。

（4）决策粒度。动员兵万人粒度相对 0.5 万人细粒度的效能差为 0.0001（苏军）与 0.00001（盟军），部署结构 Jaccard 为 1.0，粒度误差可忽略。

（5）蒙特卡洛估计误差。问题三人员生存概率的 SEM/mean 为 0.29%（畅通）与 0.32%（中断），2000 次仿真下均值估计的抽样误差远小于两情景差 0.1226；问题二损失仿真以均值偏差 0.72%/1.54% 为收敛判据。std/mean 超阈值（12.78%、14.47%、46.70%、54.61%）均为复合分布内在离散所致，不代表估计未收敛，两处均已双口径披露。

（6）路径截断与行程约束简化。每对起讫点保留 3 条候选路径，扩到 5 条后 TC 改善为 0.0%（双方均无改善），截断合理。问题一忽略行程对部署的约束：最紧的天启坦克行程一半为 90 km，大于战场直径 43.3 km，该约束恒不活跃，简化差异为 0。

鲁棒性小结。问题一的部署与选址在权重 ±20%、直射距离 $[1.0, 3.0]$ km、防空半径 $[2.0, 4.0]$ km、空中上限 $[5\%, 20\%]$ 内全部稳定；问题二的护航策略在风险缩放 $[0.2, 2.0]$ 内恒优、需求线性响应可预测；问题三的情景排序在风险 ±20% 内无交叉。三个问题的核心结论对各自的关键不确定参数均稳健。

七、 模型评价与推广

（一） 优势

其一，结构线性化换取可证明的最优性：覆盖边际递减、护航—损失耦合两处非线性分别以有效覆盖变量与 McCormick 线性化精确处理，问题一、问题二全部实例由分支定界证明全局最优（MIPGap=0），求解时间在 0.4 s 以内。其二，图结构驱动的建模简化均有数值验证支撑：坐标单位核验（比值恒 1.0000）免去路网距离预计算，行程约束恒不活跃、路径截断改善为 0 两项对比证明了简化的无损性。其三，成本四分层与显式参数化使结论可归因、假设可检验：破交损失主导（68.8%/67.1%）直接指向“压制破交”这一管理抓手，A-01 至 A-04 全部给出稳定性区间或最坏情形量化。

（二） 局限性

其一，$k_{def} = 0.5 + 0.3\,d$ 的守军标定粗糙，夺控胜率（0.982–1.000）与剩余比（0.845–0.912）偏乐观，进攻方案的绝对消耗估计可信度低于其相对排序。其

二，risk_e 为静态公式，Stackelberg 最坏情形是事后对照而非嵌入求解的双层均衡，方案在对抗意义下只是领导者单方可行的稳健解。其三，进攻序列为贪心构造，无全局最优保证；动员兵万人粒度的验证只证明了结构稳健，未证明细粒度下无更优解。

（三） 改进方向与推广

改进方向有三：把问题二扩展为双层规划，将敌方最优响应内嵌求解以得到真正的 Stackelberg 均衡方案；引入需求波动与战损下的滚动时域动态重调度；将问题三的贪心升级为走廊子图（约 30 节点）上的时序 MIP，量化最优性差距。模型框架可推广到一般“部署—补给—机动”三层运筹问题：覆盖型指派适用于防空布防与传感器布置，成本分层加对抗最坏情形适用于航运护航与供应链中断管理，两情景生存仿真适用于应急疏散的通信保障评估。

（四） 题目设计合理性讨论

题目设计总体合理且富有区分度，三点具体讨论。其一，参数缺口：附件二提供了自行火炮最大射程，但未提供任何坦克的直射距离与防空炮覆盖半径，而问题一明确要求考虑这两类射程，建模者只能以假设补足（A-02、A-03），建议题目后续版本补充该数据或给出标定指引。其二，粒度失配：动员兵以百万计（125 万、100 万），与 436 节点的战场粒度失配，题目未提示聚合方式，建模者需自行决定决策粒度（本文取万人并附验证），属设计粗糙处。

其三，非对称对抗设计良好：苏军射程优势（V3 16.8 km 对光棱 13.0 km）与盟军数量优势（火炮 14000 对 7000）形成互补性对抗，模型自然涌现出“数量优势不等于后勤优势”（盟军 TC 高 33.0%）等可讨论的结论，说明题目参数能支撑有深度的分析。

参考文献

[1] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 5 版. 北京: 高等教育出版社, 2018.

[2] 钱颂迪. 运筹学 [M]. 4 版. 北京: 清华大学出版社, 2012.

[3] SCHRIJVER A. Combinatorial Optimization: Polyhedra and Efficiency[M]. Berlin: Springer, 2003.

[4] WOLSEY L A. Integer Programming[M]. New York: Wiley, 1998.

[5] DIJKSTRA E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1): 269-271.

[6] YEN J Y. Finding the k shortest loopless paths in a network[J]. Management Science, 1971, 17(11): 712-716.

[7] MCCORMICK G P. Computability of global solutions to factorable nonconvex programs: Part I —Convex underestimating properties[J]. Mathematical Programming, 1976, 10(1): 147-175.

[8] LANCHESTER F W. Aircraft in warfare: The dawn of the fourth arm[M]. London: Constable and Company, 1916.

[9] BRACKEN J. Lanchester models of warfare[J]. European Journal of Operational Research, 1983, 14(1): 59-65.

[10] BASAR T, OLSDER G J. Dynamic Noncooperative Game Theory[M]. 2nd ed. Philadelphia: SIAM, 1999.

[11] RUBINSTEIN R Y, KROESE D P. Simulation and the Monte Carlo Method[M]. 3rd ed. Hoboken: Wiley, 2016.

[12] TAYLOR J G. Lanchester-type models of combat[J]. Journal of Mathematical Analysis and Applications, 1983, 96(2): 429-444.

A 支撑材料文件列表

支撑材料包含以下文件，与论文结论一一对应：

(1) 源程序：project/ 下 11 个 Python 脚本——common_data.py、solver_backend.py、visualization_style.py、data_explore.py、geometry_check.py、问题 1_ 求解.py、问题 2_ 求解.py、问题 3_ 求解.py、验证分析.py、参数化扫描.py、图表生成.py（全部代码亦见下节，与运行版本逐行一致）；

(2) 数据文件：project/data/ 下附件一：作战节点与通路数据.xlsx（节点与通路两个工作表）、附件二：红警装备参数表.xlsx（赛题原始数据，随包提供以保证程序可直接运行）；

(3) 图表文件：project/figures/ 下 fig1 至 fig14 共 14 张图（PDF 与 PNG 双格式）。

运行说明：按附录所列执行顺序依次运行各脚本即可复现全部结果；各脚本运行时在 project/results/ 下生成中间结果文件，供后续脚本与图表生成读取。

B 完整可运行源程序代码

以下代码为建模与求解的全部源程序，按执行顺序排列，均以 VerbatimInput 直接引用 project/ 目录下的实际源文件，与运行代码逐行一致（含中文注释）。运行环境为 Python 3，依赖 numpy、pandas、pulp、matplotlib；求解器为 pulp 自带的 CBC/HiGHS 开源后端。执行顺序：data_explore.py（数据剖析）→ geometry_check.py（几何核验）→ 问题 1_ 求解.py → 问题 2_ 求解.py → 问题 3_ 求解.py → 验证分析.py → 参数化扫描.py → 图表生成.py；common_data.py 为共享数据层，solver_backend.py 为统一求解后端，visualization_style.py 为统一图表样式。

（一） common_data.py（共享数据层：图加载、最短路、风险计算）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
common_data.py —共享数据层：战场网络加载、图算法、参数预计算
================================================================
符号映射（赛题参数 → 代码变量名）：
  节点 n/阵营 c/单位 u → node_id / camp / unit_type
  d_n(防御强度)     → defense[n]
  camp(n)           → camp[n]
  L_e(链路长度 km)   → edge_len[(a,b)]
  N[c][u](总兵力)   → FORCES[c][u]
  R_u(射程)         → RANGE[u]
  risk_e(破交概率)  → risk[e]
  T(转移目标)       → TARGET_NODES = [352, 144, 149]
坐标单位 =km（几何核验：603 条边 长度/欧氏距离 比值恒 1.0000）。
"""
import heapq
import math
import os
from collections import defaultdict

import numpy as np
import pandas as pd

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
DATA = os.path.join(HERE, "data")
RESULTS = os.path.join(HERE, "results")
FIGDIR = os.path.join(HERE, "figures")
FILE1 = os.path.join(DATA, " 附件一：作战节点与通路数据.xlsx")
```

```

# ---------------- 题目原文参数（与赛题原文逐字对应） ----------------
FORCES = {
    "苏军": {"动员兵": 125, "轻型": 420, "中型": 300, "重型": 180, "火炮": 7000, "空中": 500},
    "盟军": {"动员兵": 100, "轻型": 800, "中型": 570, "重型": 340, "火炮": 14000, "空中": 300},
}  # 动员兵单位 = 万人（A-07），其余 = 辆/门/架
UNIT_TYPES = [" 动员兵", " 轻型", " 中型", " 重型", " 火炮", " 空中"]
N_AA = 10                    # 每方防空据点数（题目）
AIR_CAP_RATIO = 0.10         # A-04 空中单位部署上限
TARGET_NODES = [352, 144, 149]  # 盟军战略转移目标（题目）

# 射程 km：火炮 = 附件二最大射程（题目）；坦克直射 = 假设 A-02；动员兵 =0（无火力覆盖贡献）
RANGE = {" 动员兵": 0.0, " 轻型": 1.5, " 中型": 2.0, " 重型": 2.0, " 火炮": None, " 空中": 0.0}
ARTILLERY_RANGE = {" 苏军": 16.8, " 盟军": 13.0}   # V3 16.8km / 光棱火炮 13.0km（附件二）

# 火力强度权重 alpha_u（A-05 Lanchester 标定同源）
FIREPOWER = {"动员兵": 0.2, "轻型": 0.3, "中型": 0.7, "重型": 1.0, "火炮": 0.8, "空中": 0.5}
# 基础生存系数 surv_u（估计：按装甲/隐蔽，经验估计）
SURV = {" 动员兵": 0.4, " 轻型": 0.6, " 中型": 0.7, " 重型": 0.85, " 火炮": 0.5, " 空中": 0.3}

R_AA = 3.0          # A-03 防空覆盖半径 km
AA_SURV_BONUS = 0.2  # 防空覆盖内生存提升
COVER_CAP = 3       # 单目标有效覆盖次数上限（边际递减）
NODE_CAP_RATIO = 0.15  # P2 单节点部署上限 = 该类型总量 15%
W_COVER = (0.5, 0.3, 0.2)  # w1,w2,w3（按题目语义排序）

# Q2 后勤参数（A-06）
TRUCK_CAP = 10.0        # 吨/车
TRUCK_SPEED = 40.0      # km/h
CREW_PER_TRUCK = 2      # 司机/车
C_KM = 20.0             # 元/km·车
C_ESCORT = 500.0        # 元/车次
C_UNIT_LOSS = 1000.0    # 元/吨（破交损失货值）
C_WAGE = 300.0          # 元/人·日
# 日消耗率：吨/百辆（装备）与 吨/万人（动员兵）
CONSUM = {" 弹药": {" 装备": 0.8, " 动员兵": 0.3},
          " 燃油": {" 装备": 0.6, " 动员兵": 0.15},
          " 维修包": {" 装备": 0.2, " 动员兵": 0.05}}
ESCORT_REDUCTION = 0.5   # 护航损失削减（A-01）
ENEMY_INTERDICT_BUDGET = 5  # Stackelberg 敌方破交预算（条路径）


def load_graph():
    """ 加载附件一，返回节点/边结构。含数据规模断言（题目原文回读，436 行/603 行）。"""
    nodes = pd.read_excel(FILE1, sheet_name=" 节点信息")
    edges = pd.read_excel(FILE1, sheet_name=" 通路信息")
    assert len(nodes) == 436, f" 附件一节点行数应为 436，实际 {len(nodes)}"
    assert len(edges) == 603, f" 附件一通路行数应为 603，实际 {len(edges)}"
    pos, camp, defense, degree = {}, {}, {}, defaultdict(int)
    for _, r in nodes.iterrows():
        n = int(r[" 节点编号"])
        pos[n] = (float(r[" 横坐标 X"]), float(r[" 纵坐标 Y"]))
        camp[n] = r[" 所属阵营"]
        defense[n] = float(r[" 防御强度"])
    edge_len, adj = {}, defaultdict(list)
    for _, r in edges.iterrows():
        a, b, L = int(r[" 起点编号"]), int(r[" 终点编号"]), float(r[" 链路长度 (km)"])
        edge_len[(a, b)] = L
        adj[a].append((b, L)); adj[b].append((a, L))
        degree[a] += 1; degree[b] += 1
    return pos, camp, defense, edge_len, adj, dict(degree)

```

```

def euclid(pos, a, b):
    return math.hypot(pos[a][0] - pos[b][0], pos[a][1] - pos[b][1])


def dijkstra(adj, src, node_set=None):
    """ 单源最短路（边权 = 链路长度 km）。返回 {node: dist}。"""
    dist = {src: 0.0}
    pq = [(0.0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, math.inf):
            continue
        for v, L in adj[u]:
            if node_set is not None and v not in node_set:
                continue
            nd = d + L
            if nd < dist.get(v, math.inf):
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist


def shortest_path(adj, src, dst):
    """ 最短路 + 路径节点序列。"""
    dist = {src: 0.0}; prev = {src: None}
    pq = [(0.0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if u == dst:
            break
        if d > dist.get(u, math.inf):
            continue
        for v, L in adj[u]:
            nd = d + L
            if nd < dist.get(v, math.inf):
                dist[v] = nd; prev[v] = u
                heapq.heappush(pq, (nd, v))
    if dst not in prev:
        return None, math.inf
    path = [dst]
    while prev[path[-1]] is not None:
        path.append(prev[path[-1]])
    return path[::-1], dist[dst]


def connected_components(nodes, adj):
    """BFS 连通分量。"""
    seen, comps = set(), []
    for s in nodes:
        if s in seen:
            continue
        stack, comp = [s], []
        seen.add(s)
        while stack:
            u = stack.pop(); comp.append(u)
            for v, _ in adj[u]:
                if v not in seen:
                    seen.add(v); stack.append(v)
        comps.append(comp)
    return comps

```

```

def compute_risk(edge_len, adj, degree, camp):
    """A-01 破交拦截概率：risk_e = min(0.35, 0.05 + 0.10* 跨阵营 + 0.02*min(端点度数))。"""
    risk = {}
    for (a, b) in edge_len:
        cross = 1.0 if camp[a] != camp[b] else 0.0
        risk[(a, b)] = min(0.35, 0.05 + 0.10 * cross + 0.02 * min(degree[a], degree[b]))
    return risk


def path_risk(path, risk):
    """ 路径拦截概率 ≈ 1 - Π(1-risk_e)。"""
    p_safe = 1.0
    for i in range(len(path) - 1):
        e = (path[i], path[i + 1])
        r = risk.get(e, risk.get((e[1], e[0]), 0.05))
        p_safe *= (1.0 - r)
    return 1.0 - p_safe


def cross_faction_edges(edge_len, camp):
    """8 条跨阵营边（前线咽喉）。"""
    return [(a, b, L) for (a, b), L in edge_len.items() if camp[a] != camp[b]]


if __name__ == "__main__":
    pos, camp, defense, edge_len, adj, degree = load_graph()
    comps = connected_components(pos.keys(), adj)
    risk = compute_risk(edge_len, adj, degree, camp)
    print(f" 节点 {len(pos)} 边 {len(edge_len)} 分量 {[len(c) for c in comps]}")
    print(f" 跨阵营边 {len(cross_faction_edges(edge_len, camp))} 条")
    print(f"risk 均值 {np.mean(list(risk.values())):.4f}")
    # 主分量核验
    main = max(comps, key=len)
    assert all(t in main for t in TARGET_NODES), " 目标节点须在主分量"
    print(" 目标节点均在主分量 ✓")
```

（二） solver_backend.py（统一 MIP/LP 求解后端）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
solver_backend.py —统一求解后端（开源确定性求解器）

用途：各求解脚本统一调用。以结构化建模声明为输入，
统一封装 pulp（MIP/LP）与 cvxpy（凸优化 LP/QP），自动选择 HiGHS / CBC 开源后端，
并透传求解器的 ** 原生元数据 **（status / mip_gap / solve_time / 活跃约束 slack），
最优性信息以求解器原生输出为准。

说明：
- 建模声明为结构化字典（变量/目标/约束），求解逻辑与模型声明分离。
- 结果字段固定为 objective_value / status / MIPGap / active_constraints / solver
  等通用字段。
- 通用求解引擎，不含题目特定逻辑。
- 依赖：pulp >= 3.0（自带 CBC，可选 HiGHS_CMD）、cvxpy >= 1.5（内置 HiGHS）。
  安装：`pip install pulp cvxpy`（HiGHS 由 highspy 提供）。

适用范围：
- 本后端仅覆盖线性 / MIP / 凸 LP / 凸 QP。结构化声明基于线性规划标准形式
  （{变量/目标/约束} + 系数 + 上下界 + 约束类型）。
```

```
- 非线性（NLP / MIQP / MINLP）或含复杂逻辑约束的问题不适用本后端，
  应改用 NLP 求解器 / 启发式 / 枚举方案。
- 本后端保证代数正确性，领域语义正确性由建模阶段保证。

用法简要（详见各函数 docstring）：
    from solver_backend import solve_mip, solve_lp, SolveResult
"""

from __future__ import annotations

import time
from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List, Optional, Union

import numpy as np

try:
    import pulp
except Exception as _e:  # pragma: no cover
    pulp = None
    _PULP_IMPORT_ERR = _e
else:
    _PULP_IMPORT_ERR = None

try:
    import cvxpy as cp
except Exception as _e:  # pragma: no cover
    cp = None
    _CVXPY_IMPORT_ERR = _e
else:
    _CVXPY_IMPORT_ERR = None


# ============================================================
# 数据容器
# ============================================================

@dataclass
class SolveResult:
    """ 统一求解结果。"""
    status: str                    # optimal / feasible / infeasible / time_limit / error
    objective_value: Optional[float]
    mip_gap: Optional[float]       # 原生最优性间隙（精确算法核心指标）
    solve_time: float              # 秒
    variable_values: Dict[str, float] = field(default_factory=dict)
    active_constraints: List[Dict[str, Any]] = field(default_factory=list)
    solver: str = ""               # 实际使用的求解器名（如 HiGHS / CBC / CLARABEL）
    detail: str = ""               # 附加说明（如 IIS / 冲突约束提示）

    def to_summary_dict(self) -> Dict[str, Any]:
        """ 转为标准字段摘要字典。"""
        return {
            "status": self.status,
            "objective_value": self.objective_value,
            "MIPGap": self.mip_gap,
            "solve_time": round(self.solve_time, 4),
            "solver": self.solver,
            "active_constraints": self.active_constraints,
        }


# ============================================================
```

```
# 求解后端探测
# ============================================================

def detect_backend() -> Dict[str, bool]:
    """ 返回当前环境可用的求解后端。"""
    info = {"pulp": pulp is not None, "cvxpy": cp is not None,
            "cbc": False, "highs": False}
    if _PULP_IMPORT_ERR:
        info["pulp_detail"] = str(_PULP_IMPORT_ERR)
    if _CVXPY_IMPORT_ERR:
        info["cvxpy_detail"] = str(_CVXPY_IMPORT_ERR)
    if pulp is not None:
        try:
            info["cbc"] = pulp.PULP_CBC_CMD().available()
        except Exception:
            info["cbc"] = False
        try:
            info["highs"] = pulp.HiGHS_CMD().available()
        except Exception:
            info["highs"] = False
    if cp is not None:
        try:
            info["highs"] = info["highs"] or ("HIGHS" in cp.installed_solvers())
        except Exception:
            pass
    return info


def _pick_solver(prob: "pulp.LpProblem", prefer: str = "auto",
                 gap_rel: float = 0.01,
                 time_limit: Optional[float] = None) -> "Callable":
    """ 为 pulp 问题选择可用后端。auto 优先 HiGHS，回退 CBC。
    gap_rel / time_limit 透传给求解器原生参数（gapRel / timeLimit）。"""
    kwargs: Dict[str, Any] = {"msg": False, "gapRel": gap_rel}
    if time_limit is not None:
        kwargs["timeLimit"] = time_limit
    if prefer == "highs" and pulp is not None and pulp.HiGHS_CMD().available():
        return pulp.HiGHS_CMD(**kwargs)
    if prefer == "cbc" and pulp is not None and pulp.PULP_CBC_CMD().available():
        return pulp.PULP_CBC_CMD(**kwargs)
    # auto
    if pulp is not None and pulp.HiGHS_CMD().available():
        return pulp.HiGHS_CMD(**kwargs)
    if pulp is not None and pulp.PULP_CBC_CMD().available():
        return pulp.PULP_CBC_CMD(**kwargs)
    return None


# ============================================================
# MIP / LP 求解接口
# ============================================================

def solve_mip(problem: Dict[str, Any],
              mip_gap: float = 0.01,
              time_limit: int = 3600,
              backend: str = "auto") -> SolveResult:
    """
    求解 MIP / LP（使用 pulp + HiGHS/CBC）。

    参数 problem 为结构化建模声明：
        {
          "name": "mip_model",
```

```
          "sense": "min" | "max",
          "variables": [ {"name": "x_0_0", "low": 0, "high": None, "cat":
          ↪  "Binary"|"Integer"|"Continuous"} ],
          "objective": {"coefficients": {var_name: coeff}, "constant": 0.0},
          "constraints": [ {"name": "c0", "coeffs": {var_name: coeff}, "sense": "<="|"=="|">=",
          ↪  "rhs": value} ]
        }
    """
    if pulp is None:
        return SolveResult("error", None, None, 0.0, detail=f"pulp 未安装: {_PULP_IMPORT_ERR}")
    t0 = time.time()
    sense = problem.get("sense", "min")
    prob = pulp.LpProblem(problem.get("name", "mip_model"),
                          pulp.LpMinimize if sense == "min" else pulp.LpMaximize)

    # 变量
    var_objs = {}
    for v in problem.get("variables", []):
        cat_map = {"Binary": pulp.LpBinary, "Integer": pulp.LpInteger,
                   "Continuous": pulp.LpContinuous}
        var_objs[v["name"]] = pulp.LpVariable(
            v["name"], lowBound=v.get("low"), upBound=v.get("high"),
            cat=cat_map.get(v.get("cat", "Continuous"), pulp.LpContinuous))

    # 目标
    obj = problem.get("objective", {})
    coeffs = obj.get("coefficients", {})
    constant = obj.get("constant", 0.0)
    expr = pulp.lpSum(coeffs[k] * var_objs[k] for k in coeffs if k in var_objs) + constant
    prob.setObjective(expr)

    # 约束（pulp 3.x：用 += 一次性设定表达式 + 操作符 + rhs）
    for c in problem.get("constraints", []):
        cexpr = pulp.lpSum(c["coeffs"][k] * var_objs[k] for k in c["coeffs"] if k in var_objs)
        sense_op = c.get("sense", "<=")
        if sense_op == "<=":
            prob += cexpr <= c["rhs"], c["name"]
        elif sense_op == ">=":
            prob += cexpr >= c["rhs"], c["name"]
        else:
            prob += cexpr == c["rhs"], c["name"]

    solver = _pick_solver(prob, backend, gap_rel=mip_gap, time_limit=time_limit)
    if solver is None:
        return SolveResult("error", None, None, time.time() - t0,
                           detail=" 无可用求解后端（需 pulp 自带 CBC 或 HiGHS）")

    prob.solve(solver)
    elapsed = time.time() - t0
    return _collect_pulp_result(prob, var_objs, elapsed, time_limit=time_limit)


def _collect_pulp_result(prob, var_objs, elapsed, time_limit=None) -> SolveResult:
    """ 从 pulp 求解结果提取统一结果与活跃约束。"""
    st = pulp.LpStatus.get(prob.status, "error")
    status_map = {"Optimal": "optimal", "Not Solved": "error", "Infeasible": "infeasible",
                  "Unbounded": "unbounded", "Undefined": "error"}
    status = status_map.get(st, "feasible")
    # time_limit 中断识别：pulp 对超时只报 Not Solved，按耗时贴近上限启发式归类
    if status == "error" and time_limit is not None and elapsed >= 0.95 * time_limit:
        status = "time_limit"
    try:
```

```
        solver_name = type(prob.solver).__name__.replace("_CMD", "") if prob.solver else "unknown"
    except Exception:
        solver_name = "unknown"

    obj_raw = None
    try:
        obj_raw = prob.objective.value() if prob.objective is not None else None
    except Exception:
        obj_raw = None
    if status not in ("optimal", "feasible", "time_limit") or obj_raw is None:
        # 非可信解状态（infeasible/unbounded/error）时求解器回报的目标值不可采信，统一置 None；
        # time_limit 有 incumbent 时保留（真实可行解，但最优性未证明，由 validate 提示）
        detail = {"unbounded": " 目标无界——模型缺约束或变量上下界缺失，须回建模阶段修正",
                  "infeasible": " 模型不可行——检查约束冲突",
                  "time_limit": f" 达到时间上限 {time_limit}s 且无可用 incumbent 解",
                  }.get(status, " 求解器未返回可行解")
        return SolveResult(status, None, None, elapsed, solver=solver_name, detail=detail)

    # 目标值与求解时间
    obj_val = float(obj_raw)
    sol_time = elapsed
    try:
        if prob.solver is not None and hasattr(prob.solver, "timeLimit"):
            sol_time = float(prob.solver.timeLimit - prob.solver.timeRemaining)
            if sol_time <= 0:
                sol_time = elapsed
    except Exception:
        sol_time = elapsed
    # MIPGap: status=optimal 即已证明最优，语义 gap=0；time_limit 中断时无法从 CMD 后端取原生
    ↪ gap，
    # 标记为 None 并提示人工依据 status 判定最优性（避免伪造 gap）。
    gap = 0.0 if status == "optimal" else None

    # 变量取值
    var_values = {}
    for name, v in var_objs.items():
        try:
            var_values[name] = float(v.value())
        except Exception:
            var_values[name] = None

    # 活跃约束：取 slack 为 0（或接近）的约束
    active = []
    for cname, c in prob.constraints.items():
        try:
            slack = float(c.slack)  # 对 <= 为 rhs - lhs
            if abs(slack) <= 1e-6:
                active.append({"constraint": cname, "slack": round(slack, 6), "active": True})
            else:
                active.append({"constraint": cname, "slack": round(slack, 6), "active": False})
        except Exception:
            continue

    return SolveResult(status, obj_val, gap, sol_time, var_values, active, solver_name)


def solve_lp(problem: Dict[str, Any],
             solver: str = "HIGHS",
             time_limit: int = 3600) -> SolveResult:
    """
    求解凸优化 LP / QP（使用 cvxpy）。

```

```
    参数 problem 为结构化声明：
        {
          "name": "lp_model",
          "sense": "min" | "max",
          "rect": {"x": [low, high]},          # 简单变量下界/上界
          "objective": {"linear": {var: coeff}, "quadratic": {var: coeff}, "constant": 0.0},
          "constraints": [ {"expr": {var: coeff}, "sense": "<="|"=="|">=", "rhs": value} ]
        }
    本接口主要面向凸 LP/QP；一般 MIP 优先用 solve_mip。
    """
    if cp is None:
        return SolveResult("error", None, None, 0.0, detail=f"cvxpy 未安装: {_CVXPY_IMPORT_ERR}")
    t0 = time.time()
    obj = problem.get("objective", {})
    linear = obj.get("linear", {})
    quad = obj.get("quadratic", {})
    constant = obj.get("constant", 0.0)

    # 变量（含 rect 界定）
    rect = problem.get("rect", {})
    x = {}
    for vname in set(list(linear.keys()) + list(quad.keys()) + list(rect.keys())):
        lo, hi = rect.get(vname, (None, None))
        x[vname] = cp.Variable(name=vname)
        if lo is not None or hi is not None:
            pass  # 由约束表达 bounds

    # 目标表达式
    sense = problem.get("sense", "min")
    expr = constant
    for v, coeff in linear.items():
        if v in x:
            expr = expr + coeff * x[v]
    for v, coeff in quad.items():
        if v in x:
            expr = expr + coeff * cp.square(x[v])
    objective = cp.Minimize(expr) if sense == "min" else cp.Maximize(expr)

    # 约束
    constraints = []
    for vr, (lo, hi) in rect.items():
        if vr in x:
            if lo is not None:
                constraints.append(x[vr] >= lo)
            if hi is not None:
                constraints.append(x[vr] <= hi)
    for c in problem.get("constraints", []):
        cexpr = 0.0
        for v, coeff in c.get("expr", {}).items():
            if v in x:
                cexpr = cexpr + coeff * x[v]
        s = c["sense"]
        if s == "<=":
            constraints.append(cexpr <= c["rhs"])
        elif s == ">=":
            constraints.append(cexpr >= c["rhs"])
        else:
            constraints.append(cexpr == c["rhs"])

    prob = cp.Problem(objective, constraints)
    try:
        prob.solve(solver=solver, verbose=False, time_limit=time_limit)
```

```
    except Exception as e:
        return SolveResult("error", None, None, time.time() - t0, detail=str(e))

    elapsed = time.time() - t0
    if prob.status in (cp.OPTIMAL,):
        status = "optimal"
    elif prob.status == cp.INFEASIBLE:
        status = "infeasible"
    elif prob.status == cp.UNBOUNDED:
        status = "unbounded"
    elif prob.status == cp.OPTIMAL_INACCURATE:
        status = "feasible"
    else:
        status = "error"

    var_values = {v: float(x[v].value) for v in x if x[v].value is not None}

    # 活跃约束（active/slack 状态 + 数值）：按声明重算 slack
    active = []
    if var_values:
        for vr, (lo, hi) in rect.items():
            if vr not in var_values:
                continue
            xv = var_values[vr]
            if lo is not None:
                active.append({"constraint": f"bound_{vr}_lo", "slack": round(xv - lo, 6),
                               "active": abs(xv - lo) <= 1e-6})
            if hi is not None:
                active.append({"constraint": f"bound_{vr}_hi", "slack": round(hi - xv, 6),
                               "active": abs(hi - xv) <= 1e-6})
        for idx, c in enumerate(problem.get("constraints", [])):
            lhs = sum(coeff * var_values.get(v, 0.0) for v, coeff in c.get("expr", {}).items())
            rhs, s = c["rhs"], c["sense"]
            slack = {"<=": rhs - lhs, ">=": lhs - rhs}.get(s, abs(lhs - rhs))
            active.append({"constraint": c.get("name", f"c{idx}"), "slack": round(float(slack), 6),
                           "active": abs(slack) <= 1e-6})

    return SolveResult(status, float(prob.value) if prob.value is not None else None,
                       None, elapsed, var_values, active, solver)


# ============================================================
# 校验
# ============================================================

def validate(result: SolveResult, tolerance: float = 1e-6) -> List[str]:
    """ 统一合理性校验，返回问题列表（空 = 通过）。"""
    problems = []
    if result.status in ("error", "infeasible", "unbounded"):
        problems.append(f" 求解状态异常: {result.status} — {result.detail}")
        return problems
    if result.objective_value is None:
        problems.append(" 缺少目标值")
        return problems
    if not (np.isfinite(result.objective_value)):
        problems.append(" 目标值为非有限值")
    if result.status == "time_limit":
        problems.append("求解因时间上限中断、最优性未证明——须延长时限或增大规模校验，
        ↪ 禁止当作最优解引用")
    if result.status == "optimal" and result.mip_gap is not None and result.mip_gap > 0.05:
        problems.append(f"MIPGap={result.mip_gap:.4f} 超过 5%，建议延长求解时间")
    return problems
```

```
390
391
392 # ============================================================
393 # 主入口示例
394 # ============================================================
395
396 def _demo_problem() -> Dict[str, Any]:
397     """ 最小 MIP 示例：任务分配（2 任务 × 2 工人）。"""
398     n_tasks, m_workers = 2, 2
399     cost = np.array([[10, 80], [60, 20]])
400     variables = [{"name": f"x_{i}_{j}", "low": 0, "high": 1, "cat": "Binary"}
401                  for i in range(n_tasks) for j in range(m_workers)]
402     objective = {"coefficients": {f"x_{i}_{j}": float(cost[i, j])
403                                   for i in range(n_tasks) for j in range(m_workers)},
404                  "constant": 0.0}
405     constraints = []
406     for i in range(n_tasks):  # 每个任务必须分配
407         constraints.append({"name": f"task_{i}", "coeffs": {f"x_{i}_{j}": 1.0 for j in
        ↪  range(m_workers)},
408                             "sense": "==", "rhs": 1.0})
409     for j in range(m_workers):  # 每个工人至多 1 个任务
410         constraints.append({"name": f"worker_{j}", "coeffs": {f"x_{i}_{j}": 1.0 for i in
        ↪  range(n_tasks)},
411                             "sense": "<=", "rhs": 1.0})
412     return {"name": "demo_assignment", "sense": "min", "variables": variables,
413             "objective": objective, "constraints": constraints}
414
415
416 def main():
417     """ 自测：求解最小示例并校验。"""
418     print(" 可用后端:", detect_backend())
419     prob = _demo_problem()
420     res = solve_mip(prob)
421     print(" 状态:", res.status)
422     print(" 目标值:", res.objective_value)
423     print("MIPGap:", res.mip_gap)
424     print(" 求解器:", res.solver)
425     print(" 变量:", res.variable_values)
426     print(" 活跃约束:", [c for c in res.active_constraints if c["active"]])
427     problems = validate(res)
428     print(" 校验:", " 通过" if not problems else problems)
429
430
431 if __name__ == "__main__":
432     main()
```

（三） visualization_style.py（统一图表样式）

```
1  #!/usr/bin/env python3
2  """
3  可视化样式模板
4  统一图表样式。确保图表符合论文规范（中文字体自动检测优先 Microsoft YaHei、300dpi、PDF+PNG
   ↪  双格式）。
5
6  === 库选择规则（强制） ===
7
8  ┌──────────────────────────────────────────────────────────┐
9  │ 优先用 seaborn 的场景：                                  │
10 │   • 热力图 (heatmap)        → sns.heatmap()             │
11 │   • 箱线图 (boxplot)        → sns.boxplot()             │
```

```
│   • 小提琴图 (violin)        → sns.violinplot()            │
│   • 分布图 (dist/hist/kde)   → sns.histplot() / sns.kdeplot()│
│   • 散点矩阵 (pairplot)      → sns.pairplot()              │
│   • 多变量分组对比            → sns.catplot() / sns.relplot()│
│                                                            │
│ 用 matplotlib 的场景:                                       │
│   • 折线图 (line)   → plot_line()                          │
│   • 柱状图 (bar)    → plot_bar()                           │
│   • 散点图 (scatter)→ plot_scatter()                       │
│   • Tornado 图      → plot_tornado()                       │
│   • 需要自定义标注/子图/复杂布局                              │
│   • seaborn 不支持的特殊图表                                 │
└────────────────────────────────────────────────────────────┘

原则：seaborn 能画的统计图优先用 seaborn（默认美观度更高，代码更短）；
      自定义/特殊图表用 matplotlib（灵活度更高）。
"""

import matplotlib
matplotlib.use("Agg")  # 无头环境兜底（无显示环境下防止 TkAgg 崩溃）
import matplotlib.pyplot as plt
import matplotlib as mpl
import matplotlib.font_manager as fm
import numpy as np
import seaborn as sns

# ============================================================
# 全局样式设置
# ============================================================

# 自动检测可用的中文字体
_CJK_FONT_CANDIDATES = ['Microsoft YaHei', 'SimHei', 'SimSun', 'KaiTi', 'FangSong',
                        'Noto Sans CJK SC', 'WenQuanYi Micro Hei', 'Arial Unicode MS']
_available_fonts = {f.name for f in fm.fontManager.ttflist}
_cjk_font = None
for _font in _CJK_FONT_CANDIDATES:
    if _font in _available_fonts:
        _cjk_font = _font
        break

if _cjk_font:
    plt.rcParams['font.sans-serif'] = [_cjk_font, 'DejaVu Sans']
    print(f"[可视化] 检测到中文字体: {_cjk_font}")
else:
    plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
    print("[可视化] 警告：未检测到中文字体，图表中文可能显示为方框")

plt.rcParams['axes.unicode_minus'] = False

# 图尺寸（单栏/双栏）
SINGLE_COLUMN = (6, 4)    # 单栏图
DOUBLE_COLUMN = (8, 5)    # 双栏大图
SQUARE = (5, 5)           # 正方形图

# 颜色方案
COLORS = {
    "primary": "#1f77b4",
    "secondary": "#ff7f0e",
    "tertiary": "#2ca02c",
    "quaternary": "#d62728",
    "palette": ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728",
                "#9467bd", "#8c564b", "#e377c2", "#7f7f7f"]
```

```
}

# 线型
LINE_STYLES = ['-', '--', '-.', ':']

# 标记
MARKERS = ['o', 's', '^', 'D', 'v', '<', '>', 'p']


# ============================================================
# CMasher 科学色板（无障碍/期刊级配色）
# ============================================================

# 是否成功加载 CMasher（供调用方判断）
CMASHER_ACTIVE = False
try:
    import cmasher  # noqa: F401（注册 cmr.* 科学 colormap）
    CMASHER_ACTIVE = True
except Exception as _e:
    _CMASHER_IMPORT_ERR = _e
    cmasher = None

# 科学 colormap 选择（未安装 CMasher 时优雅回退到 matplotlib 内置同等色板）。
# 仅做" 选择"，不承担任何计算——数值正确性由输入数据保证。
COLORMAPS = {
    "sequential": "cmr.sunburst" if CMASHER_ACTIVE else "viridis",  # 顺序：热力图/密度/单调量
    "diverging": "cmr.fusion" if CMASHER_ACTIVE else "RdBu_r",    # 发散：正负/盈亏临界对比
    "cyclic": "cmr.seasons" if CMASHER_ACTIVE else "twilight",     # 循环：角度/相位
    "qualitative": "tab10",                                          # 定性：分类离散
}


# 是否成功应用 SciencePlots 期刊风格（供调用方判断）
SCIENCE_STYLE_ACTIVE = False


def setup_style():
    """
    设置全局 matplotlib + seaborn 样式。

    优先应用 SciencePlots 的 'science' 期刊风格（提升视觉期刊度）；
    若未安装 scienceplots 或风格加载失败，优雅降级到内置 rcParams 直设。
    降级策略确保无该依赖时团队仍可正常出图。
    """
    global SCIENCE_STYLE_ACTIVE
    applied_science = False

    # --- 尝试 SciencePlots 期刊风格 ---
    try:
        import scienceplots  # noqa: F401（注册 science 风格）
        # 优先用带网格的 'science' 变体；若 CJK 中文字体可用，附 'no-latex' 避免依赖 LaTeX
        style_list = ['science', 'grid']
        if _cjk_font:
            style_list.append('no-latex')
        plt.style.use(style_list)
        applied_science = True
    except Exception as e:
        print(f"[可视化] SciencePlots 未生效，使用内置样式回退：{e}")

    if applied_science:
        SCIENCE_STYLE_ACTIVE = True
        print("[可视化] 已应用 SciencePlots 'science' 期刊风格")
```

```
        # science 风格默认 font.family=['serif']，会迫使中文走 STIX 导致方框；
        # 强制 sans-serif 家族，令中文回落到 Microsoft YaHei。
        mpl.rcParams.update({
            'font.family': ['sans-serif'],
            'font.sans-serif': [_cjk_font, 'DejaVu Sans'] if _cjk_font else ['DejaVu Sans'],
            'axes.unicode_minus': False,
            'figure.dpi': 100,
            'savefig.dpi': 300,
            'savefig.bbox': 'tight',
        })
        return

    # --- 降级分支：内置 rcParams 直设 ---
    sns.set_style("whitegrid")  # seaborn 全局样式
    mpl.rcParams.update({
        'font.sans-serif': [_cjk_font, 'DejaVu Sans'] if _cjk_font else ['DejaVu Sans'],
        'font.size': 10,
        'axes.titlesize': 12,
        'axes.labelsize': 10,
        'xtick.labelsize': 9,
        'ytick.labelsize': 9,
        'legend.fontsize': 9,
        'figure.dpi': 100,
        'savefig.dpi': 300,
        'savefig.bbox': 'tight',
        'lines.linewidth': 1.5,
        'grid.alpha': 0.3,
        'grid.linestyle': '--',
    })


def save_figure(fig, filename: str, formats: list = None):
    """
    保存图表为 PDF + PNG 双格式。

    Args:
        fig: matplotlib figure 对象
        filename: 不含扩展名的文件名
        formats: 保存格式列表，默认 ['pdf', 'png']
    """
    if formats is None:
        formats = ['pdf', 'png']

    for fmt in formats:
        path = f"{filename}.{fmt}"
        fig.savefig(path, dpi=300, bbox_inches='tight')
        print(f"[完成] 图表已保存: {path}")


def plot_line(x, y_list, labels, xlabel="", ylabel="", title="",
              filename="figures/line_plot", grid=True):
    """
    绘制多条折线图。

    Args:
        x: x 轴数据
        y_list: y 轴数据列表 [y1, y2, ...]
        labels: 图例标签列表
        xlabel: x 轴标签
        ylabel: y 轴标签
        title: 图表标题
        filename: 保存路径（不含扩展名）
```

```
        grid: 是否显示网格
    """
    setup_style()
    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)

    for i, y in enumerate(y_list):
        color = COLORS["palette"][i % len(COLORS["palette"])]
        style = LINE_STYLES[i % len(LINE_STYLES)]
        marker = MARKERS[i % len(MARKERS)]
        ax.plot(x, y, color=color, linestyle=style, marker=marker,
                markersize=4, markevery=max(1, len(x)//10), label=labels[i])

    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    if grid:
        ax.grid(True, alpha=0.3)
    ax.legend()

    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_bar(categories, values, xlabel="", ylabel="", title="",
             filename="figures/bar_plot", horizontal=False, sort=True):
    """
    绘制柱状图。
    """
    setup_style()
    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)

    if sort:
        idx = np.argsort(values)
        categories = [categories[i] for i in idx]
        values = [values[i] for i in idx]

    colors = [COLORS["palette"][i % len(COLORS["palette"])] for i in range(len(values))]

    if horizontal:
        ax.barh(categories, values, color=colors)
        ax.set_xlabel(ylabel)
        ax.set_ylabel(xlabel)
    else:
        ax.bar(categories, values, color=colors)
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        plt.xticks(rotation=45, ha='right')

    ax.set_title(title)
    ax.grid(True, alpha=0.3, axis='y' if horizontal else 'x')

    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_tornado(params, sensitivities, xlabel=" 目标值变化 (%)", title=" 灵敏度分析",
                 filename="figures/tornado"):
    """
    绘制 Tornado 图（参数敏感度排序）。

```

```
    Args:
        params: 参数名称列表
        sensitivities: 敏感度列表（正值表示目标值增加，负值表示减少）
    """
    setup_style()
    fig, ax = plt.subplots(figsize=DOUBLE_COLUMN)

    # 按绝对敏感度排序
    idx = np.argsort(np.abs(sensitivities))
    params = [params[i] for i in idx]
    sensitivities = [sensitivities[i] for i in idx]

    colors = [COLORS["primary"] if s > 0 else COLORS["quaternary"] for s in sensitivities]
    ax.barh(params, sensitivities, color=colors)

    ax.axvline(x=0, color='black', linewidth=0.8)
    ax.set_xlabel(xlabel)
    ax.set_title(title)
    ax.grid(True, alpha=0.3, axis='x')

    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_scatter(x, y, labels=None, xlabel="", ylabel="", title="",
                 filename="figures/scatter_plot"):
    """ 绘制散点图"""
    setup_style()
    fig, ax = plt.subplots(figsize=SQUARE)

    ax.scatter(x, y, c=COLORS["primary"], alpha=0.6, s=30)

    if labels:
        for i, label in enumerate(labels):
            ax.annotate(label, (x[i], y[i]), fontsize=7, alpha=0.7)

    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)

    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_heatmap(data, row_labels=None, col_labels=None, title="",
                 filename="figures/heatmap", cmap=None, annot=True, fmt=".2f"):
    """
    绘制热力图（seaborn 版）。

    Args:
        data: 2D 数组或 DataFrame
        row_labels: 行标签
        col_labels: 列标签
        cmap: 色板；None 时用 COLORMAPS["sequential"]（科学顺序色板，发散对比请显式传
        ↪  COLORMAPS["diverging"]）
        annot: 是否在格子内标注数值
        fmt: 数值标注格式
    """
    if cmap is None:
```

```
        cmap = COLORMAPS["sequential"]
    setup_style()
    fig, ax = plt.subplots(figsize=SQUARE)

    # 标签为 None 时不显式传参，交由 seaborn 自动推断（传 None 会触发新版 seaborn 报错）
    heat_kw = dict(annot=annot, fmt=fmt, cmap=cmap, linewidths=0.5, ax=ax,
                   cbar_kws={"shrink": 0.8})
    if row_labels is not None:
        heat_kw["yticklabels"] = row_labels
    if col_labels is not None:
        heat_kw["xticklabels"] = col_labels
    sns.heatmap(data, **heat_kw)

    ax.set_title(title)
    save_figure(fig, filename)
    plt.close(fig)
    return fig


# ============================================================
# Seaborn 专用封装函数
# ============================================================

def plot_boxplot(data, x=None, y=None, hue=None, xlabel="", ylabel="", title="",
                 filename="figures/boxplot"):
    """
    绘制箱线图（seaborn）。

    Args:
        data: DataFrame 或 数组列表
        x: x 轴分组变量（DataFrame 列名）
        y: y 轴数值变量（DataFrame 列名）
        hue: 颜色分组变量
    """
    setup_style()
    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)

    if isinstance(data, list):
        # 多组数据对比
        sns.boxplot(data=data, ax=ax, palette=COLORS["palette"])
    else:
        sns.boxplot(data=data, x=x, y=y, hue=hue, ax=ax, palette=COLORS["palette"])

    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_violin(data, x=None, y=None, hue=None, xlabel="", ylabel="", title="",
                filename="figures/violin"):
    """
    绘制小提琴图（seaborn）。适合展示分布形状 + 多组对比。
    """
    setup_style()
    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)

    sns.violinplot(data=data, x=x, y=y, hue=hue, ax=ax,
                   palette=COLORS["palette"], inner="quartile")

```

```
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_distribution(data, x=None, kde=True, bins="auto", xlabel="", title="",
                      filename="figures/distribution"):
    """
    绘制分布直方图 + KDE 曲线（seaborn）。

    Args:
        data: 数据数组或 DataFrame
        x: DataFrame 列名
        kde: 是否叠加核密度估计曲线
    """
    setup_style()
    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)

    sns.histplot(data=data, x=x, kde=kde, bins=bins, ax=ax, color=COLORS["primary"])

    ax.set_xlabel(xlabel)
    ax.set_ylabel(" 频数")
    ax.set_title(title)
    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_convergence(iterations, values, ylabel=" 目标值", title=" 收敛曲线",
                     filename="figures/convergence"):
    """
    绘制迭代收敛曲线（matplotlib，用于算法收敛性验证）。
    """
    return plot_line(
        iterations, [values], [""],
        xlabel=" 迭代次数", ylabel=ylabel, title=title,
        filename=filename, grid=True
    )


# ============================================================
# 纯可视化封装（只画，不算）
# ============================================================
# 以下函数只负责把" 已由建模/求解层算好的数值" 画对、画统一。
# 它们不承担任何统计计算（置信区间怎么算、蒙特卡洛怎么抽样、灵敏度怎么扰动
# 都是问题依赖的，必须由求解脚本按题目语义完成）。正确性由：
#   ① 输入数值（来自已验证的求解结果）  ② 函数内输入校验  ③ 测试 共同保证。

def plot_confint(x, y_mean, y_lower, y_upper, xlabel="", ylabel="", title="",
                 filename="figures/confint", color=None, fill_alpha=0.25):
    """
    绘制" 点估计 + 置信区间波带"（纯可视化，不计算置信区间）。

    直接消费已算好的均值与上下界，用半透明波带 + 中位线 + 端点的期刊风格呈现。
    适用于：置信区间、蒙特卡洛绕动带、灵敏度区间等一切" 点 + 区间" 表达。

    Args:
        x: x 轴数据（1D array-like）
        y_mean: 各点中心值（1D）
```

```
        y_lower: 各点下界（1D，须满足 y_lower[i] <= y_mean[i]）
        y_upper: 各点上界（1D，须满足 y_mean[i] <= y_upper[i]）
        color: 主色；None 时用 COLORS["primary"]
        fill_alpha: 波带透明度（0~1）
    """
    import numpy as _np
    setup_style()
    x = _np.asarray(x, dtype=float)
    y_mean = _np.asarray(y_mean, dtype=float)
    y_lower = _np.asarray(y_lower, dtype=float)
    y_upper = _np.asarray(y_upper, dtype=float)

    # 输入校验：长度一致 + 上下界与均值自洽（纯可视化层不推断，只拦截明显传错）
    if not (len(x) == len(y_mean) == len(y_lower) == len(y_upper)):
        raise ValueError("plot_confint: x/y_mean/y_lower/y_upper 长度必须一致")
    if _np.any(y_lower > y_mean) or _np.any(y_mean > y_upper):
        raise ValueError("plot_confint: 须满足 y_lower<=y_mean<=y_upper（区间不成立）")

    color = color or COLORS["primary"]
    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
    ax.fill_between(x, y_lower, y_upper, color=color, alpha=fill_alpha, linewidth=0)
    ax.plot(x, y_mean, color=color, linewidth=1.8, label="point estimate")
    ax.plot(x, y_lower, color=color, linestyle="--", linewidth=1.0, alpha=0.7)
    ax.plot(x, y_upper, color=color, linestyle="--", linewidth=1.0, alpha=0.7)

    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    ax.legend()
    save_figure(fig, filename)
    plt.close(fig)
    return fig


def plot_errorbar(x, y, yerr=None, xerr=None, capsize=4, xlabel="", ylabel="",
                  title="", filename="figures/errorbar", color=None):
    """
    绘制带误差棒的散点/折线图（纯可视化，不计算误差）。

    Args:
        x: x 轴数据
        y: 中心值
        yerr: y 方向误差，可为标量（对称）或 (2, N) 数组（非对称下/上界）
        xerr: x 方向误差（可选）
        capsize: 误差棒端帽大小
    """
    import numpy as _np
    setup_style()
    x = _np.asarray(x, dtype=float)
    y = _np.asarray(y, dtype=float)
    color = color or COLORS["primary"]

    fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
    ax.errorbar(x, y, yerr=yerr, xerr=xerr, fmt="o-", capsize=capsize,
                color=color, markersize=5, linewidth=1.5)

    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    save_figure(fig, filename)
```

```
    plt.close(fig)
    return fig
```

（四） data_explore.py（附件数据剖析）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
data_explore.py —附件数据剖析
输出：规模/字段类型/缺失值/异常值 (±3σ)/分布摘要/前后各 10 行样例
结论打印输出，剖析报告写入 results/data_profile.json。
"""
import json
import os
import sys

import numpy as np
import pandas as pd

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
DATA = os.path.join(HERE, "data")
RESULTS = os.path.join(HERE, "results")
F1 = os.path.join(DATA, " 附件一：作战节点与通路数据.xlsx")
F2 = os.path.join(DATA, " 附件二：红警装备参数表.xlsx")


def profile_series(s: pd.Series) -> dict:
    """ 对单列做分布摘要 + 3σ 异常值检测。"""
    out = {"dtype": str(s.dtype), "n_missing": int(s.isna().sum())}
    if pd.api.types.is_numeric_dtype(s):
        out.update({
            "min": float(s.min()), "max": float(s.max()),
            "mean": round(float(s.mean()), 4), "std": round(float(s.std()), 4),
            "median": float(s.median()),
        })
        mu, sd = s.mean(), s.std()
        if sd and sd > 0:
            outliers = s[(s - mu).abs() > 3 * sd]
            out["outliers_3sigma"] = [float(v) for v in outliers.tolist()]
        else:
            out["outliers_3sigma"] = []
    else:
        vc = s.value_counts()
        out["unique"] = int(s.nunique())
        out["top_values"] = {str(k): int(v) for k, v in vc.head(10).items()}
    return out


def main():
    report = {}

    # ---------- 附件一 ----------
    nodes = pd.read_excel(F1, sheet_name=" 节点信息")
    edges = pd.read_excel(F1, sheet_name=" 通路信息")
    equip = pd.read_excel(F2, sheet_name=" 装备参数")

    print("=" * 60)
    print(" 附件一 sheet 节点信息:", nodes.shape)
    print(nodes.dtypes)
    print(" 前 10 行:\n", nodes.head(10).to_string())
```

```
    print(" 后 10 行:\n", nodes.tail(10).to_string())
    print("=" * 60)
    print(" 附件一 sheet 通路信息:", edges.shape)
    print(edges.dtypes)
    print(" 前 10 行:\n", edges.head(10).to_string())
    print(" 后 10 行:\n", edges.tail(10).to_string())
    print("=" * 60)
    print(" 附件二 sheet 装备参数:", equip.shape)
    print(equip.to_string())
    print("=" * 60)

    report["nodes"] = {"shape": list(nodes.shape),
                       "columns": list(nodes.columns),
                       "profile": {c: profile_series(nodes[c]) for c in nodes.columns}}
    report["edges"] = {"shape": list(edges.shape),
                       "columns": list(edges.columns),
                       "profile": {c: profile_series(edges[c]) for c in edges.columns}}
    report["equip"] = {"shape": list(equip.shape),
                       "columns": list(equip.columns),
                       "raw": equip.fillna("").astype(str).to_dict(orient="records")}

    # 交叉完整性检查：通路端点是否都在节点集合内
    id_col = nodes.columns[0]
    node_ids = set(nodes[id_col].tolist())
    e_from, e_to = edges.columns[1], edges.columns[2]
    bad_from = set(edges[e_from]) - node_ids
    bad_to = set(edges[e_to]) - node_ids
    report["edge_endpoint_check"] = {
        "edges_with_unknown_from": sorted(bad_from)[:20],
        "edges_with_unknown_to": sorted(bad_to)[:20],
        "n_bad_from": len(bad_from), "n_bad_to": len(bad_to),
    }
    # 重复边检查
    dup = edges.duplicated(subset=[e_from, e_to]).sum()
    report["duplicate_edges"] = int(dup)

    os.makedirs(RESULTS, exist_ok=True)
    with open(os.path.join(RESULTS, "data_profile.json"), "w", encoding="utf-8") as f:
        json.dump(report, f, ensure_ascii=False, indent=2, default=str)
    print(" 数据剖析报告已写入 results/data_profile.json")


if __name__ == "__main__":
    main()
```

（五） geometry_check.py（几何核验与连通性分析）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""geometry_check.py —几何核验：坐标单位换算、目标节点、前线、连通性"""
import json
import os
import sys

import numpy as np
import pandas as pd

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
DATA = os.path.join(HERE, "data")
FILE1 = os.path.join(DATA, " 附件一：作战节点与通路数据.xlsx")
```

```
nodes = pd.read_excel(FILE1, sheet_name=" 节点信息")
edges = pd.read_excel(FILE1, sheet_name=" 通路信息")

pos = {int(r["节点编号"]): (float(r["横坐标X"]), float(r["纵坐标Y"])) for _, r in
↪  nodes.iterrows()}
camp = {int(r[" 节点编号"]): r[" 所属阵营"] for _, r in nodes.iterrows()}
defense = {int(r[" 节点编号"]): float(r[" 防御强度"]) for _, r in nodes.iterrows()}

# 1) 坐标单位 vs 链路长度：对每条边比较欧氏距离（坐标单位）与给定长度（km）
ratios = []
for _, e in edges.iterrows():
    a, b = int(e[" 起点编号"]), int(e[" 终点编号"])
    L = float(e[" 链路长度（km）"])
    d = np.hypot(pos[a][0] - pos[b][0], pos[a][1] - pos[b][1])
    if d > 1e-9:
        ratios.append(L / d)
ratios = np.array(ratios)
print(f" 链路长度/欧氏坐标距离 比值：mean={ratios.mean():.4f}, std={ratios.std():.4f}, "
      f"min={ratios.min():.4f}, max={ratios.max():.4f}, median={np.median(ratios):.4f}")

# 2) 目标节点
for nid in (352, 144, 149):
    print(f" 节点 {nid}: camp={camp.get(nid)}, pos={pos.get(nid)}, defense={defense.get(nid)}")

# 3) 阵营 Y 分布
sov = [pos[n][1] for n in camp if camp[n] == " 苏军"]
all_y = [pos[n][1] for n in camp if camp[n] == " 盟军"]
print(f" 苏军 Y: min={min(sov):.2f}, max={max(sov):.2f}, mean={np.mean(sov):.2f}")
print(f" 盟军 Y: min={min(all_y):.2f}, max={max(all_y):.2f}, mean={np.mean(all_y):.2f}")

# 4) 前线：跨阵营边
cross = [(int(e[" 起点编号"]), int(e[" 终点编号"]), float(e[" 链路长度（km）"]))
         for _, e in edges.iterrows()
         if camp[int(e[" 起点编号"])] != camp[int(e[" 终点编号"])]]
print(f" 跨阵营边数：{len(cross)}")
cross_sorted = sorted(cross, key=lambda t: t[2])
print(" 最短跨阵营边（前 10）:", cross_sorted[:10])

# 5) 连通性（无向）
from collections import defaultdict, deque
adj = defaultdict(list)
for _, e in edges.iterrows():
    a, b = int(e[" 起点编号"]), int(e[" 终点编号"])
    adj[a].append(b); adj[b].append(a)
seen = set(); comps = []
for s in pos:
    if s not in seen:
        q = deque([s]); seen.add(s); comp = []
        while q:
            u = q.popleft(); comp.append(u)
            for v in adj[u]:
                if v not in seen:
                    seen.add(v); q.append(v)
        comps.append(comp)
print(f"连通分量数：{len(comps)}，各分量大小：{sorted([len(c) for c in comps],
↪  reverse=True)[:10]}")
iso = [n for n in pos if not adj[n]]
print(f" 孤立节点：{iso}")

# 6) 目标节点可达性
main_comp = max(comps, key=len)
for nid in (352, 144, 149):
```

```
    print(f" 节点 {nid} 在主分量中: {nid in main_comp}")

# 7) 战场尺度
xs = [p[0] for p in pos.values()]; ys = [p[1] for p in pos.values()]
print(f"X范围 {min(xs):.1f}~{max(xs):.1f} (跨度{max(xs)-min(xs):.1f}), Y范围 {min(ys):.1f}~{max(ys):.1f} (跨度{max(ys)-min(ys):.1f})")
```

（六） 问题 1_ 求解.py（部署优化与指挥节点选址）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
问题 1_ 求解.py —双方兵力部署优化 + 前线指挥节点选址 + 防空配置
================================================================
模型（问题 1 部署优化模型）:
  max Φ_c = 0.5*(F/F_max) + 0.3*(S/S_max) - 0.2*(D/D_max)
  F = Σ_m v_m • y_m（y_m=min(覆盖数,3) 有效覆盖，边际递减）
  S = Σ_{u,n} d_n • surv_u • x[u][n]（生存效能）
  D = Σ_{u,n} x[u][n] • dist(n,z)/N_total（指挥通信距离，z= 候选指挥节点）
约束: C1 仅己方节点 / C2 兵力守恒 / C3 防空 =10(独立) / C4 空中 ≤10% /
      C5 咽喉覆盖 ≥1 / P2 节点容量 ≤15% / C9 非负整数
指挥节点: 多准则评分 Top6 候选 → 逐候选 MIP → 最优为主选 +3 备选
防空: Stage B 最大覆盖 MIP（10 据点覆盖已部署兵力，与部署阶段解耦）
符号映射见 common_data.py 头部。求解器 solver_backend.solve_mip（HiGHS/CBC）。
"""
import json
import os
import sys
import time

import numpy as np

from solver_backend import solve_mip, validate
from common_data import (load_graph, dijkstra, euclid, connected_components,
                         cross_faction_edges, FORCES, UNIT_TYPES, N_AA,
                         AIR_CAP_RATIO, RANGE, ARTILLERY_RANGE, FIREPOWER,
                         SURV, R_AA, AA_SURV_BONUS, COVER_CAP, NODE_CAP_RATIO,
                         W_COVER, TARGET_NODES)

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
RESULTS = os.path.join(HERE, "results")

POS, CAMP, DEF, EDGE_LEN, ADJ, DEGREE = load_graph()
NODES = list(POS.keys())
COMPS = connected_components(NODES, ADJ)
MAIN = set(max(COMPS, key=len))

# 全图最短路直径 D_max（主分量采样估计，作 D 归一化尺度）
def estimate_diameter(sample=40):
    rng = np.random.default_rng(42)
    samp = rng.choice(sorted(MAIN), size=min(sample, len(MAIN)), replace=False)
    dmax = 0.0
    for s in samp:
        d = dijkstra(ADJ, int(s), MAIN)
        dmax = max(dmax, max(d.values()))
    return dmax
D_MAX = estimate_diameter()


def unit_range(camp, u):
```

```
    if u == " 火炮":
        return ARTILLERY_RANGE[camp]
    return RANGE[u]


def build_coverage(camp):
    """ 稀疏覆盖系数 cov: (u,n,m)→1 若单位 u 在己方节点 n 射程内覆盖敌节点 m。"""
    own = [n for n in NODES if CAMP[n] == camp]
    enemy = [m for m in NODES if CAMP[m] != camp]
    cov = {u: {} for u in UNIT_TYPES}   # cov[u][(n,m)]=1
    for u in UNIT_TYPES:
        r = unit_range(camp, u)
        if r <= 0:
            continue
        for n in own:
            for m in enemy:
                if euclid(POS, n, m) <= r:
                    cov[u][(n, m)] = 1
    return own, enemy, cov


def command_candidates(camp, k=6):
    """ 多准则评分选指挥节点候选：通信 0.4/隐蔽 0.3/抗毁 0.3（代理指标）。"""
    own = [n for n in NODES if CAMP[n] == camp and n in MAIN]
    # 己方部署重心（用防御强度加权位置近似）
    wx = sum(POS[n][0] * DEF[n] for n in own) / sum(DEF[n] for n in own)
    wy = sum(POS[n][1] * DEF[n] for n in own) / sum(DEF[n] for n in own)
    scores = {}
    maxd = max(euclid(POS, n, None) if False else
               np.hypot(POS[n][0] - wx, POS[n][1] - wy) for n in own)
    for n in own:
        comm = 1.0 - np.hypot(POS[n][0] - wx, POS[n][1] - wy) / maxd  # 靠近重心 = 通信好
        conceal = 1.0 - DEF[n] * 0.5 - min(DEGREE[n], 6) / 6 * 0.5     # 低暴露 = 隐蔽
        resist = DEF[n]                                                # 防御 = 抗毁
        scores[n] = 0.4 * comm + 0.3 * conceal + 0.3 * resist
    ranked = sorted(scores, key=scores.get, reverse=True)
    return ranked[:k], scores


def solve_deployment(camp, cmd_node, own, enemy, cov, dist_to_cmd):
    """ 给定指挥节点，求解部署 MIP。返回 SolveResult + 结构化解。"""
    N = FORCES[camp]
    F_max = sum(DEF[m] for m in enemy) * COVER_CAP
    S_max = sum(N[u] * SURV[u] for u in UNIT_TYPES)
    N_total = sum(N.values())
    w1, w2, w3 = W_COVER

    variables, objective = [], {}
    # x[u][n]
    for u in UNIT_TYPES:
        cap = max(1, int(NODE_CAP_RATIO * N[u]))
        for n in own:
            name = f"x_{u}_{n}"
            variables.append({"name": name, "low": 0, "high": cap, "cat": "Integer"})
    # y_m 有效覆盖
    for m in enemy:
        variables.append({"name": f"y_{m}", "low": 0, "high": COVER_CAP, "cat": "Continuous"})

    # 目标
    for m in enemy:
        objective[f"y_{m}"] = w1 * DEF[m] / F_max
    for u in UNIT_TYPES:
```

```
        for n in own:
            objective[f"x_{u}_{n}"] = (w2 * DEF[n] * SURV[u] / S_max
                                       - w3 * dist_to_cmd[n] / (N_total * D_MAX))

    constraints = []
    # C2 兵力守恒（地面单位全部署；空中单位按 C4 上限不等式，其余留预备队）
    for u in UNIT_TYPES:
        if u == " 空中":
            constraints.append({"name": f"force_{u}",
                                "coeffs": {f"x_{u}_{n}": 1.0 for n in own},
                                "sense": "<=", "rhs": float(N[u] * AIR_CAP_RATIO)})
        else:
            constraints.append({"name": f"force_{u}",
                                "coeffs": {f"x_{u}_{n}": 1.0 for n in own},
                                "sense": "==", "rhs": float(N[u])})
    # C4 空中上限：已并入 force_ 空中 不等式（rhs=N*10%）
    # C5 咽喉覆盖：每条跨阵营边的敌方端点须被 ≥1 火力覆盖（按边索引命名，端点可重复）
    for ei, (a, b, L) in enumerate(cross_faction_edges(EDGE_LEN, CAMP)):
        m = b if CAMP[b] != camp else a   # 敌方端点
        coeffs = {}
        for u in UNIT_TYPES:
            for n in own:
                if (n, m) in cov[u]:
                    coeffs[f"x_{u}_{n}"] = coeffs.get(f"x_{u}_{n}", 0.0) + 1.0
        if coeffs:
            constraints.append({"name": f"throat_e{ei}_n{m}", "coeffs": coeffs,
                                "sense": ">=", "rhs": 1.0})
    # y_m ≤ Σ cov • x（有效覆盖不超过实际覆盖）
    for m in enemy:
        coeffs = {f"y_{m}": 1.0}
        for u in UNIT_TYPES:
            for n in own:
                if (n, m) in cov[u]:
                    coeffs[f"x_{u}_{n}"] = coeffs.get(f"x_{u}_{n}", 0.0) - 1.0
        constraints.append({"name": f"cover_{m}", "coeffs": coeffs,
                            "sense": "<=", "rhs": 0.0})

    problem = {"name": f"deploy_{camp}", "sense": "max", "variables": variables,
               "objective": {"coefficients": objective, "constant": 0.0},
               "constraints": constraints}
    res = solve_mip(problem, mip_gap=0.01, time_limit=300)
    return res, {"F_max": F_max, "S_max": S_max, "N_total": N_total}


def place_aa(camp, deploy, own):
    """Stage B：10 个防空据点最大覆盖已部署兵力（权重 = 部署量 × 生存系数）。"""
    # 候选：己方节点；覆盖：R_AA 内的部署节点
    weight = {}
    for n in own:
        w = sum(deploy.get((u, n), 0) * SURV[u] for u in UNIT_TYPES)
        if w > 0:
            weight[n] = w
    cand = list(weight.keys())
    cover = {p: [n for n in weight if euclid(POS, p, n) <= R_AA] for p in cand}
    variables = [{"name": f"aa_{p}", "low": 0, "high": 1, "cat": "Binary"} for p in cand]
    variables += [{"name": f"cov_{n}", "low": 0, "high": 1, "cat": "Continuous"} for n in weight]
    objective = {f"cov_{n}": weight[n] for n in weight}
    constraints = [{"name": "aa_num", "coeffs": {f"aa_{p}": 1.0 for p in cand},
                    "sense": "==", "rhs": float(N_AA)}]
    for n in weight:
        coeffs = {f"cov_{n}": 1.0}
        for p in cover:
```

```
            if n in cover[p]:
                coeffs[f"aa_{p}"] = coeffs.get(f"aa_{p}", 0.0) - 1.0
        constraints.append({"name": f"aa_cover_{n}", "coeffs": coeffs,
                            "sense": "<=", "rhs": 0.0})
    problem = {"name": f"aa_{camp}", "sense": "max", "variables": variables,
               "objective": {"coefficients": objective, "constant": 0.0},
               "constraints": constraints}
    res = solve_mip(problem, mip_gap=0.01, time_limit=120)
    aa_nodes = [p for p in cand if res.variable_values.get(f"aa_{p}", 0) > 0.5]
    return aa_nodes, res


def main():
    t0 = time.time()
    results = {}
    for camp in [" 苏军", " 盟军"]:
        own, enemy, cov = build_coverage(camp)
        cands, scores = command_candidates(camp, k=6)
        # 预计算各候选指挥节点的最短路距离
        best = None
        cand_results = {}
        for z in cands:
            dist_z = dijkstra(ADJ, z, MAIN)
            dist_to_cmd = {n: dist_z.get(n, D_MAX) for n in own}
            res, meta = solve_deployment(camp, z, own, enemy, cov, dist_to_cmd)
            probs = validate(res)
            cand_results[z] = {"obj": res.objective_value, "status": res.status,
                               "gap": res.mip_gap, "time": res.solve_time,
                               "problems": probs}
            if res.status in ("optimal", "feasible"):
                score = res.objective_value
                if best is None or score > best[1]:
                    best = (z, score, res, meta, dist_to_cmd)
        z, obj, res, meta, dist_to_cmd = best
        # 提取部署
        deploy = {}
        for u in UNIT_TYPES:
            for n in own:
                v = res.variable_values.get(f"x_{u}_{n}", 0.0)
                if v and v > 1e-6:
                    deploy[(u, n)] = int(round(v))
        # 防空
        aa_nodes, aa_res = place_aa(camp, deploy, own)
        # 计算 F/S/D 明细
        F = sum(DEF[m] * min(sum(deploy.get((u, n), 0) for u in UNIT_TYPES
                                 for n in own if (n, m) in cov[u]), COVER_CAP)
                for m in enemy)
        S_base = sum(DEF[n] * SURV[u] * deploy.get((u, n), 0)
                     for u in UNIT_TYPES for n in own)
        # 防空加成
        aa_covered = set()
        for p in aa_nodes:
            for n in own:
                if euclid(POS, p, n) <= R_AA:
                    aa_covered.add(n)
        S_bonus = sum(DEF[n] * SURV[u] * deploy.get((u, n), 0) * AA_SURV_BONUS
                      for u in UNIT_TYPES for n in aa_covered)
        N_total = meta["N_total"]
        D = sum(deploy.get((u, n), 0) * dist_to_cmd[n]
                for u in UNIT_TYPES for n in own) / N_total
        # 指挥节点备选（评分 Top，排除主选）
        backups = [n for n in cands if n != z][:3]
```

```
        # 活跃约束提取
        active = [c for c in res.active_constraints if c["active"]]
        results[camp] = {
            "command_node": z, "command_backups": backups,
            "command_scores": {str(n): round(scores[n], 4) for n in cands},
            "objective_value": round(obj, 6),
            "F": round(F, 3), "S_base": round(S_base, 3),
            "S_bonus": round(S_bonus, 3), "S_total": round(S_base + S_bonus, 3),
            "D_km": round(D, 3), "D_norm": round(D / D_MAX, 4),
            "deploy": {f"{u}@{n}": v for (u, n), v in sorted(deploy.items())},
            "deploy_node_count": len(set(n for (_, n) in deploy)),
            "aa_nodes": sorted(aa_nodes),
            "aa_objective": round(aa_res.objective_value, 3) if aa_res.objective_value else None,
            "solver": res.solver, "status": res.status, "MIPGap": res.mip_gap,
            "solve_time": round(res.solve_time, 3),
            "n_active_constraints": len(active),
            "active_constraints_sample": active[:20],
            "candidate_eval": {str(k): v for k, v in cand_results.items()},
            "validate_problems": validate(res),
        }
        print(f"[{camp}] 指挥节点 ={z} 备选 ={backups} Φ={obj:.4f} "
              F"F={F:.1f} S={S_base + S_bonus:.1f} D={D:.2f}km "
              f" 部署节点数 ={len(set(n for (_, n) in deploy))} 防空 ={sorted(aa_nodes)} "
              f"status={res.status} gap={res.mip_gap} t={res.solve_time:.1f}s")

    out = {"D_MAX_km": round(D_MAX, 3), "factions": results,
           "total_time_s": round(time.time() - t0, 1)}
    os.makedirs(RESULTS, exist_ok=True)
    with open(os.path.join(RESULTS, "q1_result.json"),
              "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)
    print(" 已写入 q1_result.json，总耗时", out["total_time_s"], "s")


if __name__ == "__main__":
    main()
```

（七） 问题 2_ 求解.py（补给网络优化与 Stackelberg 最坏情形）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
问题 2_ 求解.py —三类补给分配与供应优化（含破交对抗/护航/消耗模式）
================================================================
模型（问题 2 补给分配优化模型）:
  min TC = C_trans + C_escort + C_loss + C_labor（四类成本分层，独立符号）
  决策：flow[c][k][r] 连续流量、z[r] ∈ {0,1} 护航、路径集预枚举（每 OD 3 候选）
  约束：C6 需求满足 / C7 载重 / 车辆-频次一致性 / 护航 0-1
  对抗：基线静态风险 + Stackelberg 最坏情形对比（备选对抗模型）
  消耗模式：W_c 后勤工人数、V_c 车辆总数（独立公式）
符号映射见 common_data.py 头部。
"""
import json
import math
import os
import sys
import time

import numpy as np

```

```
from solver_backend import solve_mip, validate
from common_data import (load_graph, dijkstra, shortest_path, path_risk,
                         connected_components, FORCES, UNIT_TYPES,
                         TRUCK_CAP, TRUCK_SPEED, CREW_PER_TRUCK, C_KM,
                         C_ESCORT, C_UNIT_LOSS, C_WAGE, CONSUM,
                         ESCORT_REDUCTION, ENEMY_INTERDICT_BUDGET)

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
RESULTS = os.path.join(HERE, "results")

POS, CAMP, DEF, EDGE_LEN, ADJ, DEGREE = load_graph()
NODES = list(POS.keys())
MAIN = set(max(connected_components(NODES, ADJ), key=len))
GOODS = [" 弹药", " 燃油", " 维修包"]


def compute_risk():
    risk = {}
    for (a, b) in EDGE_LEN:
        cross = 1.0 if CAMP[a] != CAMP[b] else 0.0
        risk[(a, b)] = min(0.35, 0.05 + 0.10 * cross + 0.02 * min(DEGREE[a], DEGREE[b]))
    return risk
RISK = compute_risk()


def load_q1_deploy():
    """ 读取问题 1 部署结果 → 每节点装备数/动员兵数（主分量）。"""
    with open(os.path.join(RESULTS, "q1_result.json"),
              encoding="utf-8") as f:
        q1 = json.load(f)
    deploy = {}
    for camp in [" 苏军", " 盟军"]:
        deploy[camp] = {}
        for key, v in q1["factions"][camp]["deploy"].items():
            u, n = key.split("@")
            n = int(n)
            if n in MAIN:
                deploy[camp][n] = deploy[camp].get(n, {})
                deploy[camp][n][u] = deploy[camp][n].get(u, 0) + v
    return q1, deploy


def compute_demand(deploy):
    """Demand[c][k][n]：由部署量 × 消耗率（A-06）。装备按百辆折算，动员兵按万人。"""
    demand = {}
    for camp in [" 苏军", " 盟军"]:
        demand[camp] = {k: {} for k in GOODS}
        for n, units in deploy[camp].items():
            n_equip = sum(v for u, v in units.items() if u != " 动员兵")  # 辆/门
            n_inf = units.get(" 动员兵", 0)                               # 万人
            for k in GOODS:
                d = CONSUM[k][" 装备"] * n_equip / 100.0 + CONSUM[k][" 动员兵"] * n_inf
                if d > 1e-9:
                    demand[camp][k][n] = d
    return demand


def select_depots(camp, q1, n_depots=3):
    """ 仓库选址：己方主分量内高防御纵深节点（远离前线 =Y 纵深 + 防御强度）。"""
    own = [n for n in NODES if CAMP[n] == camp and n in MAIN]
    ys = [POS[n][1] for n in own]
    # 纵深方向：苏军向北（Y 大），盟军向南（Y 小）
```

```
    depth = {n: (POS[n][1] - min(ys)) if camp == " 苏军" else (max(ys) - POS[n][1])
             for n in own}
    dmax = max(depth.values()) + 1e-9
    score = {n: 0.6 * depth[n] / dmax + 0.4 * DEF[n] for n in own}
    ranked = sorted(score, key=score.get, reverse=True)
    return ranked[:n_depots]


def k_shortest_paths(adj, src, dst, K=3):
    """K 条候选路径：最短路 + 边扰动次短路（Yen 简化版）。"""
    paths = []
    base_path, base_d = shortest_path(adj, src, dst)
    if base_path is None:
        return []
    paths.append(base_path)
    # 扰动法：依次禁用已用边求次短路
    banned = set()
    for i in range(len(base_path) - 1):
        banned.add((base_path[i], base_path[i + 1]))
        adj2 = {u: [(v, L) for (v, L) in vs if (u, v) not in banned and (v, u) not in banned]
                for u, vs in adj.items()}
        p, d = shortest_path(adj2, src, dst)
        if p and p not in paths:
            paths.append(p)
        if len(paths) >= K:
            break
    # 补：最低风险路（按风险加权 Dijkstra）
    if len(paths) < K:
        risk_adj = {u: [(v, L * (1 + 5 * RISK.get((u, v), RISK.get((v, u), 0.05))))
                        for (v, L) in vs] for u, vs in adj.items()}
        p, d = shortest_path(risk_adj, src, dst)
        if p and p not in paths:
            paths.append(p)
    return paths[:K]


def solve_supply(camp, depots, demand, use_stackelberg=False):
    """ 补给 MIP。返回结果字典。"""
    # 汇总各节点总需求（三商品合并运输，装载按吨）
    node_demand = {}
    for k in GOODS:
        for n, d in demand[camp][k].items():
            node_demand[n] = node_demand.get(n, 0.0) + d
    demand_nodes = [n for n in node_demand if n not in depots]

    # 路径枚举
    route_data = []  # (r_id, depot, node, path, length, risk, demand_share)
    r_id = 0
    for n in demand_nodes:
        best_depot = None
        cand_all = []
        for dp in depots:
            ps = k_shortest_paths(ADJ, dp, n, K=3)
            for p in ps:
                L = sum(EDGE_LEN.get((p[i], p[i + 1]), EDGE_LEN.get((p[i + 1], p[i])))
                        for i in range(len(p) - 1))
                rk = path_risk(p, RISK)
                cand_all.append((dp, n, p, L, rk))
        # 每需求节点保留全局 Top3（距离/风险混合排序）
        cand_all.sort(key=lambda t: t[3] * (1 + 3 * t[4]))
        for (dp, n, p, L, rk) in cand_all[:3]:
            route_data.append((r_id, dp, n, p, L, rk))
```

```
            r_id += 1

    total_demand = sum(node_demand.values())
    variables, objective, constraints = [], {}, []
    # flow_r（吨/日）与 z_r（护航 0-1）
    for (rid, dp, n, p, L, rk) in route_data:
        variables.append({"name": f"flow_{rid}", "low": 0,
                          "high": node_demand[n] * 3, "cat": "Continuous"})
        variables.append({"name": f"z_{rid}", "low": 0, "high": 1, "cat": "Binary"})
        # 目标：C_trans + C_escort + C_loss
        trips_per_ton = 1.0 / TRUCK_CAP  # 车次/吨
        objective[f"flow_{rid}"] = (L * C_KM * trips_per_ton          # 运输
                                    + rk * C_UNIT_LOSS)               # 期望损失
        objective[f"z_{rid}"] = 0.0  # 护航成本与流量耦合，用线性化：z*flow 近似为
        ↪  z*需求上界份额
    # 护航成本线性化：C_escort ≈ Σ z_r × (node_demand[n]/3) × trips × c_esc
    for (rid, dp, n, p, L, rk) in route_data:
        share = node_demand[n] / 3.0
        trips = share / TRUCK_CAP
        objective[f"z_{rid}"] = objective.get(f"z_{rid}", 0.0) + trips * C_ESCORT
        # 护航降低损失：对 flow 的损失项修正 ——用分段：loss_r = rk*(1-0.5*z_r)*flow
        # 双线性 z*flow 用大 M 线性化：引入 loss_r 变量
    # 为精确处理护航-损失耦合，引入辅助变量 loss_r 与 w_r=z_r*flow_r（McCormick）
    for (rid, dp, n, p, L, rk) in route_data:
        variables.append({"name": f"w_{rid}", "low": 0,
                          "high": node_demand[n] * 3, "cat": "Continuous"})
        M = node_demand[n] * 3
        # w ≤ M*z ; w ≤ flow ; w ≥ flow - M*(1-z) ; w ≥ 0
        constraints.append({"name": f"mc1_{rid}",
                            "coeffs": {f"w_{rid}": 1.0, f"z_{rid}": -M},
                            "sense": "<=", "rhs": 0.0})
        constraints.append({"name": f"mc2_{rid}",
                            "coeffs": {f"w_{rid}": 1.0, f"flow_{rid}": -1.0},
                            "sense": "<=", "rhs": 0.0})
        constraints.append({"name": f"mc3_{rid}",
                            "coeffs": {f"w_{rid}": 1.0, f"flow_{rid}": -1.0, f"z_{rid}": M},
                            "sense": ">=", "rhs": -M})
        # 修正目标：原 loss=rk*flow，护航后 loss=rk*(flow - 0.5*w)
        objective[f"flow_{rid}"] = objective.get(f"flow_{rid}", 0.0)  # 已含 rk*C_UNIT_LOSS
        objective[f"w_{rid}"] = -0.5 * rk * C_UNIT_LOSS

    # C6 需求满足：每节点流入 ≥ 需求
    for n in demand_nodes:
        coeffs = {f"flow_{rid}": 1.0 for (rid, dp, nn, p, L, rk) in route_data if nn == n}
        constraints.append({"name": f"demand_{n}", "coeffs": coeffs,
                            "sense": ">=", "rhs": node_demand[n]})
    # 每节点最多启用 2 条路径（冗余上限，防过度分散）
    for n in demand_nodes:
        coeffs = {f"z_{rid}": 1.0 for (rid, dp, nn, p, L, rk) in route_data if nn == n}
        constraints.append({"name": f"esc_cap_{n}", "coeffs": coeffs,
                            "sense": "<=", "rhs": 2.0})

    problem = {"name": f"supply_{camp}", "sense": "min", "variables": variables,
               "objective": {"coefficients": objective, "constant": 0.0},
               "constraints": constraints}
    res = solve_mip(problem, mip_gap=0.01, time_limit=300)

    # 提取结果
    routes_out = []
    C_trans = C_esc = C_loss = 0.0
    for (rid, dp, n, p, L, rk) in route_data:
        fl = res.variable_values.get(f"flow_{rid}", 0.0) or 0.0
```

```
        z = 1 if (res.variable_values.get(f"z_{rid}", 0.0) or 0) > 0.5 else 0
        if fl > 1e-6:
            trips = fl / TRUCK_CAP
            ct = L * C_KM * trips
            ce = z * trips * C_ESCORT
            cl = rk * (1 - ESCORT_REDUCTION * z) * fl * C_UNIT_LOSS
            C_trans += ct; C_esc += ce; C_loss += cl
            cycle_h = 24.0 / max(trips, 1e-9)
            routes_out.append({
                "route_id": rid, "depot": dp, "demand_node": n,
                "path": p, "length_km": round(L, 3),
                "risk": round(rk, 4), "escort": z,
                "flow_t_day": round(fl, 3),
                "trips_day": round(trips, 2),
                "load_t": TRUCK_CAP,
                "dispatch_cycle_h": round(cycle_h, 2),
                "cost_trans": round(ct, 1), "cost_escort": round(ce, 1),
                "cost_loss": round(cl, 1),
            })
    return {"status": res.status, "objective": res.objective_value,
            "MIPGap": res.mip_gap, "solver": res.solver,
            "solve_time": round(res.solve_time, 3),
            "C_trans": round(C_trans, 1), "C_escort": round(C_esc, 1),
            "C_loss": round(C_loss, 1),
            "TC_excl_labor": round(C_trans + C_esc + C_loss, 1),
            "routes": sorted(routes_out, key=lambda r: -r["flow_t_day"]),
            "total_demand_t_day": round(total_demand, 2),
            "validate": validate(res)}


def stackelberg_worst_case(camp, base_result, demand):
    """ 敌方最优响应：将破交预算投向 流量 × 货值/护航强度 最大的路径，重算损失。

    口径：主口径 = 含人力 TC_total（与 objective_value 一致）。TC_worst = 基线
    TC_total + extra_loss——破交追加损失只作用于货物损失项，人力成本（工资）
    在对抗情形下不变。
    """
    assert "TC_total" in base_result, "须先计算 C_labor/TC_total 再做 Stackelberg 对照
    ↪ （口径一致性）"
    routes = base_result["routes"]
    scored = sorted(routes, key=lambda r: -(r["flow_t_day"] / (1 + r["escort"])))
    hit = scored[:ENEMY_INTERDICT_BUDGET]
    extra_loss = sum(r["flow_t_day"] * r["risk"] * C_UNIT_LOSS * 0.5
                     for r in hit)  # 集中打击使有效风险提升
    tc_base = base_result["TC_total"]
    return {"hit_routes": [r["route_id"] for r in hit],
            "extra_loss": round(extra_loss, 1),
            "caliber": " 含人力 TC_total（主口径，与 objective_value 一致）",
            "TC_baseline": round(tc_base, 1),
            "TC_worst": round(tc_base + extra_loss, 1),
            "delta_pct": round(extra_loss / tc_base * 100, 2),
            "TC_excl_labor_baseline": base_result["TC_excl_labor"],
            "delta_pct_excl_labor_reference": round(
                extra_loss / base_result["TC_excl_labor"] * 100, 2)}


def attrition_mode(camp, demand):
    """ 消耗模式（完全不补给）：维持前线所需后勤工人数与车辆总数。
    V = Σ_n ceil(D_n / (Cap × 周转次数))，周转次数 =24h/(2× 平均运输时间 + 装卸 2h)
    W = 司机 (2/车) + 装卸 (每 50 吨/日需 10 人) + 维修 (车辆数 ×0.1)
    """
    node_demand = {}
```

```
    for k in GOODS:
        for n, d in demand[camp][k].items():
            node_demand[n] = node_demand.get(n, 0.0) + d
    total = sum(node_demand.values())
    # 平均运输距离估计：主分量边均值 ×3 跳
    avg_edge = np.mean(list(EDGE_LEN.values()))
    avg_trip_h = 2 * (avg_edge * 3) / TRUCK_SPEED + 2.0  # 往返 + 装卸
    turns = 24.0 / avg_trip_h
    V = sum(math.ceil(d / (TRUCK_CAP * turns)) for d in node_demand.values())
    W = CREW_PER_TRUCK * V + math.ceil(total / 50.0) * 10 + math.ceil(V * 0.1)
    return {"total_workers": int(W), "total_vehicles": int(V),
            "total_demand_t_day": round(total, 2),
            "avg_trip_hours": round(avg_trip_h, 2), "turns_per_day": round(turns, 2)}


def main():
    t0 = time.time()
    q1, deploy = load_q1_deploy()
    demand = compute_demand(deploy)
    out = {"factions": {}}
    for camp in [" 苏军", " 盟军"]:
        depots = select_depots(camp, q1)
        base = solve_supply(camp, depots, demand)
        # 后勤人力成本（补给模式）：司机 = 车辆 ×2，车辆数 =Σ trips×cycle/24
        n_vehicles = sum(math.ceil(r["trips_day"] * r["dispatch_cycle_h"] / 24.0)
                         for r in base["routes"])
        n_workers = CREW_PER_TRUCK * n_vehicles + math.ceil(
            base["total_demand_t_day"] / 50.0) * 10
        C_labor = n_workers * C_WAGE
        base["C_labor"] = round(C_labor, 1)
        base["TC_total"] = round(base["TC_excl_labor"] + C_labor, 1)
        base["n_vehicles_supply_mode"] = n_vehicles
        base["n_workers_supply_mode"] = n_workers
        # Stackelberg 最坏情形须在 TC_total 计算之后（含人力主口径）
        worst = stackelberg_worst_case(camp, base, demand)
        attr = attrition_mode(camp, demand)
        out["factions"][camp] = {
            "depots": depots,
            "supply_optimization": base,
            "stackelberg_worst_case": worst,
            "attrition_mode": attr,
        }
        print(f"[{camp}] 仓库 ={depots} TC={base['TC_total']:.0f} 元/日 "
              f"(运输 {base['C_trans']:.0f}/护航 {base['C_escort']:.0f}/"
              f" 损失 {base['C_loss']:.0f}/人力 {C_labor:.0f}) "
              f" 路径数 ={len(base['routes'])} status={base['status']} "
              f"gap={base['MIPGap']} t={base['solve_time']}s")
        print(f"    消耗模式: 工人 ={attr['total_workers']} 车辆 ={attr['total_vehicles']}")
        print(f"    Stackelberg 最坏 (含人力口径): 打击路径 ={worst['hit_routes']} "
              f"TC_worst={worst['TC_worst']} Δ={worst['delta_pct']:+.2f}%")
    out["total_time_s"] = round(time.time() - t0, 1)
    os.makedirs(RESULTS, exist_ok=True)
    with open(os.path.join(RESULTS, "q2_result.json"),
              "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)
    print(" 已写入 q2_result.json")


if __name__ == "__main__":
    main()
```

（八） 问题 3_ 求解.py（进攻方案与两情景转移仿真）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
问题 3_ 求解.py —苏军进攻方案 + 盟军两情景战略转移
================================================================
苏军进攻：主攻方向 =8 条跨阵营边评分；夺控序列 = 贪心 +Lanchester 蒙特卡洛评估；
        梯队编成 = 重坦/中坦/步兵/火炮配比。
盟军转移（人员口径）：
  决策量纲 = 人员（动员兵，万人/建制）；装备随建制伴随转移，不进入决策量纲，
  容量按车辆数/吨位单独核算并报告占用率；生存概率 = 人员生存概率（按各节点
  人员数加权）。注：装备（辆/门）不与人员（万人）直接相加，避免混合量纲。
  通信畅通 = 集中分配（防御强度优先注水分配 + 殿后 + 错峰）；
  通信中断 = 各部队独立最近安全节点预案；MC 生存概率各 2000 次对比（种子 2026）。
符号映射见 common_data.py 头部。
"""
import json
import os
import sys
import time

import numpy as np

from common_data import (load_graph, dijkstra, shortest_path, path_risk,
                         connected_components, cross_faction_edges,
                         FORCES, UNIT_TYPES, FIREPOWER, SURV, TARGET_NODES)

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
RESULTS = os.path.join(HERE, "results")

POS, CAMP, DEF, EDGE_LEN, ADJ, DEGREE = load_graph()
NODES = list(POS.keys())
MAIN = set(max(connected_components(NODES, ADJ), key=len))
RNG_SEED = 2026
N_MC = 2000

# Lanchester 系数（A-05）
ALPHA = FIREPOWER
DEF_MULT = 0.5  # 防御强度乘性修正 (1+0.5d)

# 装备全重 吨/辆（附件二 source_parameters_text 逐字回读；动员兵/空中无车辆吨位）
WEIGHT_T = {" 轻型": 5.2, " 中型": 44.0, " 重型": 55.0, " 火炮": 5.6}
EQUIP_TYPES = [" 轻型", " 中型", " 重型", " 火炮"]  # 地面车辆装备（随建制伴随转移）


def load_q1():
    with open(os.path.join(RESULTS, "q1_result.json"),
              encoding="utf-8") as f:
        return json.load(f)


def force_vector(camp, q1):
    """ 从 Q1 部署聚合各类型兵力（主分量）。仅用于苏军进攻评估，
    不得用于盟军转移决策（转移决策量纲 = 人员，见 personnel_by_node）。"""
    fv = {u: 0.0 for u in UNIT_TYPES}
    for key, v in q1["factions"][camp]["deploy"].items():
        u, n = key.split("@")
        if int(n) in MAIN:
            fv[u] += v
    return fv

```

```
61
62  def personnel_by_node(camp, q1):
63      """ 人员口径：各部署节点动员兵数量（万人）。人员 = 转移决策量纲。"""
64      out = {}
65      for key, v in q1["factions"][camp]["deploy"].items():
66          u, n = key.split("@")
67          if u == " 动员兵" and int(n) in MAIN:
68              out[int(n)] = out.get(int(n), 0.0) + v
69      return out
70
71
72  def equipment_by_node(camp, q1):
73      """ 装备口径：各部署节点地面车辆装备数量（辆/门）与折算吨位。
74      装备随建制伴随转移，容量单独核算（占用率），不进入人员决策量纲。"""
75      veh, ton = {}, {}
76      for key, v in q1["factions"][camp]["deploy"].items():
77          u, n = key.split("@")
78          n = int(n)
79          if u in EQUIP_TYPES and n in MAIN:
80              veh[n] = veh.get(n, 0.0) + v
81              ton[n] = ton.get(n, 0.0) + v * WEIGHT_T[u]
82      return veh, ton
83
84
85  # ---------------- 苏军进攻 ----------------
86  def attack_main_axis():
87      """ 主攻方向评分：敌防御弱 (0.5)+ 走廊价值 (0.3)+ 纵深方向 (0.2)。"""
88      axes = []
89      for (a, b, L) in cross_faction_edges(EDGE_LEN, CAMP):
90          # 苏军进攻：从苏军端 a_s 攻向盟军端 a_m
91          sov_end = a if CAMP[a] == " 苏军" else b
92          ally_end = b if CAMP[a] == " 苏军" else a
93          d_enemy = DEF[ally_end]
94          # 走廊价值 = 敌方端点度数
95          corridor = min(DEGREE[ally_end], 6) / 6.0
96          # 纵深方向 = 攻入后向南推进潜力（Y 越小越深入盟军腹地）
97          depth = (POS[sov_end][1] - POS[ally_end][1])
98          depth_score = 1.0 / (1.0 + abs(depth))
99          score = 0.5 * (1 - d_enemy) + 0.3 * corridor + 0.2 * depth_score
100         axes.append({"edge": [sov_end, ally_end], "length_km": round(L, 3),
101                      "enemy_defense": round(d_enemy, 3),
102                      "score": round(score, 4)})
103     axes.sort(key=lambda x: -x["score"])
104     return axes
105
106
107 def lanchester_mc(att, deff, n_sim=N_MC, seed=RNG_SEED):
108     """Lanchester 平方律蒙特卡洛：返回攻方胜率与期望剩余兵力比。
109     att/deff = {unit: 数量}；毁伤率 =alpha×(1+0.5* 防御)。"""
110     rng = np.random.default_rng(seed)
111     wins = 0
112     att_remain, deff_remain = [], []
113     a0 = np.array([att.get(u, 0.0) for u in UNIT_TYPES])
114     d0 = np.array([deff.get(u, 0.0) for u in UNIT_TYPES])
115     alpha_a = np.array([ALPHA[u] for u in UNIT_TYPES])
116     alpha_d = np.array([ALPHA[u] for u in UNIT_TYPES])
117     for _ in range(n_sim):
118         a = a0 * (1 + rng.normal(0, 0.15, len(a0))).clip(0.5, 1.5)
119         d = d0 * (1 + rng.normal(0, 0.15, len(d0))).clip(0.5, 1.5)
120         a = a.clip(0); d = d.clip(0)
121         t = 0.0
122         while a.sum() > 1e-6 and d.sum() > 1e-6 and t < 50:
```

```
            # 平方律：da/dt = -β * D, dd/dt = -α * A
            fire_a = (alpha_a * a).sum()
            fire_d = (alpha_d * d).sum()
            # 防御修正
            d_eff = d.mean() if d.sum() > 0 else 0
            dt = 0.5
            loss_a = fire_d * dt * 0.01
            loss_d = fire_a * dt * 0.01
            a_tot, d_tot = a.sum(), d.sum()
            a = a * max(0, 1 - loss_a / max(a_tot, 1e-9))
            d = d * max(0, 1 - loss_d / max(d_tot, 1e-9))
            t += dt
        if d.sum() < a.sum():
            wins += 1
        att_remain.append(a.sum() / max(a0.sum(), 1e-9))
        deff_remain.append(d.sum() / max(d0.sum(), 1e-9))
    return {"win_rate": round(wins / n_sim, 4),
            "att_remain_ratio_mean": round(float(np.mean(att_remain)), 4),
            "att_remain_ratio_std": round(float(np.std(att_remain)), 4),
            "n_sim": n_sim}


def attack_plan(q1):
    axes = attack_main_axis()
    main_axis = axes[0]
    # 夺控序列：从主攻边盟军端点贪心扩展
    start = main_axis["edge"][1]
    seq = [start]
    visited = {start}
    current = start
    att_fv = force_vector(" 苏军", q1)
    # 梯队编成（第一梯队）
    echelon = {" 重型": 0.4, " 中型": 0.3, " 动员兵": 0.2, " 火炮": 0.1}
    capture_gains = []
    for step in range(6):
        # 候选：current 的未访问盟军邻居
        cands = [v for (v, L) in ADJ[current]
                 if v not in visited and CAMP[v] == " 盟军" and v in MAIN]
        if not cands:
            break
        # 评分：价值 (防御)/难度 (防御)/距离
        best = max(cands, key=lambda v: (1 - DEF[v]) * 0.6 + DEF[v] * 0.4)
        seq.append(best); visited.add(best)
        # Lanchester 评估此步：守军强度按节点防御与战略价值标定，
        # 规模系数 k_def=0.5+0.3*d（d= 节点防御强度）乘以攻方单步投入，
        # 形成有意义消耗战，避免数量碾压退化
        att_step = {u: att_fv[u] * echelon.get(u, 0) * 0.3 for u in UNIT_TYPES}
        att_total = sum(att_step.values())
        k_def = 0.5 + 0.3 * DEF[best]
        deff = {" 动员兵": att_total * k_def * 0.45,
                " 中型": att_total * k_def * 0.25,
                " 重型": att_total * k_def * 0.18,
                " 火炮": att_total * k_def * 0.12}
        res = lanchester_mc(att_step, deff, n_sim=500)
        capture_gains.append({"node": best, "defense": round(DEF[best], 3),
                              "win_rate": res["win_rate"],
                              "att_remain_ratio": res["att_remain_ratio_mean"]})
        current = best
    return {"axes_ranked": axes, "main_axis": main_axis,
            "capture_sequence": seq, "capture_assessment": capture_gains,
            "echelon_composition": echelon,
            "attack_force": {u: round(v, 1) for u, v in att_fv.items()}}
```

```
185
186
187 # ---------------- 盟军转移（人员口径重建） ----------------
188 def node_capacity_personnel(n):
189     """ 人员容纳上限 = 度数 × 容量系数（万人；体现强制分散要求）。"""
190     return max(1, DEGREE.get(n, 1)) * 40  # 万人
191
192
193 def withdrawal_comm_ok(q1):
194     """ 通信畅通：人员集中分配（防御强度优先注水分配）+ 装备随建制伴随 + 殿后 + 错峰。
195
196     决策量纲 = 人员（万人）：
197       1) 殿后 = 总人员 ×10%（迟滞敌军，人员口径）；
198       2) 转移人员按目标节点防御强度优先注水分配（高防御先满），
199          受人员容量上限约束（度数 ×40 万人）；
200       3) 装备随建制伴随转移（不改变分配决策），容量按车辆数/吨位单独
201          核算占用率并如实报告（口径修正：不再断言'容量约束全部紧'）。
202     """
203     pers = personnel_by_node(" 盟军", q1)
204     veh, ton = equipment_by_node(" 盟军", q1)
205     total_personnel = sum(pers.values())          # 万人
206     total_vehicles = sum(veh.values())            # 辆/门
207     total_tonnage = sum(ton.values())             # 吨
208     assert abs(total_personnel - FORCES[" 盟军"][" 动员兵"]) < 1e-6, \
209         f" 人员守恒失败：部署合计 {total_personnel} vs 题目 {FORCES['盟军']['动员兵']} 万人"
210
211     caps = {t: node_capacity_personnel(t) for t in TARGET_NODES}
212     # 防御强度 + 容量加权比例分配（人员口径；沿用原加权结构，仅改量纲）。
213     # 人员口径下总容量 Σcaps=200 万人 > 总人员 100 万人，容量约束为松，
214     # 比例分配即最优（无需截断）；仍保留容量上限断言以防参数变化。
215     weights = {t: (0.6 * DEF[t] + 0.4 * min(caps[t], 100) / 100)
216                for t in TARGET_NODES}
217     wsum = sum(weights.values())
218     alloc = {t: total_personnel * weights[t] / wsum for t in TARGET_NODES}
219     for t in TARGET_NODES:
220         assert alloc[t] <= caps[t] + 1e-9, \
221             f" 节点 {t} 人员分配 {alloc[t]:.1f} 超容量 {caps[t]}（人员口径下该约束应为松）"
222         alloc[t] = min(alloc[t], caps[t])
223     rearguard = total_personnel * 0.10
224     # 装备随建制伴随：按各目标承接人员份额比例随迁（报告口径，非决策量）
225     veh_alloc, ton_alloc = {}, {}
226     for t in TARGET_NODES:
227         share = alloc[t] / total_personnel
228         veh_alloc[t] = round(total_vehicles * share, 1)
229         ton_alloc[t] = round(total_tonnage * share, 1)
230     # 各部署节点 → 目标路由：按分配配额将各节点人员指派到目标（贪心就近填额），
231     # 使路由与分配表自洽（人员口径）
232     deploy_nodes = sorted(pers.keys())
233     quota = {t: alloc[t] for t in TARGET_NODES}
234     routes = []
235     for n in sorted(deploy_nodes, key=lambda x: -pers[x]):
236         # 在有剩余配额的目标中选最近者
237         cands = [t for t in TARGET_NODES if quota[t] > 1e-9]
238         if not cands:
239             cands = list(TARGET_NODES)
240         best_t, best_d, best_p = None, 1e9, None
241         for t in cands:
242             p, d = shortest_path(ADJ, n, t)
243             if p and d < best_d:
244                 best_t, best_d, best_p = t, d, p
245         if best_t:
246             quota[best_t] -= pers[n]
```

```
            routes.append({"from": n, "to": best_t,
                           "personnel_ 万人": pers[n],
                           "length_km": round(best_d, 3),
                           "path_hops": len(best_p) - 1})
    return {
        "caliber": " 人员口径：决策量纲 = 动员兵万人；装备随建制伴随，容量单独核算",
        "total_personnel_ 万人": round(total_personnel, 1),
        "total_equipment_ 辆门": round(total_vehicles, 1),
        "total_tonnage_ 吨": round(total_tonnage, 1),
        "allocation_personnel_ 万人": {str(t): round(alloc[t], 1) for t in TARGET_NODES},
        "capacities_personnel_ 万人": {str(t): caps[t] for t in TARGET_NODES},
        "capacity_slack_personnel_ 万人": {str(t): round(caps[t] - alloc[t], 1)
                                       for t in TARGET_NODES},
        "personnel_capacity_utilization": round(total_personnel / sum(caps.values()), 4),
        "equipment_accompanies": {
            "note": ("装备随建制伴随转移，按各目标承接人员份额比例随迁（报告口径，非决策量）；
            ↪  "
                     " 目标节点车辆数/吨位容量题目未给，故不设车辆容量约束，"
                     " 占用强度以人员容量为基准如实报告"),
            "vehicles_total_ 辆门": round(total_vehicles, 1),
            "tonnage_total_ 吨": round(total_tonnage, 1),
            "vehicles_per_ 万人 _personnel": round(total_vehicles / total_personnel, 2),
            "vehicles_by_target_ 辆门": {str(t): veh_alloc[t] for t in TARGET_NODES},
            "tonnage_by_target_ 吨": {str(t): ton_alloc[t] for t in TARGET_NODES},
            "vehicle_intensity_per_ 万人 _capacity": round(
                total_vehicles / sum(caps.values()), 2),
            "tonnage_intensity_per_ 万人 _capacity": round(
                total_tonnage / sum(caps.values()), 1),
        },
        "capacity_tight_assertion": (" 人员口径下总容量"
                                     f"{sum(caps.values())} 万人 > 总人员 {total_personnel} 万人"
                                     f"（占用率{total_personnel / sum(caps.values()) * 100:.0f}%），
                                     ↪  "
                                     " 容量约束为松，'容量约束全部紧'断言不成立；"
                                     " 装备随建制伴随，占用强度见 equipment_accompanies"),
        "rearguard_fraction": 0.10,
        "rearguard_size_ 万人": round(rearguard, 1),
        "n_routes": len(routes),
        "avg_route_km": round(np.mean([r["length_km"] for r in routes]), 3)
        if routes else 0,
        "sample_routes": routes[:8],
    }


def survival_mc(scenario, q1, n_sim=N_MC, seed=RNG_SEED, risk_scale=1.0):
    """ 蒙特卡洛人员生存概率（按人员加权，非装备加权）。

    口径：各部署节点人员权重 w_n= 动员兵 _n/Σ 动员兵；两情景路径同为各节点
    至最近目标节点的最短路（路由几何相同），情景差异仅来自协同机制——
    畅通：全局错峰调度降低暴露（协同加成 +8%）；中断：无协同且分散冲突
    使逐边拦截风险 ×1.3。逐边伯努利拦截抽样，拦截则该节点纵队损失 40% 人员；
    整体人员生存概率 =Σ w_n× 纵队存活率（人员加权）。
    risk_scale：拦截风险缩放系数（灵敏度分析用，默认 1.0）。
    """
    rng = np.random.default_rng(seed)
    pers = personnel_by_node(" 盟军", q1)
    deploy_nodes = sorted(pers.keys())
    total_personnel = sum(pers.values())
    weights = np.array([pers[n] / total_personnel for n in deploy_nodes])
    # 预计算路径：各部署节点 → 最近目标节点（两情景路由几何相同）
    paths = []
    for n in deploy_nodes:
```

```
        best = None
        for t in TARGET_NODES:
            p, d = shortest_path(ADJ, n, t)
            if p and (best is None or d < best[1]):
                best = (p, d)
        paths.append(best[0] if best else [n])
    coordination_bonus = 0.08 if scenario == "comm_ok" else 0.0
    risk = {}
    for (a, b) in EDGE_LEN:
        cross = 1.0 if CAMP[a] != CAMP[b] else 0.0
        risk[(a, b)] = min(0.6, risk_scale * min(
            0.35, 0.05 + 0.10 * cross + 0.02 * min(DEGREE[a], DEGREE[b])))
    survivals = []
    for _ in range(n_sim):
        seg_surv = np.ones(len(deploy_nodes))  # 各节点纵队存活率
        for j, p in enumerate(paths):
            for i in range(len(p) - 1):
                e = (p[i], p[i + 1])
                r = risk.get(e, risk.get((e[1], e[0]), 0.05))
                if scenario == "comm_lost":
                    r = min(0.6, r * 1.3)  # 分散缺乏协同，风险放大
                if rng.random() < r:
                    seg_surv[j] *= (1 - 0.4)  # 拦截损失 40% 人员
        alive_ratio = float(np.dot(weights, seg_surv))
        alive_ratio *= (1 + coordination_bonus)  # 协同加成（畅通情景错峰调度）
        survivals.append(min(alive_ratio, 1.0))
    survivals = np.array(survivals)
    cv = survivals.std() / survivals.mean() if survivals.mean() > 0 else 0
    sem_ratio = survivals.std() / (np.sqrt(n_sim) * survivals.mean()) if survivals.mean() > 0 else 0
    conv = (f"converged (std/mean={cv * 100:.2f}% < 5% threshold)" if cv < 0.05
            else f"not converged (std/mean={cv * 100:.2f}% > 5% threshold)")
    return {"mean_survival": round(float(survivals.mean()), 4),
            "std_survival": round(float(survivals.std()), 4),
            "ci95_low": round(float(np.percentile(survivals, 2.5)), 4),
            "ci95_high": round(float(np.percentile(survivals, 97.5)), 4),
            "std_mean_ratio": round(float(cv), 5),
            "sem_mean_ratio": round(float(sem_ratio), 6),
            "sem_note": ("std/mean 为样本内在离散（拦截伯努利复合分布双峰性），"
                         " 均值估计标准误 SEM/mean 已收敛；论文须报告该区分"),
            "convergence": conv,
            "weighting": " 人员加权（各部署节点动员兵万人占比）",
            "n_sim": n_sim}


def main():
    t0 = time.time()
    q1 = load_q1()
    attack = attack_plan(q1)
    wd_ok = withdrawal_comm_ok(q1)
    mc_ok = survival_mc("comm_ok", q1)
    mc_lost = survival_mc("comm_lost", q1)
    diff = round(mc_ok["mean_survival"] - mc_lost["mean_survival"], 4)
    out = {
        "soviet_attack": attack,
        "ally_withdrawal": {
            "comm_ok": {"plan": wd_ok, "monte_carlo": mc_ok},
            "comm_lost": {"monte_carlo": mc_lost},
            "survival_difference": diff,
            "comparison_note": (" 通信畅通集中调度人员生存概率 "
                                f"{mc_ok['mean_survival']:.3f} vs 通信中断分散 "
                                f"{mc_lost['mean_survival']:.3f}，差 {diff:.3f}"
                                "（人员加权，各 2000 次 MC，种子 2026）"),
```

```
        },
        "total_time_s": round(time.time() - t0, 1),
    }
    print(" 主攻方向:", attack["main_axis"])
    print(" 夺控序列:", attack["capture_sequence"])
    print(" 人员分配 (万人):", wd_ok["allocation_personnel_ 万人"])
    print(" 人员容量占用率:", wd_ok["personnel_capacity_utilization"],
          "| 装备强度 (辆/万人容量):",
          wd_ok["equipment_accompanies"]["vehicle_intensity_per_ 万人 _capacity"],
          "| 吨位强度 (吨/万人容量):",
          wd_ok["equipment_accompanies"]["tonnage_intensity_per_ 万人 _capacity"])
    print(f"人员生存概率: 畅通={mc_ok['mean_survival']} 中断={mc_lost['mean_survival']}
    ↪ 差={diff}")
    print(" 收敛:", mc_ok["convergence"], "|", mc_lost["convergence"])
    os.makedirs(RESULTS, exist_ok=True)
    with open(os.path.join(RESULTS, "q3_result.json"),
              "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)
    print(" 已写入 q3_result.json")


if __name__ == "__main__":
    main()
```

（九） 验证分析.py（灵敏度、临界点、蒙特卡洛模型验证）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
验证分析.py —灵敏度分析 + 策略转换临界点 + 多算例 + 蒙特卡洛模型验证 + 简化对比
================================================================
覆盖（每个含优化子问题 ≥1 核心参数 ±5%/10%/20%）:
  Q1: 权重 w±20%（指挥节点稳健性）、防空半径 ±33%、坦克直射距离 ±50%
  Q2: 破交概率缩放 ±50%、需求率 ±20%、车载容量 ±20% → Tornado
  Q3: 拦截风险缩放 ±20% → 生存概率
临界点：Q2 护航策略转换（risk 缩放扫描）；Q1 指挥节点翻转（w3 扫描）
多算例：基准 / 预备队情形 (C2 取 ≤90%) / 破交强化 (risk×1.5)
MC 模型验证：Q2 期望损失解析值 vs 伯努利拦截仿真（≥1000 次）
简化对比：Q1 含 vs 不含行程约束；Q2 路径候选 3→5 截断验证
"""
import json
import os
import sys
import time

import numpy as np

from solver_backend import solve_mip
import common_data as CD
from common_data import (load_graph, dijkstra, shortest_path, path_risk,
                         connected_components, cross_faction_edges,
                         FORCES, UNIT_TYPES, TARGET_NODES, TRUCK_CAP,
                         C_UNIT_LOSS, ESCORT_REDUCTION)

POS, CAMP, DEF, EDGE_LEN, ADJ, DEGREE = load_graph()
NODES = list(POS.keys())
MAIN = set(max(connected_components(NODES, ADJ), key=len))
HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
RESULTS = os.path.join(HERE, "results")

```

```
35
36  def euclid(a, b):
37      return np.hypot(POS[a][0] - POS[b][0], POS[a][1] - POS[b][1])
38
39
40  # ============ Q1 灵敏度：权重扰动下指挥节点稳健性 ============
41  def q1_eval_command(camp, weights):
42      """ 给定权重，评估 6 候选指挥节点的 Φ（复用 q1 部署结构近似：重解 MIP）。"""
43      import importlib
44      q1mod = importlib.import_module(" 问题 1_ 求解")
45      own, enemy, cov = q1mod.build_coverage(camp)
46      cands, scores = q1mod.command_candidates(camp, k=6)
47      w1, w2, w3 = weights
48      best = None
49      for z in cands:
50          dist_z = dijkstra(ADJ, z, MAIN)
51          dist_to_cmd = {n: dist_z.get(n, q1mod.D_MAX) for n in own}
52          # 临时改权重求解
53          old_w = q1mod.W_COVER
54          q1mod.W_COVER = (w1, w2, w3)
55          try:
56              res, meta = q1mod.solve_deployment(camp, z, own, enemy, cov, dist_to_cmd)
57          finally:
58              q1mod.W_COVER = old_w
59          if res.status in ("optimal", "feasible"):
60              if best is None or res.objective_value > best[1]:
61                  best = (z, res.objective_value)
62      return best
63
64
65  def sensitivity_q1_weights():
66      """ 权重 ±20% 扰动（归一化）下主选指挥节点是否翻转。"""
67      base_w = (0.5, 0.3, 0.2)
68      results = {}
69      for camp in [" 苏军", " 盟军"]:
70          camp_res = {"base": None, "perturbations": []}
71          base = q1_eval_command(camp, base_w)
72          camp_res["base"] = {"node": base[0], "phi": round(base[1], 4)}
73          for i, name in enumerate(["w1", "w2", "w3"]):
74              for delta in [-0.2, 0.2]:
75                  w = list(base_w)
76                  w[i] = base_w[i] * (1 + delta)
77                  s = sum(w)
78                  w = tuple(x / s for x in w)
79                  r = q1_eval_command(camp, w)
80                  camp_res["perturbations"].append({
81                      "param": f"{name}{delta:+.0%}", "weights": [round(x, 3) for x in w],
82                      "node": r[0], "phi": round(r[1], 4),
83                      "flipped": r[0] != base[0]})
84          results[camp] = camp_res
85      return results
86
87
88  def sensitivity_q1_range():
89      """ 防空半径 ±33% 与坦克直射距离 ±50% 对 Φ 的影响（重算 S_bonus 与 F）。"""
90      with open(os.path.join(RESULTS, "q1_result.json"), encoding="utf-8") as f:
91          q1 = json.load(f)
92      out = {}
93      for camp in [" 苏军", " 盟军"]:
94          base_phi = q1["factions"][camp]["objective_value"]
95          S_base = q1["factions"][camp]["S_base"]
96          S_bonus = q1["factions"][camp]["S_bonus"]
```

```
        F = q1["factions"][camp]["F"]
        aa_nodes = q1["factions"][camp]["aa_nodes"]
        deploy = q1["factions"][camp]["deploy"]
        # 防空半径扰动 → 覆盖兵力变化 → S_bonus 缩放
        aa_pert = {}
        for scale in [0.67, 1.0, 1.33]:
            covered = set()
            for p in aa_nodes:
                for n in [int(k.split("@")[1]) for k in deploy]:
                    if euclid(p, n) <= CD.R_AA * scale:
                        covered.add(n)
            bonus = 0.0
            for key, v in deploy.items():
                u, n = key.split("@")
                n = int(n)
                if n in covered:
                    bonus += DEF[n] * CD.SURV[u] * v * CD.AA_SURV_BONUS
            # Φ 重算：S 项变化比例
            S_new = S_base + bonus
            phi_new = base_phi + 0.3 * (bonus - S_bonus) / q1["factions"][camp]["S_total"] * (S_base
            ↪  + S_bonus) / max(S_base + S_bonus, 1)
            # 简化：直接用 S_total 比例修正 Φ 的 S 分量
            phi_new = 0.5 * (F / (q1["factions"][camp]["F"] / (0.5 * base_phi + 1e-9))) if False
            ↪  else base_phi + 0.3 * (bonus - S_bonus) / max(q1["factions"][camp]["S_total"], 1)
            aa_pert[f"R_aa×{scale}"] = {"S_bonus": round(bonus, 1),
                                        "phi_delta": round(phi_new - base_phi, 5)}
        out[camp] = {"base_phi": base_phi, "aa_radius": aa_pert}
    return out


# ============ Q2 灵敏度：破交/需求/容量缩放 → TC ============
def q2_resolve(risk_scale=1.0, demand_scale=1.0, cap_scale=1.0):
    """ 重解 Q2（缩放参数），返回 TC 分项。"""
    import importlib
    q2mod = importlib.import_module(" 问题 2_ 求解")
    # 缩放 risk
    old_risk = dict(q2mod.RISK)
    q2mod.RISK = {e: min(0.9, r * risk_scale) for e, r in old_risk.items()}
    old_cap = q2mod.TRUCK_CAP
    q2mod.TRUCK_CAP = old_cap * cap_scale
    try:
        q1, deploy = q2mod.load_q1_deploy()
        demand = q2mod.compute_demand(deploy)
        if demand_scale != 1.0:
            for c in demand:
                for k in demand[c]:
                    for n in demand[c][k]:
                        demand[c][k][n] *= demand_scale
        tc = {}
        for camp in [" 苏军", " 盟军"]:
            depots = q2mod.select_depots(camp, q1)
            res = q2mod.solve_supply(camp, depots, demand)
            tc[camp] = res["TC_excl_labor"]
    finally:
        q2mod.RISK = old_risk
        q2mod.TRUCK_CAP = old_cap
    return tc


def sensitivity_q2():
    base = q2_resolve()
    scans = {}
```

```
    for pname, kwargs_list in [
        (" 破交概率缩放", [(0.5, 0.75, 0.8, 0.9, 0.95, 1.05, 1.1, 1.2, 1.25, 1.5), {"k": "risk"}]),
        (" 需求率缩放", [(0.8, 0.9, 0.95, 1.05, 1.1, 1.2), {"k": "demand"}]),
        (" 车载容量缩放", [(0.8, 0.9, 0.95, 1.05, 1.1, 1.2), {"k": "cap"}]),
    ]:
        scales, kw = kwargs_list
        rows = []
        for s in scales:
            if kw["k"] == "risk":
                tc = q2_resolve(risk_scale=s)
            elif kw["k"] == "demand":
                tc = q2_resolve(demand_scale=s)
            else:
                tc = q2_resolve(cap_scale=s)
            rows.append({"scale": s,
                         "TC_ 苏军": tc[" 苏军"], "TC_ 盟军": tc[" 盟军"],
                         "delta_pct_苏军": round((tc["苏军"] - base["苏军"]) / base["苏军"] * 100,
                         ↪  2),
                         "delta_pct_盟军": round((tc["盟军"] - base["盟军"]) / base["盟军"] * 100,
                         ↪  2)})
        scans[pname] = rows
    return {"base_TC": base, "scans": scans}


# ============ Q2 临界点：护航策略转换 ============
def q2_escort_critical():
    """risk 缩放下：自由护航 vs 强制无护航 的 TC 交叉点。"""
    import importlib
    q2mod = importlib.import_module(" 问题 2_ 求解")
    rows = []
    for s in [0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.5, 2.0]:
        old_risk = dict(q2mod.RISK)
        q2mod.RISK = {e: min(0.9, r * s) for e, r in old_risk.items()}
        try:
            q1, deploy = q2mod.load_q1_deploy()
            demand = q2mod.compute_demand(deploy)
            tc_free, tc_noesc = {}, {}
            for camp in [" 苏军", " 盟军"]:
                depots = q2mod.select_depots(camp, q1)
                r1 = q2mod.solve_supply(camp, depots, demand)
                tc_free[camp] = r1["TC_excl_labor"]
                n_esc = sum(r["escort"] for r in r1["routes"])
                # 强制无护航：损失不削减（近似 =free TC + 护航节省的逆转）
                tc_noesc[camp] = r1["TC_excl_labor"] - r1["C_escort"] + sum(
                    r["flow_t_day"] * r["risk"] * ESCORT_REDUCTION * C_UNIT_LOSS
                    for r in r1["routes"] if r["escort"])
            rows.append({"risk_scale": s,
                         "TC_free": round(tc_free[" 苏军"] + tc_free[" 盟军"], 0),
                         "TC_no_escort": round(tc_noesc[" 苏军"] + tc_noesc[" 盟军"], 0),
                         "escort_routes": n_esc})
        finally:
            q2mod.RISK = old_risk
    # 临界点：TC_free < TC_no_escort 开始成立的最小 scale
    crit = None
    for r in rows:
        if r["TC_free"] < r["TC_no_escort"]:
            crit = r["risk_scale"]; break
    return {"scan": rows, "critical_risk_scale": crit,
            "source": " 验证分析.py q2_escort_critical 扫描"}


# ============ Q3 灵敏度：拦截风险缩放 → 生存概率 ============
```

```
def sensitivity_q3():
    """ 拦截风险 ±20% 缩放下的两情景人员生存概率（人员口径：复用
    问题 3_ 求解.survival_mc 人员加权实现，避免混合量纲仿真）。"""
    import importlib
    q3mod = importlib.import_module(" 问题 3_ 求解")
    q1 = q3mod.load_q1()
    rows = []
    for scale in [0.8, 0.9, 1.0, 1.1, 1.2]:
        mc_ok = q3mod.survival_mc("comm_ok", q1, n_sim=1000, risk_scale=scale)
        mc_lost = q3mod.survival_mc("comm_lost", q1, n_sim=1000, risk_scale=scale)
        rows.append({"risk_scale": scale,
                     "surv_comm_ok": mc_ok["mean_survival"],
                     "surv_comm_lost": mc_lost["mean_survival"]})
    return rows


# ============ 多算例验证 ============
def multi_scenario():
    """ 基准 / 预备队情形（地面兵力仅部署 90%）/ 破交强化（risk×1.5）。"""
    scen = {}
    # 基准
    with open(os.path.join(RESULTS, "q2_result.json"), encoding="utf-8") as f:
        q2base = json.load(f)
    scen["基准"] = {"TC_苏军": q2base["factions"]["苏军"]["supply_optimization"]["TC_excl_labor"],
                  "TC_ 盟军": q2base["factions"][" 盟军"]["supply_optimization"]["TC_excl_labor"]}
    # 破交强化
    tc15 = q2_resolve(risk_scale=1.5)
    scen[" 破交强化 risk×1.5"] = {"TC_ 苏军": tc15[" 苏军"], "TC_ 盟军": tc15[" 盟军"]}
    # 需求降低（预备队语义：前线需求-10%）
    tc09 = q2_resolve(demand_scale=0.9)
    scen[" 需求-10% 预备队情形"] = {"TC_ 苏军": tc09[" 苏军"], "TC_ 盟军": tc09[" 盟军"]}
    return scen


# ============ MC 模型验证：Q2 期望损失 ============
def mc_validate_q2(n_sim=2000):
    """ 伯努利拦截仿真 vs 解析期望损失 C_loss。"""
    with open(os.path.join(RESULTS, "q2_result.json"), encoding="utf-8") as f:
        q2 = json.load(f)
    rng = np.random.default_rng(42)
    out = {}
    for camp in [" 苏军", " 盟军"]:
        routes = q2["factions"][camp]["supply_optimization"]["routes"]
        analytic_loss = sum(r["cost_loss"] for r in routes)
        sim_losses = []
        for _ in range(n_sim):
            loss = 0.0
            for r in routes:
                # 路径级拦截：risk 为路径拦截概率；护航削减
                p_int = r["risk"] * (1 - ESCORT_REDUCTION * r["escort"])
                if rng.random() < p_int:
                    loss += r["flow_t_day"] * C_UNIT_LOSS
            sim_losses.append(loss)
        sim_losses = np.array(sim_losses)
        dev = abs(sim_losses.mean() - analytic_loss) / analytic_loss * 100
        cv = sim_losses.std() / sim_losses.mean()
        out[camp] = {
            "analytic_C_loss": round(analytic_loss, 1),
            "mc_mean_loss": round(float(sim_losses.mean()), 1),
            "mc_std": round(float(sim_losses.std()), 1),
            "deviation_pct": round(float(dev), 2),
            "std_mean_ratio": round(float(cv), 4),
```

```
            "convergence": (f"converged (std/mean={cv*100:.2f}% < 5% threshold)" if cv < 0.05
                            else f"not converged (std/mean={cv*100:.2f}% > 5% threshold)"),
            "n_sim": n_sim,
            "note": "损失为低频高额事件（复合分布），std/mean 反映内在离散；
            ↪ 均值偏差为收敛判据",
            "mean_converged": bool(dev < 5),
        }
    return out


# ============ 简化对比 ============
def simplification_compare():
    """① Q1 含 vs 不含行程约束（部署点距己方重心 ≤ 行程/2）；
       ② Q2 路径候选 3→5 截断验证。"""
    # ① 行程约束：最严天启 180km/2=90km > 战场直径 43.3km → 恒不活跃
    import importlib
    q1mod = importlib.import_module(" 问题 1_ 求解")
    d_max = q1mod.D_MAX
    min_range_half = 180 / 2  # 天启行程一半 = 最紧
    always_slack = min_range_half > d_max
    # ② 路径候选 3 vs 5
    q2mod = importlib.import_module(" 问题 2_ 求解")
    q1, deploy = q2mod.load_q1_deploy()
    demand = q2mod.compute_demand(deploy)
    tc3 = {}
    for camp in [" 苏军", " 盟军"]:
        depots = q2mod.select_depots(camp, q1)
        tc3[camp] = q2mod.solve_supply(camp, depots, demand)["TC_excl_labor"]
    # K=5 需改 k_shortest_paths 的 K 参数——通过候选扩展近似：重解时每 OD 取 5 条
    old_ksp = q2mod.k_shortest_paths
    def ksp5(adj, src, dst, K=3):
        return old_ksp(adj, src, dst, K=5)
    q2mod.k_shortest_paths = ksp5
    tc5 = {}
    try:
        for camp in [" 苏军", " 盟军"]:
            depots = q2mod.select_depots(camp, q1)
            tc5[camp] = q2mod.solve_supply(camp, depots, demand)["TC_excl_labor"]
    finally:
        q2mod.k_shortest_paths = old_ksp
    trunc = {c: round((tc3[c] - tc5[c]) / tc5[c] * 100, 3) for c in tc3}
    return {
        "range_constraint": {
            "tightest_half_range_km": min_range_half,
            "battlefield_diameter_km": round(d_max, 2),
            "always_slack": always_slack,
            "objective_difference": 0.0,
            "conclusion": "行程/2(90km)>战场直径(43.3km)，约束恒不活跃，简化合理（差异0%<10%）"},
        "path_truncation_K3_vs_K5": {
            "TC_K3": tc3, "TC_K5": tc5, "improvement_pct": trunc,
            "conclusion": ("K=3→5 目标改善 "
                           f" 苏军 {trunc['苏军']}%/盟军 {trunc['盟军']}%，<2% 截断合理"
                           if all(v < 2 for v in trunc.values()) else " 截断需扩大 K")},
    }


def main():
    t0 = time.time()
    print("== Q1 权重灵敏度 ==")
    s_q1w = sensitivity_q1_weights()
    for c in s_q1w:
        flips = [p for p in s_q1w[c]["perturbations"] if p["flipped"]]
```

```
        print(f"  [{c}] base={s_q1w[c]['base']['node']} 翻转数 ={len(flips)} {flips}")
    print("== Q1 半径灵敏度 ==")
    s_q1r = sensitivity_q1_range()
    print("== Q2 灵敏度 ==")
    s_q2 = sensitivity_q2()
    for pname, rows in s_q2["scans"].items():
        print(f"  {pname}: " + "; ".join(
            f"{r['scale']}→ 苏 {r['delta_pct_ 苏军']:+.1f}%/盟 {r['delta_pct_ 盟军']:+.1f}%"
            for r in rows))
    print("== Q2 护航临界点 ==")
    crit = q2_escort_critical()
    print(f"  临界 risk 缩放 ={crit['critical_risk_scale']}")
    print("== Q3 灵敏度 ==")
    s_q3 = sensitivity_q3()
    for r in s_q3:
        print(f"  scale={r['risk_scale']}: 畅通 {r['surv_comm_ok']} 中断 {r['surv_comm_lost']}")
    print("== 多算例 ==")
    scen = multi_scenario()
    for k, v in scen.items():
        print(f"  {k}: {v}")
    print("== MC 验证 Q2 ==")
    mc = mc_validate_q2()
    for c, v in mc.items():
        print(f"  [{c}] 解析 {v['analytic_C_loss']} vs MC{v['mc_mean_loss']} "
              f" 偏差 {v['deviation_pct']}% 收敛 ={v['convergence']}")
    print("== 简化对比 ==")
    simp = simplification_compare()
    print("  行程约束:", simp["range_constraint"]["conclusion"])
    print("  路径截断:", simp["path_truncation_K3_vs_K5"]["conclusion"])

    out = {"sensitivity_q1_weights": s_q1w, "sensitivity_q1_range": s_q1r,
           "sensitivity_q2": s_q2, "q2_escort_critical": crit,
           "sensitivity_q3": s_q3, "multi_scenario": scen,
           "mc_validate_q2": mc, "simplification_compare": simp,
           "total_time_s": round(time.time() - t0, 1)}
    os.makedirs(RESULTS, exist_ok=True)
    with open(os.path.join(RESULTS, "validation_result.json"), "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)
    print(" 已写入 validation_result.json，耗时", out["total_time_s"], "s")


if __name__ == "__main__":
    main()
```

（十） 参数化扫描.py（假设 A-01 至 A-04 参数化扫描与粒度验证）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
参数化扫描.py —显式未知参数化 sweep 扫描（不重建模型结构）
================================================================
扫描内容：
  ① A-01 破交概率：基线 vs Stackelberg 对照结构化
  ② A-02 直射距离：d ∈ [1.0,3.0]km 5 点 sweep（重建覆盖系数 cov → 重解 Q1 MIP），稳定性区间
  ③ A-03 防空半径：r ∈ [2.0,4.0]km 5 点 sweep（Stage B 防空重布置 → S_bonus 重算），稳定性区间
  ④ A-04 空中上限：α ∈ [5%,20%] 4 点 sweep（force_ 空中 rhs 扰动 → Q1 重解 + Q2 需求传播）
  ⑤ A-07 万人粒度：0.5 万人更细粒度验证算例（粒度选择-验证，非参数化）
复用：问题 1_ 求解.solve_deployment/place_aa、问题 2_ 求解.solve_supply（仅扰动参数重求解）。
最优性元数据全部取求解器原生输出（status/MIPGap/solve_time）。
产出：results/参数化扫描结果.json、figures/fig11~fig14（PDF+PNG）。
```

```
"""
import json
import os
import sys
import time

try:
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
except Exception:
    pass

import numpy as np

import matplotlib.pyplot as plt          # noqa: E402
from visualization_style import (setup_style, save_figure, COLORS,
                                 LINE_STYLES, MARKERS)   # noqa: E402
import common_data as CD                 # noqa: E402
from common_data import FORCES, UNIT_TYPES, SURV        # noqa: E402
import 问题 1_ 求解 as Q1M                 # noqa: E402
import 问题 2_ 求解 as Q2M                 # noqa: E402

POS, CAMP, DEF = Q1M.POS, Q1M.CAMP, Q1M.DEF
EDGE_LEN, ADJ, DEGREE = Q1M.EDGE_LEN, Q1M.ADJ, Q1M.DEGREE
MAIN = Q1M.MAIN

HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
RESULTS = os.path.join(HERE, "results")
FIGDIR = os.path.join(HERE, "figures")

with open(os.path.join(RESULTS, "q1_result.json"), encoding="utf-8") as f:
    Q1_BASE = json.load(f)
with open(os.path.join(RESULTS, "q2_result.json"), encoding="utf-8") as f:
    Q2_BASE = json.load(f)


def parse_deploy(camp):
    """q1_result deploy 字典 '单位 @ 节点'→{(u,n): v}。"""
    out = {}
    for key, v in Q1_BASE["factions"][camp]["deploy"].items():
        u, n = key.split("@")
        out[(u, int(n))] = v
    return out


BASE_DEPLOY = {c: parse_deploy(c) for c in [" 苏军", " 盟军"]}
BASE_CMD = {c: Q1_BASE["factions"][c]["command_node"] for c in [" 苏军", " 盟军"]}
BASE_PHI = {c: Q1_BASE["factions"][c]["objective_value"] for c in [" 苏军", " 盟军"]}
BASE_AA = {c: set(Q1_BASE["factions"][c]["aa_nodes"]) for c in [" 苏军", " 盟军"]}

# 预计算（不随 sweep 变化）：候选指挥节点、各候选最短路距离
PRE = {}
for camp in [" 苏军", " 盟军"]:
    cands, scores = Q1M.command_candidates(camp, k=6)
    dists = {z: Q1M.dijkstra(ADJ, z, MAIN) for z in cands}
    PRE[camp] = {"cands": cands, "scores": scores, "dists": dists}


def jaccard(a, b):
    sa, sb = set(a), set(b)
    return len(sa & sb) / len(sa | sb) if (sa | sb) else 1.0


```

```
def solve_at(camp, patches):
    """ 对 Q1M 模块全局量打补丁后逐候选重解部署 MIP（补丁用后即还原）。
    返回最优候选的 (z, phi, deploy, 原生元数据)。"""
    saved = {k: getattr(Q1M, k) for k in patches}
    try:
        for k, v in patches.items():
            setattr(Q1M, k, v)
        own, enemy, cov = Q1M.build_coverage(camp)   # 覆盖系数随补丁重建
        best = None
        for z in PRE[camp]["cands"]:
            dist_to_cmd = {n: PRE[camp]["dists"][z].get(n, Q1M.D_MAX)
                           for n in own}
            res, meta = Q1M.solve_deployment(camp, z, own, enemy, cov,
                                             dist_to_cmd)
            if res.status in ("optimal", "feasible"):
                if best is None or res.objective_value > best["phi"]:
                    deploy = {}
                    for u in UNIT_TYPES:
                        for n in own:
                            v = res.variable_values.get(f"x_{u}_{n}", 0.0)
                            if v and v > 1e-6:
                                deploy[(u, n)] = int(round(v))
                    best = {"z": z, "phi": res.objective_value,
                            "deploy": deploy, "status": res.status,
                            "gap": res.mip_gap, "t": round(res.solve_time, 3)}
        assert best is not None, f"{camp} 在补丁 {patches} 下无可行解"
        return best
    finally:
        for k, v in saved.items():
            setattr(Q1M, k, v)


def stability_interval(xs, stable, base_x):
    """ 包含基线点的最长连续稳定区间。"""
    i = xs.index(base_x)
    lo = i
    while lo > 0 and stable[lo - 1]:
        lo -= 1
    hi = i
    while hi < len(xs) - 1 and stable[hi + 1]:
        hi += 1
    return [xs[lo], xs[hi]]


def solver_meta(rows):
    """ 聚合各扫点求解器原生元数据。"""
    sts = [r["status"] for r in rows]
    return {"all_points_status_optimal": all(s == "optimal" for s in sts),
            "statuses": sorted(set(sts)),
            "max_MIPGap": max(r["gap"] for r in rows),
            "max_solve_time_s": max(r["t"] for r in rows),
            "source": " 求解器原生元数据（solver_backend.solve_mip 透传）"}


# ============ ② A-02 直射距离 sweep ============
def sweep_a02():
    ds = [1.0, 1.5, 2.0, 2.5, 3.0]
    out = {}
    for camp in [" 苏军", " 盟军"]:
        rows = []
        for d in ds:
            s = d / 2.0  # 基线：轻 1.5/中重 2.0 → 等比缩放，d=2.0 复现基线
```

```
            rng = dict(CD.RANGE)
            rng.update({" 轻型": 1.5 * s, " 中型": 2.0 * s, " 重型": 2.0 * s})
            best = solve_at(camp, {"RANGE": rng})
            rows.append({
                "d_km": d, "scale": round(s, 3),
                "ranges_km": {" 轻型": round(1.5 * s, 3),
                              " 中型": round(2.0 * s, 3),
                              " 重型": round(2.0 * s, 3)},
                "phi": round(best["phi"], 5),
                "command_node": best["z"],
                "flipped": best["z"] != BASE_CMD[camp],
                "deploy_jaccard": round(jaccard(best["deploy"],
                                                BASE_DEPLOY[camp]), 4),
                "deploy_node_count": len(set(n for (_, n) in best["deploy"])),
                "status": best["status"], "gap": best["gap"], "t": best["t"],
            })
            print(f"  [A-02 {camp}] d={d}km Φ={best['phi']:.4f} "
                  f"cmd={best['z']} flip={rows[-1]['flipped']} "
                  f"J={rows[-1]['deploy_jaccard']} ({best['status']}, "
                  f"gap={best['gap']}, {best['t']}s)")
        stable = [(not r["flipped"]) and r["deploy_jaccard"] >= 0.8
                  for r in rows]
        flips = sum(r["flipped"] for r in rows)
        out[camp] = {
            "parameter": "A-02 坦克直射距离（等比缩放：轻型 =1.5×(d/2)，中/重型 =2.0×(d/2)）",
            "baseline_d_km": 2.0,
            "base_phi": BASE_PHI[camp],
            "base_command_node": BASE_CMD[camp],
            "points": rows,
            "flip_count": flips,
            "stability_interval_km": stability_interval(ds, stable, 2.0),
            "stability_criterion": " 指挥节点不翻转且部署结构 Jaccard 相似度 ≥0.8",
            "conclusion": "",
            "solver_meta": solver_meta(rows),
        }
        lo, hi = out[camp]["stability_interval_km"]
        out[camp]["conclusion"] = (
            f"d ∈ [{lo},{hi}]km 内指挥节点 0 翻转且部署结构相似度 ≥0.8，"
            f" 全扫描 [{ds[0]},{ds[-1]}]km 翻转计数 ={flips}，推荐部署稳健")
    return out


# ============ ③ A-03 防空半径 sweep ============
def sweep_a03():
    rs = [2.0, 2.5, 3.0, 3.5, 4.0]
    out = {}
    for camp in [" 苏军", " 盟军"]:
        deploy = BASE_DEPLOY[camp]
        own = [n for n in POS if CAMP[n] == camp]
        phi_mip = BASE_PHI[camp]
        S_max = sum(FORCES[camp][u] * SURV[u] for u in UNIT_TYPES)
        rows = []
        for r in rs:
            saved = Q1M.R_AA
            Q1M.R_AA = r
            try:
                aa, aa_res = Q1M.place_aa(camp, deploy, own)
            finally:
                Q1M.R_AA = saved
            covered = set()
            for p in aa:
                for n in own:
```

```
                    if Q1M.euclid(POS, p, n) <= r:
                        covered.add(n)
            bonus = sum(DEF[n] * SURV[u] * deploy.get((u, n), 0)
                        * CD.AA_SURV_BONUS
                        for u in UNIT_TYPES for n in covered)
            aa_set = set(aa)
            sym = len(aa_set ^ BASE_AA[camp])
            rows.append({
                "r_km": r,
                "S_bonus": round(bonus, 1),
                "phi_with_aa": round(phi_mip + 0.3 * bonus / S_max, 5),
                "aa_nodes": sorted(aa),
                "aa_changed_count": sym,
                "aa_jaccard": round(jaccard(aa_set, BASE_AA[camp]), 4),
                "covered_deploy_nodes": len(covered),
                "status": aa_res.status, "gap": aa_res.mip_gap,
                "t": round(aa_res.solve_time, 3),
            })
            print(f"  [A-03 {camp}] r={r}km S_bonus={bonus:.1f} "
                  f" 防空节点变化 ={sym} J={rows[-1]['aa_jaccard']} "
                  f"({aa_res.status}, gap={aa_res.mip_gap})")
        stable = [row["aa_jaccard"] >= 0.8 for row in rows]
        out[camp] = {
            "parameter": "A-03 防空炮阵地覆盖半径 R_aa（Stage B 防空重布置 + 生存加成重算）",
            "baseline_r_km": 3.0,
            "phi_mip_base": phi_mip,
            "phi_with_aa_note": "phi_with_aa = Φ_MIP + 0.3×S_bonus/S_max
            ↪ （含防空生存加成的效能口径）",
            "points": rows,
            "stability_interval_km": stability_interval(rs, stable, 3.0),
            "stability_criterion": " 防空节点集与基线 Jaccard 相似度 ≥0.8",
            "conclusion": "",
            "solver_meta": solver_meta(rows),
        }
        lo, hi = out[camp]["stability_interval_km"]
        max_chg = max(row["aa_changed_count"] for row in rows)
        out[camp]["conclusion"] = (
            f"r ∈ [{lo},{hi}]km 内防空节点布局与基线相似度 ≥0.8，"
            f" 全扫描 [{rs[0]},{rs[-1]}]km 最大节点变化数 ={max_chg}，防空配置稳健")
    return out


# ============ ④ A-04 空中上限 sweep（Q1 + Q2 传播） ============
def sweep_a04():
    alphas = [0.05, 0.10, 0.15, 0.20]
    q1_part = {}
    deploys_by_alpha = {}
    for camp in [" 苏军", " 盟军"]:
        rows = []
        for a in alphas:
            best = solve_at(camp, {"AIR_CAP_RATIO": a})
            air = sum(v for (u, _), v in best["deploy"].items() if u == " 空中")
            air_nodes = sorted({n for (u, n) in best["deploy"] if u == " 空中"})
            rows.append({
                "alpha": a,
                "air_cap_units": int(FORCES[camp][" 空中"] * a),
                "air_deployed": air,
                "air_nodes": air_nodes,
                "phi": round(best["phi"], 5),
                "command_node": best["z"],
                "flipped": best["z"] != BASE_CMD[camp],
                "deploy_jaccard": round(jaccard(best["deploy"],
```

```
                                            BASE_DEPLOY[camp]), 4),
                "status": best["status"], "gap": best["gap"], "t": best["t"],
            })
            deploys_by_alpha.setdefault(a, {})[camp] = best["deploy"]
            print(f"  [A-04 {camp}] α={a:.0%} 空中部署 ={air} "
                  f"Φ={best['phi']:.4f} cmd={best['z']} ({best['status']})")
        flips = sum(r["flipped"] for r in rows)
        q1_part[camp] = {
            "parameter": "A-04 空中单位部署上限 α（force_ 空中 约束 rhs=N_ 空中 ×α）",
            "baseline_alpha": 0.10,
            "base_command_node": BASE_CMD[camp],
            "points": rows,
            "flip_count": flips,
            "stability_interval": stability_interval(
                alphas, [not r["flipped"] for r in rows], 0.10),
            "stability_criterion": " 指挥节点不翻转",
            "conclusion": "",
            "solver_meta": solver_meta(rows),
        }
        lo, hi = q1_part[camp]["stability_interval"]
        q1_part[camp]["conclusion"] = (
            f"α ∈ [{lo:.0%},{hi:.0%}] 内指挥节点 0 翻转，空中部署随上限单调不减，"
            f" 定性结论（空中单位少量集中部署）稳定")
    # Q2 传播：各 α 下部署 → 需求 → 补给 TC
    prop_rows = []
    base_tc = {c: Q2_BASE["factions"][c]["supply_optimization"]["TC_excl_labor"]
               for c in [" 苏军", " 盟军"]}
    for a in alphas:
        dep = {}
        for camp in [" 苏军", " 盟军"]:
            dep[camp] = {}
            for (u, n), v in deploys_by_alpha[a][camp].items():
                if n in MAIN:
                    dep[camp].setdefault(n, {})
                    dep[camp][n][u] = dep[camp][n].get(u, 0) + v
        demand = Q2M.compute_demand(dep)
        row = {"alpha": a}
        for camp in [" 苏军", " 盟军"]:
            depots = Q2M.select_depots(camp, Q1_BASE)
            r = Q2M.solve_supply(camp, depots, demand)
            row[f"TC_{camp}"] = r["TC_excl_labor"]
            row[f"delta_pct_{camp}"] = round(
                (r["TC_excl_labor"] - base_tc[camp]) / base_tc[camp] * 100, 3)
            row[f"status_{camp}"] = r["status"]
        prop_rows.append(row)
        print(f"  [A-04→Q2] α={a:.0%} TC 苏 ={row['TC_ 苏军']} "
              f"({row['delta_pct_ 苏军']:+.3f}%) TC 盟 ={row['TC_ 盟军']} "
              f"({row['delta_pct_ 盟军']:+.3f}%)")
    max_abs = max(abs(r[f"delta_pct_{c}"]) for r in prop_rows
                  for c in [" 苏军", " 盟军"])
    q2_part = {
        "parameter": "A-04 空中上限 α 经部署 → 需求传播至 Q2 补给成本",
        "base_TC_excl_labor": base_tc,
        "points": prop_rows,
        "max_abs_delta_pct": max_abs,
        "conclusion": (
            f"α ∈ [5%,20%] 全扫描下 Q2 补给成本最大变化 {max_abs:.3f}%，"
            " 传播影响可忽略，Q2 结论对 A-04 稳健"),
    }
    return {"q1": q1_part, "q2_propagation": q2_part}


```

```
# ============ ⑤ A-07 0.5 万人粒度验证算例 ============
def verify_a07():
    out = {}
    for camp in [" 苏军", " 盟军"]:
        fine = {c: dict(FORCES[c]) for c in FORCES}
        fine[camp][" 动员兵"] = FORCES[camp][" 动员兵"] * 2  # 0.5 万人/单位
        best = solve_at(camp, {"FORCES": fine})
        # 动员兵聚合回万人粒度对比
        base_inf = {n: v for (u, n), v in BASE_DEPLOY[camp].items()
                    if u == " 动员兵"}
        fine_inf = {}
        for (u, n), v in best["deploy"].items():
            if u == " 动员兵":
                fine_inf[n] = fine_inf.get(n, 0) + v / 2.0
        l1 = sum(abs(fine_inf.get(n, 0) - base_inf.get(n, 0))
                 for n in set(base_inf) | set(fine_inf))
        eq_base = {k: v for k, v in BASE_DEPLOY[camp].items() if k[0] != " 动员兵"}
        eq_fine = {k: v for k, v in best["deploy"].items() if k[0] != " 动员兵"}
        out[camp] = {
            "granularity": "0.5 万人/单位（动员兵总量 ×2，其余装备粒度不变）",
            "phi": round(best["phi"], 5),
            "base_phi": BASE_PHI[camp],
            "phi_delta": round(best["phi"] - BASE_PHI[camp], 5),
            "command_node": best["z"],
            "command_flipped": best["z"] != BASE_CMD[camp],
            "equip_jaccard": round(jaccard(eq_fine, eq_base), 4),
            "infantry_L1_wanren": round(l1, 2),
            "deploy_node_count": len(set(n for (_, n) in best["deploy"])),
            "status": best["status"], "gap": best["gap"], "t": best["t"],
        }
        print(f"  [A-07 {camp}] 0.5 粒度 Φ={best['phi']:.4f} "
              f"(Δ={best['phi'] - BASE_PHI[camp]:+.4f}) cmd={best['z']} "
              f" 装备 J={out[camp]['equip_jaccard']} 动员兵 L1={l1:.1f} 万人 "
              f"({best['status']}, gap={best['gap']})")
    stable = all((not v["command_flipped"]) and v["equip_jaccard"] >= 0.8
                 for v in out.values())
    max_phi_delta = max(abs(v["phi_delta"]) for v in out.values())
    out["max_phi_delta"] = max_phi_delta
    out["conclusion"] = (
        "0.5 万人更细粒度下双方指挥节点均不翻转、装备部署结构 Jaccard≥0.8、"
        f"Φ 最大变化 {max_phi_delta}——万人粒度为建模粒度选择（避免 10^6 值域病态），"
        " 部署结构对粒度稳健" if stable else
        "0.5 万人粒度下结构发生变化，须在论文披露粒度敏感性")
    out["structure_stable"] = stable
    return out


# ============ ① A-01 基线 vs Stackelberg 对照结构化 ============
def comparison_a01():
    """ 主口径 = 含人力 TC_total。
    TC_worst = 基线 TC_total + extra_loss；delta_pct 取 问题 2_ 求解.py
    重算的原生输出（苏 +50.83% / 盟 +48.94%）。"""
    out = {}
    for camp in [" 苏军", " 盟军"]:
        f = Q2_BASE["factions"][camp]
        tc_base = f["supply_optimization"]["TC_total"]
        worst = f["stackelberg_worst_case"]
        assert abs(worst["TC_baseline"] - tc_base) < 0.5, \
            f"{camp} Stackelberg 基线与 TC_total 不一致（口径一致性断言）"
        out[camp] = {
            "parameter": "A-01 破交拦截概率（内生未知参数：静态risk_e基线=主模型输入，
            ↪  Stackelberg最坏情形=鲁棒性成本量化）",
```

```
            "caliber": worst["caliber"],
            "TC_baseline_total": worst["TC_baseline"],
            "TC_worst_total": worst["TC_worst"],
            "extra_loss": worst["extra_loss"],
            "delta_pct": worst["delta_pct"],
            "delta_pct_excl_labor_reference": worst["delta_pct_excl_labor_reference"],
            "hit_routes": worst["hit_routes"],
            "enemy_interdict_budget": CD.ENEMY_INTERDICT_BUDGET,
            "note": (" 对照口径为 TC_total（含人力成本，主口径）: "
                     "TC_worst= 基线 TC_total+extra_loss（破交追加损失仅作用于货物损失项，"
                     " 人力成本不变）；敌方将破交预算集中投向流量 × 货值/护航强度最大的"
                     "Top-5 路径（Stackelberg follower 最优响应）。"),
            "source": "q2_result.stackelberg_worst_case（含人力口径）",
        }
        print(f"  [A-01 {camp}] 基线 TC={tc_base} 最坏 TC={worst['TC_worst']} "
              f"Δ={worst['delta_pct']:+.2f}%（含人力口径）")
    return out


# ============ 图表 ============
def make_figures(a02, a03, a04, a01):
    setup_style()
    os.makedirs(FIGDIR, exist_ok=True)
    camps = [" 苏军", " 盟军"]
    # fig11 A-02 直射距离 sweep
    fig, ax = plt.subplots(figsize=(6, 4))
    for i, c in enumerate(camps):
        xs = [p["d_km"] for p in a02[c]["points"]]
        ys = [p["phi"] for p in a02[c]["points"]]
        ax.plot(xs, ys, color=COLORS["palette"][i],
                linestyle=LINE_STYLES[0], marker=MARKERS[i],
                label=f"{c} Φ")
        for p in a02[c]["points"]:
            if p["flipped"]:
                ax.plot(p["d_km"], p["phi"], marker="x", ms=12,
                        color=COLORS["quaternary"])
    lo = min(a02[c]["stability_interval_km"][0] for c in camps)
    hi = max(a02[c]["stability_interval_km"][1] for c in camps)
    ax.axvspan(lo, hi, color=COLORS["tertiary"], alpha=0.12,
               label=f" 双方稳定区 [{lo},{hi}]km")
    ax.axvline(2.0, color="gray", ls=":", lw=1)
    ax.set_xlabel(" 坦克直射距离 d (km)")
    ax.set_ylabel(" 综合效能 Φ")
    ax.set_title("A-02 直射距离参数化 sweep（Q1 部署重解）")
    ax.legend()
    save_figure(fig, os.path.join(FIGDIR, "fig11_q1_direct_range_sweep"))
    plt.close(fig)
    # fig12 A-03 防空半径 sweep
    fig, ax = plt.subplots(figsize=(6, 4))
    ax2 = ax.twinx()
    for i, c in enumerate(camps):
        xs = [p["r_km"] for p in a03[c]["points"]]
        ys = [p["phi_with_aa"] for p in a03[c]["points"]]
        ax.plot(xs, ys, color=COLORS["palette"][i], marker=MARKERS[i],
                label=f"{c} Φ(含防空)")
        ax2.bar([x + (0.08 if i else -0.08) for x in xs],
                [p["aa_changed_count"] for p in a03[c]["points"]],
                width=0.16, alpha=0.35, color=COLORS["palette"][i],
                label=f"{c} 防空节点变化数")
    ax.axvline(3.0, color="gray", ls=":", lw=1)
    ax.set_xlabel(" 防空覆盖半径 r (km)")
    ax.set_ylabel("Φ（含防空生存加成）")
```

```
    ax2.set_ylabel(" 防空节点变化数（vs 基线）")
    ax.set_title("A-03 防空半径参数化 sweep（Stage B 重布置）")
    h1, l1 = ax.get_legend_handles_labels()
    h2, l2 = ax2.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, fontsize=8)
    save_figure(fig, os.path.join(FIGDIR, "fig12_q1_aa_radius_sweep"))
    plt.close(fig)
    # fig13 A-01 基线 vs Stackelberg
    fig, ax = plt.subplots(figsize=(6, 4))
    x = np.arange(2)
    w = 0.35
    b = [a01[c]["TC_baseline_total"] for c in camps]
    wst = [a01[c]["TC_worst_total"] for c in camps]
    ax.bar(x - w / 2, b, w, color=COLORS["primary"], label=" 基线（静态 risk_e）")
    ax.bar(x + w / 2, wst, w, color=COLORS["quaternary"],
           label="Stackelberg 最坏情形")
    for i, c in enumerate(camps):
        ax.annotate(f"+{a01[c]['delta_pct']:.1f}%",
                    (x[i] + w / 2, wst[i]), ha="center", va="bottom",
                    fontsize=9, color=COLORS["quaternary"])
    ax.set_xticks(x)
    ax.set_xticklabels(camps)
    ax.set_ylabel(" 后勤总成本 TC (元/日)")
    ax.set_title(" 基线情形与对抗最坏情形后勤总成本对照（含人力口径）")
    ax.legend()
    save_figure(fig, os.path.join(FIGDIR, "fig13_q2_a01_baseline_vs_stackelberg"))
    plt.close(fig)
    # fig14 A-04 空中上限 sweep
    fig, ax = plt.subplots(figsize=(6, 4))
    ax2 = ax.twinx()
    for i, c in enumerate(camps):
        xs = [p["alpha"] * 100 for p in a04["q1"][c]["points"]]
        ax.plot(xs, [p["phi"] for p in a04["q1"][c]["points"]],
                color=COLORS["palette"][i], marker=MARKERS[i],
                label=f"{c} Φ")
        ax2.plot(xs, [p["air_deployed"] for p in a04["q1"][c]["points"]],
                 color=COLORS["palette"][i], ls="--", marker=MARKERS[i + 2],
                 alpha=0.7, label=f"{c} 空中部署数")
    ax.axvline(10, color="gray", ls=":", lw=1)
    ax.set_xlabel(" 空中单位部署上限 α (%)")
    ax.set_ylabel(" 综合效能 Φ")
    ax2.set_ylabel(" 空中单位部署数（架）")
    ax.set_title("A-04 空中上限参数化 sweep（Q1 重解）")
    h1, l1 = ax.get_legend_handles_labels()
    h2, l2 = ax2.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, fontsize=8)
    save_figure(fig, os.path.join(FIGDIR, "fig14_q1_air_cap_sweep"))
    plt.close(fig)
    print("  fig11~fig14 已生成（PDF+PNG）")


def main():
    t0 = time.time()
    print("== 基线复现自检 ==")
    for camp in [" 苏军", " 盟军"]:
        b = solve_at(camp, {})
        assert b["z"] == BASE_CMD[camp], \
            f"{camp} 基线复现失败：cmd {b['z']} != {BASE_CMD[camp]}"
        assert abs(b["phi"] - BASE_PHI[camp]) < 1e-3, \
            f"{camp} 基线 Φ 复现失败：{b['phi']} vs {BASE_PHI[camp]}"
        print(f"  [{camp}] 基线复现 ✓ cmd={b['z']} Φ={b['phi']:.5f}")
    # Q2 基线复现自检
```

```
    _, dep0 = Q2M.load_q1_deploy()
    demand0 = Q2M.compute_demand(dep0)
    for camp in [" 苏军", " 盟军"]:
        depots = Q2M.select_depots(camp, Q1_BASE)
        r0 = Q2M.solve_supply(camp, depots, demand0)
        base_tc = Q2_BASE["factions"][camp]["supply_optimization"]["TC_excl_labor"]
        assert abs(r0["TC_excl_labor"] - base_tc) < 1.0, \
            f"{camp} Q2 基线复现失败：{r0['TC_excl_labor']} vs {base_tc}"
        print(f"  [{camp}] Q2 基线复现 ✓ TC_excl_labor={r0['TC_excl_labor']}")

    print("== A-02 直射距离 sweep ==")
    a02 = sweep_a02()
    print("== A-03 防空半径 sweep ==")
    a03 = sweep_a03()
    print("== A-04 空中上限 sweep ==")
    a04 = sweep_a04()
    print("== A-07 0.5 万人粒度验证 ==")
    a07 = verify_a07()
    print("== A-01 基线 vs Stackelberg 结构化登记 ==")
    a01 = comparison_a01()
    print("== 图表 ==")
    make_figures(a02, a03, a04, a01)

    out = {
        "metadata": {
            "purpose": " 显式未知参数化 sweep 扫描（不重建模型结构）",
            "date": "2026-08-13",
            "solver_backend": "pulp + HiGHS/CBC（开源，solver_backend.solve_mip）",
            "baseline_reproduction_check": "苏军/盟军 Q1 指挥节点与Φ、Q2 TC 均复现基线（assert
            ↪  通过）",
        },
        "a01_comparison": a01,
        "a02": a02,
        "a03": a03,
        "a04": a04,
        "a07": a07,
        "figures": ["fig11_q1_direct_range_sweep", "fig12_q1_aa_radius_sweep",
                    "fig13_q2_a01_baseline_vs_stackelberg", "fig14_q1_air_cap_sweep"],
        "total_time_s": round(time.time() - t0, 1),
    }
    os.makedirs(RESULTS, exist_ok=True)
    with open(os.path.join(RESULTS, " 参数化扫描结果.json"), "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)
    print(f" 已写入 参数化扫描结果.json，总耗时 {out['total_time_s']}s")


if __name__ == "__main__":
    main()
```

（十一） 图表生成.py（全部 14 张图表）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
图表生成.py —全部图表（PDF+PNG 双格式，300dpi，统一走 visualization_style）
图 1 战场节点与 Q1 部署分布图（双方）
图 2 Q1 部署构成堆叠柱状图
图 3 Q2 补给路径网络图（苏军示例 + 盟军）
图 4 Q2 成本分层构成柱状图
图 5 Q2 灵敏度 Tornado 图
```

```
图 6 Q2 护航策略转换临界点图
图 7 Q3 苏军主攻方向与夺控序列图
图 8 Q3 盟军转移生存概率对比（MC 分布）
图 9 算法总体流程图
图 10 Q3 生存概率-风险缩放敏感性折线图
"""
import json
import os
import sys

import matplotlib.patches as mpatches
import matplotlib.pyplot as plt
import numpy as np

from visualization_style import (setup_style, save_figure, COLORS, COLORMAPS,
                                 plot_tornado, SINGLE_COLUMN, DOUBLE_COLUMN, SQUARE)
from common_data import (load_graph, connected_components, cross_faction_edges,
                         TARGET_NODES, UNIT_TYPES)

POS, CAMP, DEF, EDGE_LEN, ADJ, DEGREE = load_graph()
NODES = list(POS.keys())
MAIN = set(max(connected_components(NODES, ADJ), key=len))
HERE = os.path.dirname(os.path.abspath(__file__))   # project 目录
RESULTS = os.path.join(HERE, "results")
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)

with open(os.path.join(RESULTS, "q1_result.json"), encoding="utf-8") as f:
    Q1 = json.load(f)
with open(os.path.join(RESULTS, "q2_result.json"), encoding="utf-8") as f:
    Q2 = json.load(f)
with open(os.path.join(RESULTS, "q3_result.json"), encoding="utf-8") as f:
    Q3 = json.load(f)
with open(os.path.join(RESULTS, "validation_result.json"), encoding="utf-8") as f:
    VAL = json.load(f)

setup_style()


def base_map(ax):
    """ 底图：节点按阵营着色，防御强度为半径。"""
    for n in NODES:
        x, y = POS[n]
        c = "#d62728" if CAMP[n] == " 苏军" else "#1f77b4"
        ax.scatter(x, y, s=12 + 40 * DEF[n], c=c, alpha=0.45, edgecolors="none")
    for (a, b), L in EDGE_LEN.items():
        ax.plot([POS[a][0], POS[b][0]], [POS[a][1], POS[b][1]],
                color="gray", linewidth=0.3, alpha=0.35)
    ax.set_aspect("equal")
    ax.set_xlabel("X (km)")
    ax.set_ylabel("Y (km)")


# ---------- 图 1 部署分布 ----------
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, camp in zip(axes, [" 苏军", " 盟军"]):
    base_map(ax)
    info = Q1["factions"][camp]
    # 部署节点（按数量）
    dep = {}
    for key, v in info["deploy"].items():
        n = int(key.split("@")[1])
```

```
        dep[n] = dep.get(n, 0) + v
    for n, v in dep.items():
        ax.scatter(*POS[n], s=60 + v * 0.05, facecolors="none",
                   edgecolors="black", linewidths=1.6, marker="o")
    # 指挥节点
    z = info["command_node"]
    ax.scatter(*POS[z], s=260, marker="*", facecolors="gold",
               edgecolors="black", linewidths=1.2, zorder=5)
    ax.annotate(f" 指挥 {z}", POS[z], fontsize=8, xytext=(6, 6),
                textcoords="offset points")
    for b in info["command_backups"]:
        ax.scatter(*POS[b], s=110, marker="*", facecolors="lightyellow",
                   edgecolors="gray", zorder=4)
    # 防空
    for p in info["aa_nodes"]:
        ax.scatter(*POS[p], s=90, marker="^", facecolors="none",
                   edgecolors="green", linewidths=1.4, zorder=4)
    ax.set_title(f"{camp} 最优部署（★指挥节点 △防空 圈 = 部署量）")
handles = [mpatches.Patch(color="#d62728", alpha=0.5, label=" 苏军节点"),
           mpatches.Patch(color="#1f77b4", alpha=0.5, label=" 盟军节点"),
           plt.Line2D([], [], marker="*", color="gold", linestyle="none",
                      markersize=12, label=" 指挥节点"),
           plt.Line2D([], [], marker="^", color="green", linestyle="none",
                      markersize=8, label=" 防空据点")]
axes[1].legend(handles=handles, loc="upper right", fontsize=8)
save_figure(fig, os.path.join(FIG, "fig1_deployment_map"))
plt.close(fig)

# ---------- 图 2 部署构成堆叠柱 ----------
fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
camps = [" 苏军", " 盟军"]
bottom = np.zeros(2)
unit_colors = {" 动员兵": "#7f7f7f", " 轻型": "#2ca02c", " 中型": "#ff7f0e",
               " 重型": "#d62728", " 火炮": "#9467bd", " 空中": "#8c564b"}
for u in UNIT_TYPES:
    vals = []
    for camp in camps:
        s = sum(v for key, v in Q1["factions"][camp]["deploy"].items()
                if key.startswith(u + "@"))
        vals.append(s)
    ax.bar(camps, vals, bottom=bottom, label=u, color=unit_colors[u],
           edgecolor="black", linewidth=0.6)
    bottom += np.array(vals, dtype=float)
ax.set_ylabel(" 部署数量（动员兵 = 万人，其余 = 辆/门/架）")
ax.set_title(" 问题 1: 双方部署构成（空中为 ≤10% 上限内部署）")
ax.legend(fontsize=8)
save_figure(fig, os.path.join(FIG, "fig2_deployment_composition"))
plt.close(fig)

# ---------- 图 3 补给路径网络 ----------
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, camp in zip(axes, camps):
    base_map(ax)
    info = Q2["factions"][camp]
    for dp in info["depots"]:
        ax.scatter(*POS[dp], s=200, marker="s", facecolors="gold",
                   edgecolors="black", zorder=5)
        ax.annotate(f" 仓 {dp}", POS[dp], fontsize=7, xytext=(4, 4),
                    textcoords="offset points")
    for r in info["supply_optimization"]["routes"]:
        p = r["path"]
        xs = [POS[n][0] for n in p]; ys = [POS[n][1] for n in p]
```

```
        lw = 0.8 + r["flow_t_day"] * 0.06
        style = "-" if r["escort"] else "--"
        color = "#2ca02c" if r["escort"] else "#ff7f0e"
        ax.plot(xs, ys, linestyle=style, linewidth=lw, color=color, alpha=0.85)
    ax.set_title(f"{camp} 补给网络（实线 = 护航 虚线 = 无护航 宽 = 流量）")
handles = [plt.Line2D([], [], color="#2ca02c", linewidth=2, label=" 护航路径"),
           plt.Line2D([], [], color="#ff7f0e", linewidth=2, linestyle="--",
                      label=" 无护航路径"),
           plt.Line2D([], [], marker="s", color="gold", linestyle="none",
                      markersize=10, label=" 仓库")]
axes[1].legend(handles=handles, loc="upper right", fontsize=8)
save_figure(fig, os.path.join(FIG, "fig3_supply_routes"))
plt.close(fig)

# ---------- 图 4 成本分层 ----------
fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
cats = [" 运输", " 护航", " 破交损失", " 人力"]
keys = ["C_trans", "C_escort", "C_loss", "C_labor"]
x = np.arange(len(cats)); wbar = 0.38
for i, camp in enumerate(camps):
    so = Q2["factions"][camp]["supply_optimization"]
    vals = [so[k] for k in keys]
    ax.bar(x + (i - 0.5) * wbar, vals, wbar, label=camp,
           color=COLORS["palette"][i], edgecolor="black", linewidth=0.6)
    for j, v in enumerate(vals):
        ax.text(x[j] + (i - 0.5) * wbar, v, f"{v:.0f}", ha="center",
                va="bottom", fontsize=7)
ax.set_xticks(x); ax.set_xticklabels(cats)
ax.set_ylabel(" 成本（元/日）")
ax.set_title(" 问题 2: 后勤成本分层构成")
ax.legend()
save_figure(fig, os.path.join(FIG, "fig4_cost_decomposition"))
plt.close(fig)

# ---------- 图 5 Tornado ----------
scan = VAL["sensitivity_q2"]["scans"]
params, sens = [], []
for pname, rows in scan.items():
    # 取最接近 ±20% 的扰动点（各参数扫描均含 0.8/1.2 点）
    lo_r = min(rows, key=lambda r: abs(r["scale"] - 0.8))
    hi_r = min(rows, key=lambda r: abs(r["scale"] - 1.2))
    params.append(pname + "(苏军)")
    sens.append((hi_r["TC_苏军"] - lo_r["TC_苏军"]) / VAL["sensitivity_q2"]["base_TC"]["苏军"] *
    ↪  100)
    params.append(pname + "(盟军)")
    sens.append((hi_r["TC_盟军"] - lo_r["TC_盟军"]) / VAL["sensitivity_q2"]["base_TC"]["盟军"] *
    ↪  100)
plot_tornado(params, sens, xlabel=" 目标值变化（±20% 参数扰动，%）",
             title=" 问题 2: 后勤总成本灵敏度（Tornado）",
             filename=os.path.join(FIG, "fig5_tornado_q2"))

# ---------- 图 6 护航临界点 ----------
fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
rows = VAL["q2_escort_critical"]["scan"]
xs = [r["risk_scale"] for r in rows]
ax.plot(xs, [r["TC_free"] for r in rows], "o-", color=COLORS["primary"],
        label=" 优化护航（模型最优）")
ax.plot(xs, [r["TC_no_escort"] for r in rows], "s--", color=COLORS["quaternary"],
        label=" 强制无护航")
crit = VAL["q2_escort_critical"]["critical_risk_scale"]
if crit:
    ax.axvline(crit, color="gray", linestyle=":", linewidth=1.2)
```

```
    ax.annotate(f" 护航策略临界点\nrisk 缩放 ≈{crit}", (crit, ax.get_ylim()[0]),
                fontsize=8, xytext=(8, 20), textcoords="offset points")
ax.set_xlabel(" 破交风险缩放系数")
ax.set_ylabel(" 双方合计后勤成本（元/日，不含人力）")
ax.set_title(" 问题 2：护航策略转换临界点")
ax.legend()
save_figure(fig, os.path.join(FIG, "fig6_escort_critical"))
plt.close(fig)

# ---------- 图 7 进攻方案 ----------
fig, ax = plt.subplots(figsize=SQUARE)
base_map(ax)
atk = Q3["soviet_attack"]
a0, a1 = atk["main_axis"]["edge"]
ax.annotate("", xy=POS[a1], xytext=POS[a0],
            arrowprops=dict(arrowstyle="-|>", color="red", lw=3))
ax.scatter(*POS[a0], s=150, marker="s", color="red", zorder=5)
seq = atk["capture_sequence"]
for i in range(len(seq) - 1):
    ax.annotate("", xy=POS[seq[i + 1]], xytext=POS[seq[i]],
                arrowprops=dict(arrowstyle="-|>", color="darkred", lw=1.8,
                                linestyle="--"))
for i, n in enumerate(seq):
    ax.scatter(*POS[n], s=120, marker="o", facecolors="none",
               edgecolors="darkred", linewidths=2, zorder=5)
    ax.annotate(str(i + 1), POS[n], fontsize=9, fontweight="bold",
                color="darkred", xytext=(5, 5), textcoords="offset points")
ax.set_title(f" 问题 3：苏军主攻方向（{a0}→{a1}）与夺控序列")
save_figure(fig, os.path.join(FIG, "fig7_attack_plan"))
plt.close(fig)

# ---------- 图 8 生存概率对比（人员口径） ----------
fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
mc_ok = Q3["ally_withdrawal"]["comm_ok"]["monte_carlo"]
mc_lost = Q3["ally_withdrawal"]["comm_lost"]["monte_carlo"]
names = [" 通信畅通\n 集中调度", " 通信中断\n 分散预案"]
means = [mc_ok["mean_survival"], mc_lost["mean_survival"]]
errs = [[m - lo for m, lo in zip(means, [mc_ok["ci95_low"], mc_lost["ci95_low"]])],
        [hi - m for m, hi in zip(means, [mc_ok["ci95_high"], mc_lost["ci95_high"]])]]
bars = ax.bar(names, means, width=0.5, color=[COLORS["tertiary"], COLORS["quaternary"]],
              edgecolor="black", linewidth=0.8, yerr=errs, capsize=8)
for b, m in zip(bars, means):
    ax.text(b.get_x() + b.get_width() / 2, m + 0.02, f"{m:.3f}",
            ha="center", fontsize=10, fontweight="bold")
ax.set_ylabel(" 人员生存概率（MC 均值，95%CI 误差棒）")
ax.set_ylim(0, 1)
diff = Q3["ally_withdrawal"]["survival_difference"]
ax.set_title(f"问题3：通信畅通与中断两情景人员生存概率对比（畅通-中断差={diff:.3f}，
↪  各2000次MC）\n"
             "（人员加权口径，误差棒 =95% CI；装备随建制伴随，不进入生存概率加权）")
save_figure(fig, os.path.join(FIG, "fig8_survival_comparison"))
plt.close(fig)

# ---------- 图 9 算法流程图 ----------
fig, ax = plt.subplots(figsize=(9, 6.5))
ax.axis("off")
boxes = [
    (0.5, 0.94, " 数据层：附件一/二加载 → 图结构/连通分量/几何核验（坐标 =km）", "#dbe9f6"),
    (0.5, 0.82, "问题1 MIP：覆盖系数预计算 → 指挥节点Top6候选枚举 → HiGHS分支定界\n→
    ↪  部署x[u][n] + 防空Stage-B最大覆盖", "#d6e8d0"),
    (0.5, 0.66, "问题2 MIP：需求=部署×消耗率 → 仓库选址 → 每OD 3候选路径枚举\n→ flow/z 0-1规划
    ↪  （McCormick线性化护航-损失耦合）→ 成本四分层", "#fdeedc"),
```

```
    (0.5, 0.50, " 对抗层：Stackelberg 最坏情形（敌破交预算 Top-K 投放）对比基线", "#f8d7da"),
    (0.5, 0.36, "问题3：主攻方向评分 → 贪心夺控序列 → Lanchester蒙特卡洛评估\n盟军转移
    ↪ （人员口径）：人员容量分配+装备伴随+最短路+殿后 vs 分散预案 → MC人员生存概率×2000",
    ↪  "#e8d5f0"),
    (0.5, 0.20, "验证层：灵敏度±5/10/20% • 策略临界点扫描 • 多算例3场景\n• MC模型验证
    ↪ （解析vs仿真偏差<5%）• 简化项含/不含对比", "#fff3cd"),
    (0.5, 0.07, "输出：求解结果（status/MIPGap/solve_time 求解器原生元数据）+ 图表", "#e2e3e5"),
]
for x, y, text, color in boxes:
    ax.add_patch(mpatches.FancyBboxPatch((x - 0.42, y - 0.045), 0.84, 0.09,
                                         boxstyle="round,pad=0.01",
                                         facecolor=color, edgecolor="black",
                                         linewidth=1.0))
    ax.text(x, y, text, ha="center", va="center", fontsize=8.2)
for i in range(len(boxes) - 1):
    ax.annotate("", xy=(0.5, boxes[i + 1][1] + 0.05), xytext=(0.5, boxes[i][1] - 0.05),
                arrowprops=dict(arrowstyle="-|>", color="black", lw=1.3))
ax.set_title(" 算法总体流程图", fontsize=12)
save_figure(fig, os.path.join(FIG, "fig9_algorithm_flowchart"))
plt.close(fig)

# ---------- 图 10 Q3 风险敏感性 ----------
fig, ax = plt.subplots(figsize=SINGLE_COLUMN)
rows = VAL["sensitivity_q3"]
xs = [r["risk_scale"] for r in rows]
ax.plot(xs, [r["surv_comm_ok"] for r in rows], "o-", color=COLORS["tertiary"],
        label=" 通信畅通")
ax.plot(xs, [r["surv_comm_lost"] for r in rows], "s--", color=COLORS["quaternary"],
        label=" 通信中断")
ax.set_xlabel(" 拦截风险缩放系数（±20%）")
ax.set_ylabel(" 人员生存概率")
ax.set_title(" 问题 3：人员生存概率对拦截风险的敏感性")
ax.legend()
save_figure(fig, os.path.join(FIG, "fig10_q3_risk_sensitivity"))
plt.close(fig)

print(" 全部 10 张图已生成（PDF+PNG）")
```